mnogobase

Документы → гибридный поиск, граф знаний и постоянная wiki → ответы с источниками.

`mnogobase` — Python-пакет и CLI для работы с собственным корпусом документов на русском и английском. Он разбирает файлы, связывает сущности из разных источников, обновляет Markdown-wiki и позволяет сравнивать четыре способа поиска на одних данных. Текущая версия — `0.1.0`; основной интерфейс — терминал.

Ваша задача	С чего начать
Получить первый ответ	Установка → первый документ
Понять разницу между RAG, wiki и графом	Выбор режима → схема поиска
Настроить модели и параметры	Текущая конфигурация → все настройки
Обрабатывать новые файлы автоматически	Очередь и worker
Измерить качество и затраты	Оценка Ragas → измерение build/update
Изучить Hybrid RAG по шагам	Учебные ноутбуки
Работать с командами, API и чатами	Командный backend и CLI-клиент
Изменить реализацию	Архитектура → карта модулей

Содержание

1. Что это такое
2. Установка и первый запуск
3. Повседневная работа: документы, ingest, wiki build, ask и compare, status, reindex, reset, все команды, удаление документов
4. Как смотреть результаты: Neo4j, Qdrant, Obsidian, compare.jsonl, логи
5. Как это устроено внутри: конвейер, один файл, состояния этапов, модель данных, слияние сущностей, wiki, поиск, гарантии
6. Справочник настроек: все ключи, переменные окружения, типы сущностей
7. Как поменять типовые вещи: где работают модели, картинки и графики
8. Как расширять код
9. Диагностика и частые проблемы
10. Тесты
11. Развёртывание на сервере
12. Структура репозитория
13. Что дальше
14. Оценка Hybrid RAG, WikiLLM и GraphRAG
15. Фоновая обработка и воспроизводимые эксперименты
16. Учебные ноутбуки
17. Командный backend, API и чат

---

1. Что это такое

`mnogobase` берёт документы (PDF, DOCX, PPTX, XLSX, HTML, Markdown, AsciiDoc, CSV, TXT) и строит из них:

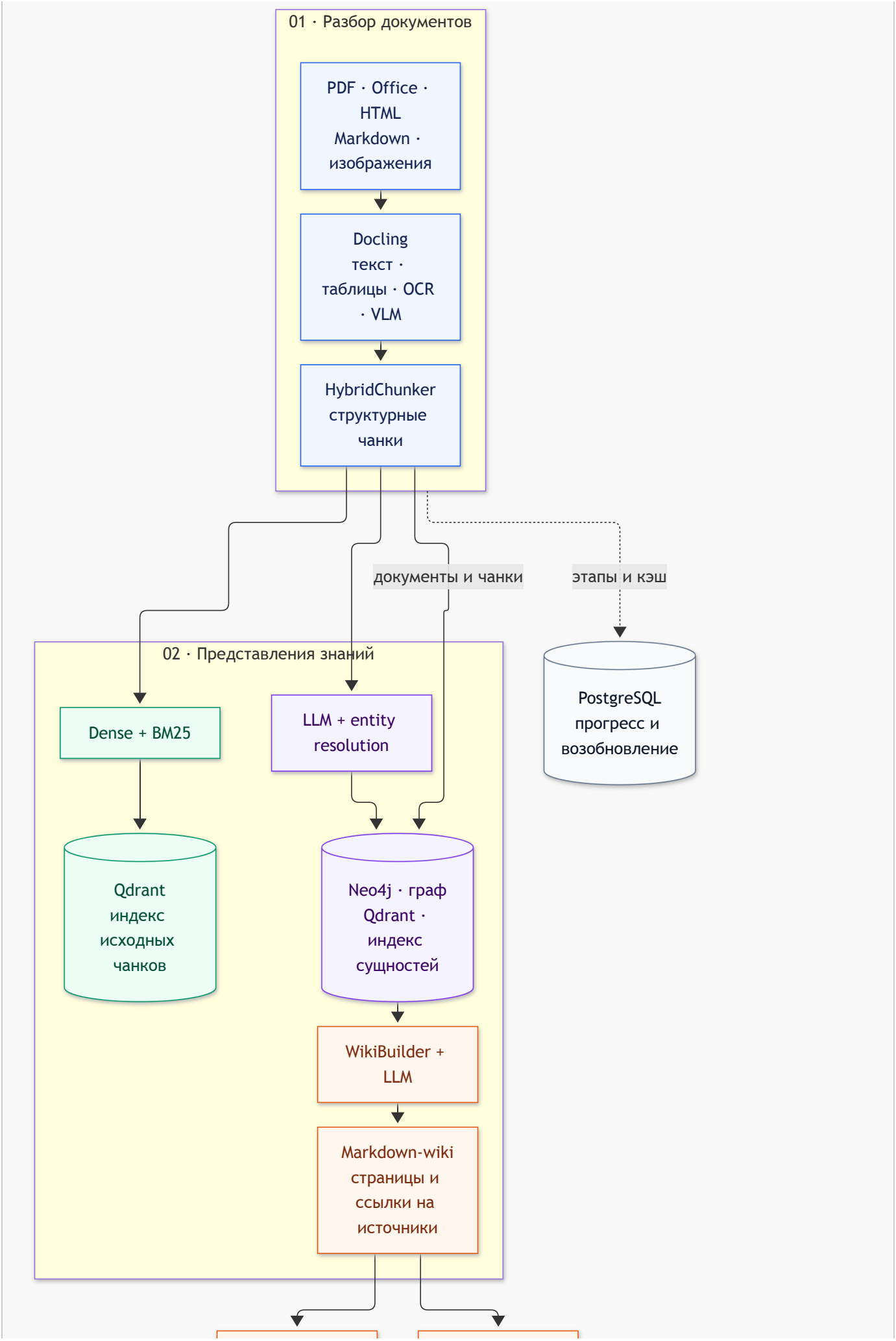
- **векторный индекс** в Qdrant: плотные векторы Qwen3-Embedding (llama.cpp server или vLLM) и разреженные BM25 для гибридного поиска;
- **граф знаний** в Neo4j: документы, чанки, сущности, связи между сущностями и чанки-доказательства для каждой связи;
- **wiki** из Markdown-страниц в духе LLM Wiki Карпаты: по странице на важную сущность, со сносками на источники. Wiki открывается в Obsidian и дописывается при каждом новом документе, а не переписывается с нуля.

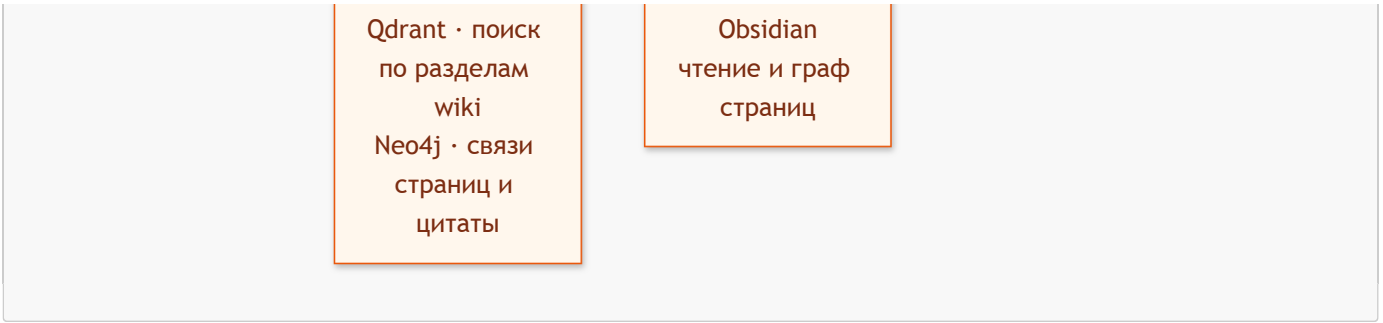
По этим слоям можно задавать вопросы в четырёх режимах (`rag`, `wiki`, `graph`, `all`) и получать ответ с нумерованными цитатами: файл, страница, id чанка или раздела wiki. Команда `compare` отвечает во всех режимах сразу и сохраняет ответы,

время и токены. Для оценки качества на датасете есть `eval run / score / report` с пятью метриками Ragas.

Обработка документов строит три связанных представления одного корпуса. На схеме показаны основные записи; подробный порядок операций и восстановления — в разделе 5. Поиск использует подготовленные данные и описан отдельно.

Для просмотра схем используйте Markdown-просмотрщик с поддержкой Mermaid.


01 · Разбор документов
PDF · Office · HTML Markdown · изображения
Docling текст · таблицы · OCR · VLM
HybridChunker структурные чанки
документы и чанки
этапы и кэш
02 · Представления знаний
Dense + BM25
LLM + entity resolution
PostgreSQL прогресс и возобновление
Qdrant индекс исходных чанков
Neo4j · граф Qdrant · индекс сущностей
WikiBuilder + LLM
Markdown-wiki страницы и ссылки на источники



Ключевые решения

Вопрос	Решение	Почему
Векторная БД	Qdrant в Docker	именованные dense и sparse векторы в одной коллекции, встроенный hybrid-поиск (RRF), payload-фильтры, есть GPU-образ
Ключевые слова	BM25 как sparse-векторы в Qdrant (`fastembed`, `Qdrant/bm25`, IDF считает Qdrant)	не нужен отдельный Elasticsearch, слияние результатов на стороне БД
Граф	Neo4j 5 Community с APOC и GDS	APOC — ограниченный обход для graph/all; GDS — отдельный PageRank и Leiden, готовые оценки помогают упорядочивать найденные связи
Извлечение графа	свой конвейер, не LightRAG/Graphiti	полный контроль над чанками Docling, wiki и поиском; три подхода сравниваются на одних данных
Разбор и чанкинг	Docling `DocumentConverter` и `HybridChunker`	структурный и токен-осознанный чанкинг, много форматов, OCR
Эмбеддер и реранкер	в `config.yaml` — Qwen3-Embedding-0.6B и Qwen3-Reranker-0.6B на llama.cpp; размерность эмбеддингов определяется автоматически	модели и бэкенды меняются через настройки; llama.cpp работает нативно на Mac или в Docker с CUDA на сервере; доступен и vLLM
LLM	один клиент для любого OpenAI-совместимого API	llama.cpp, vLLM, LM Studio, OpenRouter, OpenAI и прокси подключаются правкой конфига. Структурированный вывод через строгую JSON Schema
Состояние	PostgreSQL	файлы, этапы, ошибки, кэш извлечения, очередь wiki; на нём держатся возобновление и инкрементальность
Интерфейс	пакет и CLI на Typer	UI, API и MCP можно навесить поверх того же пакета
Язык wiki	английский по умолчанию	канонические английские имена сущностей склеивают русские и английские упоминания
Логи	structlog: консоль и JSONL-файл	формат готов для Loki/Grafana и трейсинга

> **Куда отправляются документы.** В текущем `config.yaml` LLM — `gpt-6-luna` на `https://codex.sale/v1`: извлечение, объединение сущностей, генерация wiki и ответов передают текст этому сервису. Эмбеддер, реранкер и VLM настроены локально. Все роли можно перенастроить, поэтому фактический маршрут данных определяется вашим конфигом и переменными окружения. Локальная LLM — в разделе 7.1.

1.1 Какой режим выбрать

Режим	Из чего собирается контекст	Когда пробовать	Что учитывать
`rag` — Hybrid RAG	исходные чанки: dense + BM25 → RRF → реранкер → соседние чанки	найти формулировку, число или объяснение в	реранкер и соседи управляются настройками

Режим	Из чего собирается контекст	Когда пробовать	Что учитывать
		документах	
`wiki` — WikiLLM	разделы сгенерированных wiki-страниц и цитируемые ими исходные чанки	получить обзор сущности по нескольким упоминаниям	нужна собранная wiki; не каждая сущность проходит `min_mentions`
`graph` — GraphRAG	стартовые сущности → окрестность связей → чанки-доказательства	исследовать отношения между объектами	зависит от качества извлечения и поиска стартовых сущностей
`all`	три вида контекста с долями общего бюджета и удалением повторов	проверить, помогает ли объединение на вашем вопросе	бюджет делится между режимами; результат оценивайте на своих данных

Здесь GraphRAG — собственная реализация поиска по графу. В ней нет community summaries и конвейера Microsoft GraphRAG. Все режимы используют один генератор ответа; `compare` помогает увидеть различия, а `eval` — измерить их на наборе вопросов.

1.2 Термины за минуту

Ingest — загрузка документа в базу знаний. **Чанк** — фрагмент документа с заголовками и страницами. **Dense** — поиск по смысловой близости векторов; **BM25** — поиск по словам. **RRF** объединяет позиции результатов этих двух поисков, **реранкер** заново оценивает найденные фрагменты относительно вопроса. **Entity resolution** объединяет упоминания одной сущности. **Evidence** — исходные чанки, подтверждающие связь или wiki-страницу. **Retrieval** — подбор контекста перед генерацией ответа.

---

2. Установка и первый запуск

2.1 Что нужно установить

Что	Зачем	Проверка
Docker Desktop	Qdrant и Neo4j	`docker info`
llama.cpp (`llama-server`)	эмбеддер и реранкер (и, по желанию, локальная LLM). На Mac — `brew install llama.cpp`, на Linux-сервере — Docker (раздел 11)	`llama-server --version`
uv	Python 3.12 и зависимости	`uv --version`
Obsidian (по желанию)	смотреть wiki и её граф	—

Python нужен версии **3.12** (`>=3.12,<3.13` в `pyproject.toml`); окружением `.venv` управляет `uv`. Дальнейшие shell-примеры рассчитаны на macOS/Linux. На Linux с NVIDIA используйте инструкцию сервера. В Windows `models up` по умолчанию использует Docker; для CUDA нужен Docker Desktop с WSL2, Linux-контейнерами и поддерживающим WSL2 драйвером NVIDIA. Для запуска моделей без GPU задайте `device: cpu`: команда поднимет CPU-контейнеры. Для установленного `llama-server` можно явно указать `--runner native`.

2.2 Первый запуск (один раз)

Все команды выполняются **из корня репозитория**: оттуда читаются `config.yaml` и `.env`, и относительно текущей папки считаются пути `.mnogobase/`, `wiki/`, `logs/`, `runs/`.

```
# 1. Для новой копии проекта создайте .env; существующий .env сохраните
cp .env.example .env
#    откройте .env и впишите LLM_API_KEY=<ваш ключ>
#    NEO4J_PASSWORD=mnogobase-dev соответствует локальному Compose
#    вне Mac добавьте MNOGOBASE_DEVICE=auto: в config.yaml задано mps

# 2. Базы данных: PostgreSQL (:5432), Qdrant (:6333), Neo4j (:7474, :7687)
docker compose up -d
```

```
# 3. Python-зависимости (пакет ставится в .venv в режиме редактирования)
uv sync

# 4. Эмбеддер (:8001), реранкер (:8002) и VLM для картинок (:8003) по config.yaml: на Mac —
#    нативный llama.cpp, на Linux-сервере — Docker (раздел 11). Первый старт качает модели
uv run mnogobase models up

# 5. Проверка: всё должно быть зелёным
uv run mnogobase doctor
```

> **Первый запуск дольше обычного.** `mnogobase models up` качает GGUF эмбеддера, реранкера и VLM с Hugging Face, а первый `ingest` — модели разметки и OCR Docling и BM25 `Qdrant/bm25`. Токенайзер чанкера по умолчанию (`chunking.tokenizer: auto`) у llama.cpp берётся из самого GGUF через `/tokenize` сервера эмбеддера, у vLLM — HF-токенайзер `embedder.model` (несколько МБ). Если Hugging Face откажет в доступе (закрытый репозиторий), примите лицензию на странице модели и выполните `hf auth login` (или задайте `HF_TOKEN`).

2.3 Файл .env

`.env` читается из текущей папки при старте любой команды. Переменные, уже заданные в окружении, имеют приоритет над `.env`. **`.env` никогда не коммитится** (он в `.gitignore`).

Переменная	Нужна	Что это
`LLM_API_KEY`	для внешнего LLM	ключ OpenAI-совместимого эндпоинта. Имя переменной задаётся в `llm.api_key_env`. Если переменная пуста, клиент отправляет заглушку `local` — локальному llama.cpp или vLLM без `--api-key` ключ не нужен
`NEO4J_PASSWORD`	да	пароль Neo4j. Имя переменной задаётся в `neo4j.password_env`. `docker-compose.yml` берёт этот же пароль при первом создании базы
`HF_TOKEN`	если Hugging Face требует авторизацию	токен для скачивания закрытых моделей (GGUF, веса vLLM) и токенайзера, когда он берётся с Hugging Face
`MNOGOBASE_*`	по желанию	переопределение любого ключа `config.yaml` (раздел 6.2)

2.4 Проверка окружения (doctor)

`uv run mnogobase doctor` печатает таблицу проверок и завершается с кодом 1, если хоть одна проверка не прошла. Предупреждение (`warn`) ошибкой не считается.

Проверка	Что делает	Типичный провал
`device`	выбирает устройство для Docling: CUDA → MPS → CPU (или то, что задано в `device`)	—
`models`	`GET <url>/models` у сервера эмбеддера и (при `enabled`) реранкера и VLM: отдаёт ли он `model` из конфига	`... is not reachable: start the local models with mnogobase models up`, `<model> is not served by <url> (it serves ...)`
`tokenizer`	загружает токенайзер чанкера: при `auto` — `/tokenize` сервера эмбеддера (`serve: llamacpp`), HF-токенайзер `embedder.model` (`vllm`) или, у `api`, репозиторий из `root` в `/v1/models` либо `/tokenize` сервера; иначе HF-репозиторий из `chunking.tokenizer`	`TokenizerError: ...` (см. раздел 9)
`llm`	`GET <llm url>/models` с ключом; 404/405 считаются нормой (`models endpoint not available`)	401/403 (неверный ключ), `<model> is not served by <url>`

Проверка	Что делает	Типичный провал
`qdrant`	доступность и размерность коллекции `mb_chunks` (при `embedder.dim: auto` — против размерности, которую вернул эмбеддер; если он недоступен, размерность не сравнивается)	`mb_chunks has dim 768, config 512: run mnogobase reindex` (после смены `embedder.dim`), `mb_chunks has dim 2560, embedder 1024: ...` (после смены модели при `auto`)
`neo4j`	подключение с логином и паролем	`ServiceUnavailable`, `AuthError`
`index`	совпадает ли эмбеддер, которым построен индекс (модель, размерность, шаблоны), с текущим конфигом	`index was built with ..., config now uses ...: run mnogobase reindex`
`types`	менялись ли типы сущностей после загрузки документов	только `warn`, см. раздел 6.3

Команды `ingest`, `remove`, `wiki build`, `ask` и `compare` перед стартом сами выполняют облегчённую проверку (`models`, `llm`, `qdrant`, `neo4j`; `ingest` — ещё и `tokenizer`), `reindex` — без `llm` и без сверки размерности коллекций (он их пересоздаёт), `reset` — только `qdrant` и `neo4j`. Если сервис недоступен, команда печатает причину, советует `mnogobase doctor` и завершается с кодом 2.

2.5 Первый документ и ответ

Когда `doctor` проходит, загрузите небольшой встроенный корпус на русском и английском. Это реальная обработка с вызовами настроенной LLM; примеры попадут в текущую базу знаний.

```
uv run mnogobase ingest tests/fixtures/
uv run mnogobase status
uv run mnogobase ask "Что такое multi-head attention?" --mode rag
uv run mnogobase compare "Что такое multi-head attention?"
```

Успешный `ingest` печатает итог по файлам и wiki. `status` показывает завершённые этапы; `ask` выводит ответ с номерами `[1]`, `[2]` и таблицу источников. У короткого корпуса wiki может содержать мало страниц из-за порога `wiki.min_mentions`. Точные ответы и сущности зависят от модели.

Дальше положите свои файлы в `documents/` и выполните `uv run mnogobase ingest documents/`. Wiki собирается в конце `ingest` автоматически; отдельный `wiki build` нужен после `--no-wiki`, ошибок страниц или изменения настроек wiki.

Что посмотреть после запуска	Где
Ответ и процитированные фрагменты	вывод `ask` / `compare` в терминале
Страницы wiki	`wiki/index.md`, `wiki/entities/*.md`; открытие в Obsidian
Граф знаний	Neo4j Browser, логин `neo4j`, пароль из `.env`
Коллекции и payload чанков	Qdrant Dashboard
Ошибки и незавершённые этапы	`uv run mnogobase status`, `logs/mnogobase.jsonl`

2.6 Что задано в config.yaml

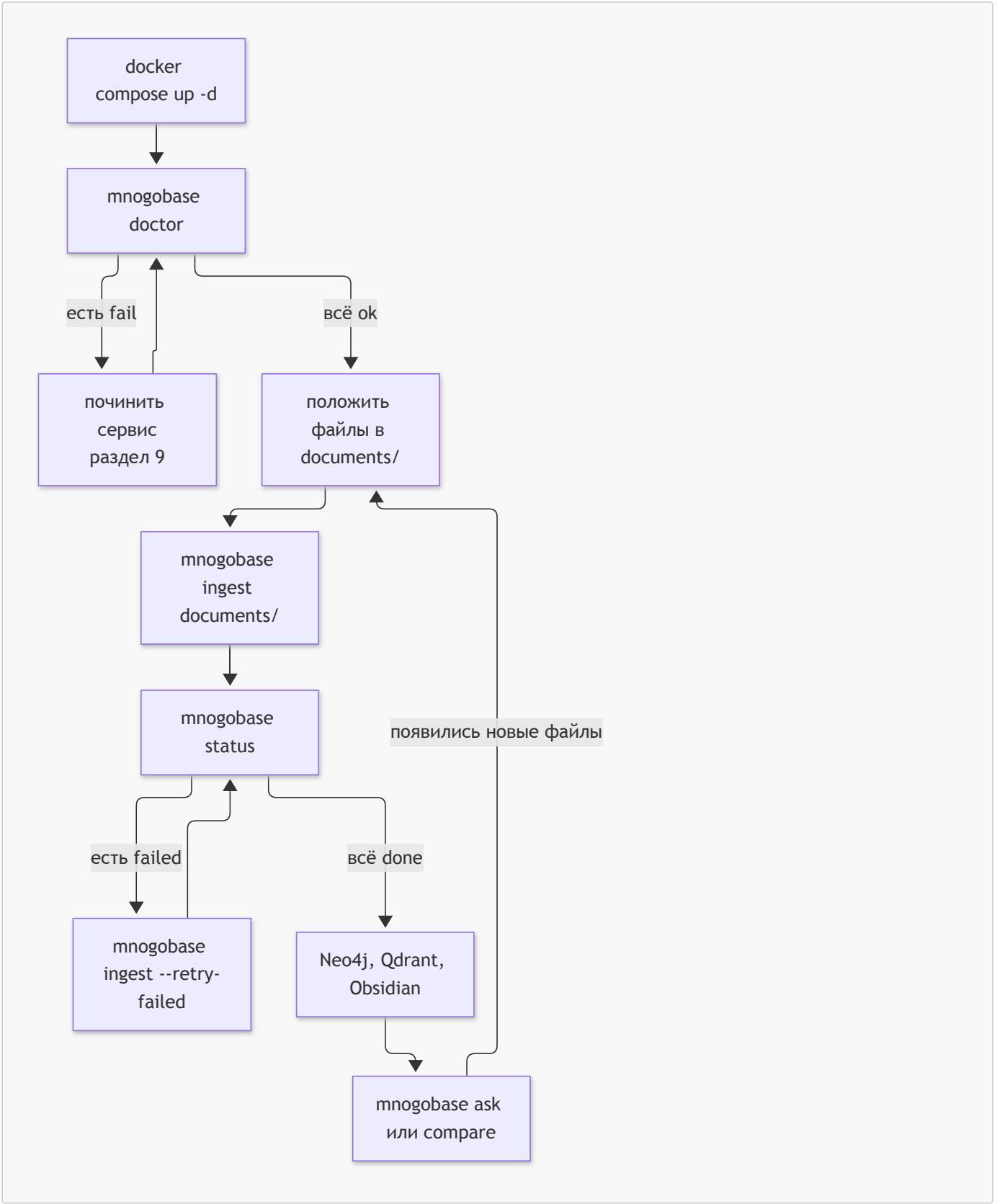
Это параметры поставляемого файла. Они отличаются от некоторых значений по умолчанию в Python-классах, приведённых в разделе 6. `.env`, переменные окружения и `--config` могут изменить итоговые настройки.

Компонент	Настройка в config.yaml	Назначение
LLM	`gpt-6-luna`, `serve: api`, `https://codex.sale/v1`	извлечение, слияние сущностей, wiki, перевод вопроса, ответ
Эмбеддер	`mradermacher/Qwen3-Embedding-0.6B-GGUF:Q8_0`, llama.cpp, порт `8001`	dense-векторы; размерность `auto`

Компонент	Настройка в config.yaml	Назначение
Реранкер	`gorse-io/Qwen3-Reranker-0.6B-GGUF:Q8_0`, llama.cpp, порт `8002`	переоценка 30 кандидатов перед выбором `k`
Vision	`prithivMLmods/Xiaomi-OCR-0-GGUF:Q6_K`, llama.cpp, порт `8003`	текстовые описания изображений; включено
Разбор	`device: mps`, OCR RU/EN	на другой платформе задайте `MNOGOBASE_DEVICE=auto`
Чанки	`max_tokens: 512`, токенайзер `auto`	лимит при разбиении документов
Поиск	`k: 8`, `neighbors: 2`, `translate: true`	гибридный поиск и расширение найденных фрагментов
Контекст и язык	`context_tokens: 20000`, `answer_language: ru`, `wiki.language: en`	бюджет оценивается по длине текста; это не размер окна модели
Режим `all`	`rag: 0.2`, `wiki: 0.2`, `graph: 0.6`	доли общего бюджета контекста
Параллельность LLM	`concurrency: 16`, `temperature: 0.1`, `timeout_s: 360`	нагрузка и ожидание ответов

3. Повседневная работа

Типичная сессия: поднять сервисы, проверить окружение, загрузить документы, проверить статус, посмотреть результаты и задавать вопросы; новые документы просто догружаются той же командой.



3.1 Куда класть документы

Кладите файлы в папку **`documents/`** в корне репозитория (она в `.gitignore`, в git ничего не попадёт):

```
mkdir -p documents
cp ~/Downloads/*.pdf documents/
```

- Подпапки можно любые: `ingest` обходит папку рекурсивно.
- Скрытые файлы и папки (имя начинается с `.`) пропускаются.

- Берутся только расширения из `parsing.extensions`: `pdf, docx, pptx, xlsx, html, htm, md, adoc, csv, txt` и картинки `png, jpg, jpeg, tif, tiff, bmp, webp` (скриншоты). Остальные файлы молча игнорируются.
- Документы могут быть на русском и на английском вперемешку. Страницы wiki пишутся на языке из `wiki.language` (по умолчанию `en`).

Папка может быть любой: `documents/` — просто соглашение. `ingest` принимает и отдельные файлы, и несколько путей сразу.

3.2 Загрузка документов (ingest)

```
uv run mnogobase ingest documents/
```

Для каждого файла по очереди выполняются этапы:

Этап	Что происходит	Где результат
`parse`	Docling разбирает файл: текст, заголовки, таблицы, OCR для сканов и скриншотов; при `vision.enabled` графики, схемы и скриншоты описывает VLM, описание становится текстом документа	`.mnogobase/cache/<doc_id>.json`, картинки и `images.json` в `.mnogobase/cache/<doc_id>/images/`
`chunk`	HybridChunker режет документ по структуре на чанки до `chunking.max_tokens` (512) токенов	`.mnogobase/cache/<doc_id>.chunks.json`
`embed`	каждый чанк (с заголовками раздела) → плотный вектор Qwen3-Embedding и BM25	Qdrant `mb_chunks`; узлы `Document`, `Chunk`, связи `HAS_CHUNK`, `NEXT` в Neo4j
`extract`	LLM достаёт из каждого чанка сущности и связи (параллельно, до `llm.concurrency` запросов)	кэш `extraction_cache` в PostgreSQL
`graph`	сущности сливаются с уже известными (в том числе RU и EN), связи пишутся в граф	Neo4j `Entity`, `MENTIONS`, `RELATED`; Qdrant `mb_entities`; `entity_ids` в payload чанков
wiki	после всех файлов: страницы сущностей с новыми упоминаниями создаются или дописываются	`wiki/entities/*.md`, `wiki/index.md`, `wiki/log.md`; Qdrant `mb_wiki_pages`

Как это работает:

- **Инкрементально.** Документ опознаётся по хешу содержимого (`doc_id`). Повторный `ingest` той же папки пропускает неизменённые файлы. Изменённый файл сначала удаляет свою старую версию из Qdrant, Neo4j и wiki, затем загружается как новый — без дублей. Переименованный или перемещённый файл без изменений не загружается заново: ссылки в источниках переводятся на новый путь, а wiki-страницы, которые цитируют документ, показывают новое имя файла после ближайшей сборки wiki (`ingest` без `--no-wiki` или `wiki build`).
- **Возобновляемо.** Если процесс прервался (Ctrl+C, упал сервис), следующий `ingest` продолжит каждый файл с первого незавершённого этапа. Результаты LLM кэшируются, поэтому повторно оплачивать извлечение не нужно.
- **Ошибки изолированы.** Один сломанный файл не останавливает остальные. Внутри `extract` неудачные чанки допускаются, пока их доля не больше `extract.max_failed_ratio` (0.2). Упавший файл при следующих запусках пропускается с сообщением `stage <этап> failed earlier; rerun with --retry-failed`.

Варианты:

```
uv run mnogobase ingest documents/statya.pdf        # один файл
uv run mnogobase ingest documents/ --no-wiki        # без обновления wiki (быстрее; wiki можно
собрать потом)
uv run mnogobase ingest --retry-failed              # перезапустить все упавшие файлы (пути
берутся из registry)
uv run mnogobase ingest documents/ --retry-failed   # перезапустить упавшие среди этих путей и
догрузить новые
```

Пока `ingest` работает, в терминале обновляется прогресс:

```
⠋ files                                          ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 4/12  0:21:40 3
processed · 1 skipped
⠋ Attention Is All You Need.pdf · extract ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 43/96 0:03:12 2 failed
```

- верхняя строка — файлы: сколько готово из найденных (с разбивкой processed / skipped / failed) и сколько времени идёт прогон;
- нижняя — текущий файл и его этап (`parse`, `chunk`, `embed`, `extract`, `graph`). На `extract` видно, сколько чанков из общего числа уже извлечено (попадания в кэш тоже считаются, поэтому возобновлённый прогон проходит быстро) и сколько не удалось; на `graph` — сколько сущностей слито; `parse`, `chunk` и `embed` показывают только время;
- после всех файлов — шаги wiki: `wiki evidence` (поиск доказательств), `wiki draft` (LLM пишет страницы) и `wiki write` (запись страниц), каждый с числом страниц.

`wiki build` и `reindex` показывают такие же полосы для своих шагов. Когда команда заканчивается, полосы исчезают и печатается сводка; вне терминала (пайп, CI) прогресс не рисуется. Подробные времена каждого этапа — в `logs/mnogobase.jsonl` (раздел 4.5). Служебные сообщения библиотек (INFO от RapidOCR, полосы загрузки моделей Hugging Face) скрыты, предупреждения и ошибки печатаются как раньше.

В конце печатается сводка `run <run_id>: processed N, skipped N, failed N`, отчёт wiki и список упавших файлов с причиной. Код выхода 1, если упал хоть один файл или хоть одна wiki-страница.

> Отдельной команды «только чанкирование» нет: чанки получаются на этапе `chunk` внутри `ingest`. Посмотреть их можно в `.mnogobase/cache/<doc_id>.chunks.json`, в Qdrant (раздел 4.2) или в Neo4j (раздел 4.1).

3.3 Сборка wiki (wiki build)

```
uv run mnogobase wiki build         # обновить страницы сущностей с новыми упоминаниями
uv run mnogobase wiki build --all   # перегенерировать все страницы (например, после смены
языка или промпта)
```

`ingest` без `--no-wiki` сам вызывает сборку wiki в конце. Отдельно `wiki build` нужен после `ingest --no-wiki`, после изменения настроек `wiki.*` или промпта `wiki_page.md`, а также для повтора упавших страниц (они остаются в очереди). Печатается отчёт `wiki: created N, updated N, deleted N, skipped N, failed N`; код выхода 1, если какая-то страница не собралась.

Ошибки при `wiki build --all` также оставляют страницы в очереди: для повтора достаточно обычного `wiki build`. Если новая страница не записалась, ссылки на неё исключаются из остальных страниц без повторных запросов к LLM.

3.4 Вопросы (ask и compare)

```
uv run mnogobase ask "Что такое механизм внимания?"               # режим all (по умолчанию)
uv run mnogobase ask "Что такое механизм внимания?" --mode graph  # или -m rag | wiki | graph |
all
uv run mnogobase ask "Что такое механизм внимания?" --k 12        # больше результатов на
каждый поиск
uv run mnogobase compare "Что такое механизм внимания?"           # все 4 режима рядом
```

`--k` должен быть положительным целым числом; значения 0 и меньше отклоняются до подключения к сервисам.

Режим	Откуда берётся контекст
`rag`	ближайшие чанки из Qdrant `mb_chunks` (гибридный поиск dense + BM25, слияние RRF)
`wiki`	разделы wiki-страниц из `mb_wiki_pages` (тот же гибридный поиск) и чанки, на которые эти разделы ссылаются

Режим	Откуда берётся контекст
`graph`	сущности, найденные по вопросу, их окрестность в графе на `graph.hops` шага и чанки-доказательства сильнейших связей
`all`	всё вместе: каждый режим получает долю общего бюджета токенов (`retrieval.budget`), повторы убираются

Ответ печатается в рамке с режимом и временем, под ним — таблица источников: номер `[n]`, отметка `cited` (LLM сослалась на этот номер в тексте), вид (`chunk`, `wiki`, `entity`, `relation`), файл и страница, `ref` (id чанка, раздела wiki или связи) и начало текста. Язык ответа задаёт `retrieval.answer_language` (в `config.yaml` — `ru`; `auto` — язык вопроса). Вопрос не на английском (больше 30% букв не латиница) перед поиском переводится на английский (задача LLM `query`), и поиск идёт по обеим формулировкам; отключается `retrieval.translate: false`. Если источников нет, печатается `The knowledge base has no relevant sources for this question.`

`compare` печатает таблицу: режим, ответ, процитированные источники, задержка, токены LLM, размер контекста, и дописывает по строке на режим в `runs/compare.jsonl` (раздел 4.4).

`ask` и `compare` работают только внутри проекта (нужны схема registry PostgreSQL и локальный `.mnogobase/postgresql-project.json`), иначе завершаются с кодом 2 и ничего не создают. Блокировку `ingest` они не берут: спрашивать можно и во время загрузки.

3.5 Состояние (status)

```
uv run mnogobase status
```

Показывает: файлы по статусам (`done`, `failed`, `processing`, `pending`), таблицу этапов (`done / failed / pending / running`), число сущностей в очереди на обновление wiki, незавершённые удаления старых версий документов, предупреждение о смене типов сущностей и таблицу ошибок (`doc_id`, этап, текст ошибки). Вне проекта сообщает, что проекта нет, и завершается с кодом 0, ничего не создавая.

3.6 Пересчёт векторов (reindex)

```
uv run mnogobase reindex
```

Нужен после смены эмбеддера (модель, размерность, шаблоны): `ingest`, `wiki build`, `ask` и `compare` сами замечают такую смену и останавливаются с просьбой запустить `reindex`. Сначала проверяет, что эмбеддер выдаёт вектор размерности `embedder.dim` (при `auto` — просто что он отвечает; иначе останавливается, ничего не тронув), затем удаляет и заново создаёт три коллекции Qdrant с новой размерностью и пересчитывает векторы: чанки — из кэша `.mnogobase/cache/<doc_id>.chunks.json` (если кэша нет, текст берётся из Neo4j), сущности — из Neo4j, разделы wiki — из файлов `wiki/entities/*.md`. Граф, wiki-файлы и кэш извлечения не меняются, LLM не вызывается. Печатает `reindexed: N chunks, N entities, N wiki sections`.

Пока `reindex` идёт, индекс помечен как незавершённый: если его прервать, `ingest` и `ask` откажутся работать, пока `reindex` не будет запущен снова и не дойдёт до конца. Пример смены модели и размерности — рецепт 7.4.

До удаления коллекций проверяется чтение документов из кэша или Neo4j и всех зарегистрированных wiki-файлов. Повреждённый кеш чанков восстанавливается для пересчёта из Neo4j. Если данных не хватает или их нельзя прочитать, команда сообщает `missing or unreadable ...; nothing was changed` и сохраняет индекс. Восстановите недостающий файл из резервной копии; wiki можно пересобрать через `wiki build --all` с прежним эмбеддером. Исчезновение данных уже во время пересчёта оставляет индекс незавершённым и требует повторного `reindex` после восстановления.

3.7 Полный сброс (reset)

```
uv run mnogobase reset            # покажет, что удалит, и спросит подтверждение
uv run mnogobase reset --yes      # без вопроса
```

Область действия `reset`:

- вне проекта (нет маркера PostgreSQL) он отказывается работать даже с `--yes` (код 2): неверная папка или `--config` не должны стереть чужие данные;
- перед подтверждением показывает, что именно удалит: **все узлы в базе Neo4j по умолчанию** (держите отдельный Neo4j на проект), коллекции Qdrant с префиксом `mb_`, таблицы registry выбранного проекта и файловые кэши в `.mnogobase/` (схема БД сохраняется, папка `cache/` удаляется), а из wiki — только то, что создал сам: `entities/`, `index.md`, `log.md`. Другие файлы в `wiki/` (например, ваши заметки и настройки Obsidian в `.obsidian/`) не трогаются, папка удаляется, только если опустела.

Очередь заданий (таблица PostgreSQL `jobs`), измерения `processing.jsonl`, логи и сохранённые отчёты остаются. Перед сбросом остановите worker; повторная постановка неизменённых файлов описана в разделе 15.1.

Отказ на вопросе подтверждения завершает команду с кодом 1.

3.8 Все команды

Глобальная опция `--config/-c PATH` ставится **перед** командой: `uv run mnogobase -c other.yaml status`. Без неё используется путь из `$MNOGOBASE_CONFIG` или `./config.yaml`; если файла `./config.yaml` нет, берутся значения по умолчанию из кода.

Команда	Что делает	Проверка сервисов	Блокировка
`doctor`	все проверки окружения (раздел 2.4)	—	нет
`status`	сводка по файлам, этапам, очереди wiki, удалениям и ошибкам	—	нет
`ingest PATH... [--no-wiki] [--retry-failed]`	загрузка документов	models, tokenizer, llm, qdrant, neo4j	да
`remove [PATH...] [--missing] [--dry-run] [--yes]`	забыть источники и обновить граф, индексы и wiki, сохранив исходные файлы	models, llm, qdrant, neo4j; без сервисов при preview	да
`wiki build [--all]`	обновить страницы сущностей с новыми упоминаниями (или все)	models, llm, qdrant, neo4j	да
`ask "ВОПРОС" [--mode/-m rag|wiki|graph|all] [--k N]`	ответ с цитатами в одном режиме	models, llm, qdrant, neo4j	нет
`compare "ВОПРОС" [--k N]`	все 4 режима рядом и запись в `runs/compare.jsonl`	models, llm, qdrant, neo4j	нет
`reindex`	пересчитать все векторы после смены эмбеддера	models, qdrant, neo4j	да
`reset [--yes]`	очистить индексы, граф, registry, кэш документов и сгенерированную wiki	qdrant, neo4j	да
`models up [РОЛИ...] [--runner auto|docker|native]`	поднять серверы ролей с `serve: llamacpp | vllm` и дождаться готовности (раздел 7.15)	—	нет
`models down` / `status` / `logs РОЛЬ [-f]`	остановить серверы моделей / где какая модель и готова ли / лог сервера роли	—	нет
`jobs enqueue PATH... [--force]`	поставить файлы в постоянную очередь	нет	транзакция очереди
`jobs list [--limit N]` / `jobs retry [--id N]`	посмотреть очередь / вернуть ошибочные задания на повтор	нет	транзакция очереди

Команда	Что делает	Проверка сервисов	Блокировка
`worker run [--watch PATH]`	обрабатывать очередь; с `--watch` также отслеживать папку	как у ingest, перед заданием	один worker; блокировка корпуса на обработку
`worker run --once`	обработать готовые задания и завершиться	как у ingest	как у worker
`eval ingest PATH... --phase build|update --output DIR`	загрузить документы и измерить общий preprocessing	как у ingest	блокировка корпуса и каталог эксперимента
`eval run DATASET --output DIR [--mode MODE] [--limit N] [--k N]`	сохранить ответы выбранных режимов; по умолчанию rag, wiki, graph	models, llm, qdrant, neo4j	блокировка корпуса и каталог эксперимента
`eval score DIR [--metric NAME]`	оценить сохранённые ответы метриками Ragas	подключения судьи и его embeddings	каталог эксперимента
`eval report DIR`	построить CSV и Markdown по сохранённым данным	нет	каталог эксперимента
`eval benchmark DATASET --output DIR [--skip-score]`	выполнить run → score → report; без судьи при `--skip-score`	по фазам	по фазам

Команды `ingest`, `remove`, `wiki build`, `reindex`, `reset`, `eval ingest`, `eval run` и обработка задания worker используют общую PostgreSQL advisory lock корпуса. Это не позволяет менять корпус во время экспериментальной генерации ответов. Обычные пишущие команды при занятой блокировке завершаются; worker откладывает обработку, наблюдатель продолжает пополнять очередь. `ask` и `compare` блокировку не берут, поэтому во время записи могут видеть промежуточное состояние.

В каталоге эксперимента `eval` используется отдельная `.eval.lock`. Один worker на проект обеспечивается PostgreSQL advisory lock. Очередь `jobs` и registry используют отдельные таблицы одной схемы PostgreSQL. Для нескольких наблюдаемых папок повторяйте `--watch`, для режимов и метрик — `--mode` и `--metric`. Полные опции доступны через `uv run mnogobase COMMAND --help`.

Коды выхода:

Код	Когда
0	успех; `status` вне проекта
1	`doctor` нашёл проблему; в `ingest` / `remove` / `wiki build` есть ошибки; отказ в подтверждении `reset` или `remove`; в `eval run` / `score` / `benchmark` есть неполные/ошибочные результаты; `worker --once` оставил незавершённые задания
2	ошибка конфига; сервис недоступен на предварительной проверке; блокировка занята; нет проекта (`ask`, `compare`, `remove`, `reset`); индекс построен другим эмбеддером, другой размерности или с другими шаблонами; `reindex`: эмбеддер выдаёт не ту размерность; не удаётся завершить удаление старой версии документа

3.9 Удаление документов (remove)

```
uv run mnogobase remove documents/old.pdf --dry-run  # показать выбор без удаления и вызовов моделей
uv run mnogobase remove documents/old.pdf --yes      # удалить знания, сохранить файл на диске
uv run mnogobase remove documents/archive/ --yes     # забыть зарегистрированные файлы каталога
uv run mnogobase remove --missing --dry-run          # посмотреть исчезнувшие источники всей базы
uv run mnogobase remove --missing --yes              # убрать их из базы
uv run mnogobase remove documents/ --missing --yes   # проверить только указанный каталог
```

Без `--yes` команда показывает выбранные пути и запрашивает подтверждение. Пути могут уже отсутствовать на диске: выбор идёт по registry, включая вложенные каталоги. Неизвестный путь ничего не удаляет. При `--missing` ошибка доступа к файлу прерывает выбор, а не считается исчезновением источника.

Если есть другая зарегистрированная копия того же содержимого, документ сохраняется, а пути цитат переводятся на оставшуюся копию. При удалении последней копии убираются исходные чанки из dense/BM25-индекса, их записи и подтверждения связей в графе, сущности без упоминаний, кэши и страницы без источников. Затронутые страницы wiki и ссылки пересобираются; `index.md` обновляется.

Сводные описания затронутых сущностей и связей не имеют привязки отдельных фраз к источникам, поэтому сбрасываются; векторы сохранившихся сущностей пересчитываются по имени. Их исходные подтверждения остаются в графе. Затронутые сгенерированные страницы и их поисковый индекс очищаются до генерации, чтобы старые утверждения не попали в новый текст. Если LLM недоступна, страницы остаются в очереди на пересборку: повторите `uv run mnogobase wiki build` (ошибка страницы даёт код 1).

Прерванное каскадное удаление записывается в `pending_removals`. Следующие `remove`, `ingest` или `wiki build` доводят его до конца, даже если забываемый исходный файл всё ещё существует. Ошибка каскада даёт код 2 с идентификатором документа. `status` показывает незавершённые удаления.

Команда сохраняет файлы на диске. Повторный `ingest` возвращает их в базу; watcher и задания очереди также могут импортировать существующий источник. Если документ не должен возвращаться, уберите его из наблюдаемого каталога. Сам watcher удаление знаний не запускает: после удаления файлов используйте `remove --missing`. Исторические логи и результаты экспериментов сохраняются.

4. Как смотреть результаты

4.1 Граф в Neo4j

1. Откройте **http://localhost:7474**.
2. Подключение: `neo4j://localhost:7687`, пользователь `neo4j`, пароль из `.env` (по умолчанию `mnogobase-dev`).
3. **Стиль (один раз).** Выполните в строке запроса `:style`, затем перетащите в открывшуюся панель файл `docs/neo4j/mnogobase.grass`. У сущностей появятся имена, у связей — предикаты, у документов и чанков — свои цвета и подписи (заголовок документа, номер чанка).
4. **Цвет по типу сущности (по желанию).** Тип хранится в свойстве `type`; чтобы раскрасить узлы, добавьте его как метку (стиль выше уже содержит цвета для всех 18 типов по умолчанию, см. раздел 6.3):

   ```
   MATCH (e:Entity) CALL apoc.create.addLabels(e, [e.type]) YIELD node RETURN count(node);
   ```

   Повторяйте после новых `ingest`. mnogobase эти метки не использует и не мешает им.

Готовые запросы (вставлять в строку запроса Neo4j Browser):

```
// Весь граф знаний: сущности и связи между ними (самые сильные связи)
MATCH (a:Entity)-[r:RELATED]->(b:Entity)
RETURN a, r, b ORDER BY r.weight DESC LIMIT 300;
```

```
// Окрестность одной сущности на 2 шага (замените имя)
MATCH (e:Entity) WHERE toLower(e.name) CONTAINS 'transformer'
MATCH p = (e)-[:RELATED*1..2]-(:Entity)
RETURN p LIMIT 150;
```

```
// Самые упоминаемые сущности
MATCH (e:Entity)
```

```
RETURN e.name AS name, e.type AS type, e.mention_count AS mentions, e.aliases AS aliases
ORDER BY mentions DESC LIMIT 30;
```

```
// Документ → его чанки → сущности, которые в них упоминаются
MATCH (d:Document)-[:HAS_CHUNK]->(c:Chunk)-[:MENTIONS]->(e:Entity)
WHERE d.path CONTAINS 'statya'          // часть имени файла
RETURN d, c, e LIMIT 200;
```

```
// Чанки одного документа по порядку (как его порезал чанкер)
MATCH (d:Document)-[:HAS_CHUNK]->(c:Chunk)
WHERE d.path CONTAINS 'statya'
RETURN c.idx AS idx, c.headings AS headings, c.n_tokens AS tokens, left(c.text, 200) AS text
ORDER BY idx;
```

```
// Откуда взят факт: связь и тексты чанков-доказательств
MATCH (a:Entity)-[r:RELATED]->(b:Entity)
WHERE toLower(a.name) CONTAINS 'transformer'
UNWIND r.evidence AS cid
MATCH (c:Chunk {chunk_id: cid})<-[:HAS_CHUNK]-(d:Document)
RETURN a.name, r.predicate, b.name, d.path, c.page_start, left(c.text, 200) LIMIT 50;
```

```
// Wiki-страницы и ссылки между ними
MATCH (p:WikiPage)-[l:LINKS_TO]->(q:WikiPage) RETURN p, l, q LIMIT 200;
```

```
// Сущности, которые слились из русских и английских упоминаний (есть кириллические алиасы)
MATCH (e:Entity) WHERE any(a IN e.aliases WHERE a =~ '.*[а-яА-ЯёЁ].*')
RETURN e.name, e.type, e.aliases ORDER BY e.mention_count DESC LIMIT 50;
```

Полная схема графа — в разделе 5.4.

4.2 Векторы и чанки в Qdrant

Откройте **http://localhost:6333/dashboard** → **Collections**:

Коллекция	Что в ней	Payload
`mb_chunks`	чанки документов	`chunk_id`, `doc_id`, `text`, `headings`, `page`, `path`, `modality`, `entity_ids`
`mb_entities`	сущности (для слияния дублей и поиска сущностей по вопросу)	`entity_id`, `name`, `type`
`mb_wiki_pages`	разделы wiki-страниц (по точке на раздел `##`)	`key`, `page_id`, `entity_id`, `path`, `section`, `text`, `chunk_ids`

В коллекции есть вкладка **Points** (содержимое и payload) и **Visualize** (2D-проекция векторов: видно, как чанки группируются по темам). Фильтр по документу в Points: `doc_id` = нужный `doc_id`.

4.3 Wiki в Obsidian

1. В Obsidian: **Open folder as vault** → выберите папку **`wiki/`** в корне репозитория. Она появится после первого `ingest` (или `wiki build`).

2. Начните с **`index.md`**: это оглавление всех страниц, сгруппированное по типам сущностей. В **`log.md`** — журнал прогонов: время, `run_id`, обработанные документы и какие страницы созданы, обновлены или удалены.

3. Страницы сущностей лежат в `entities/<slug>.md`. Каждая страница содержит:

   - свойства (frontmatter): `id` (это `entity_id`), `type`, `aliases`, `sources` (число разных файлов-источников), `updated`, `version`. `aliases` Obsidian использует при поиске и в подсказках ссылок, поэтому русские и английские названия находят одну страницу;
   - текст со сносками `[^chunk_id]`: каждое утверждение ссылается на чанк-источник, а внизу в разделе **Sources** указаны файл и страница;
   - раздел **Related**: связи из графа (`predicate → [[slug|Имя]]` для исходящих, `←` для входящих). Ссылкой связь становится, только если у соседа есть страница.

4. **Граф:** `Cmd/Ctrl+G` открывает общий граф, «Open local graph» в меню страницы — окрестность текущей страницы.

5. **Раскраска по типам и фильтр** настраиваются сами: каждый `wiki build` дописывает в `wiki/.obsidian/graph.json`

   - фильтр `-path:/^(index|log)\.md$/`: `index.md` и `log.md` пропадают из графа, но остаются в списке файлов и в поиске (регулярное выражение нужно, потому что `-file:log` скрыл бы и страницы вроде `logistic-regression`);
   - по цветовой группе на каждый тип из `extract.entity_types` с запросом по свойству (`[type:Person]`, `[type:"Research Area"]` для имён с пробелами). Цвета берутся по порядку из фиксированной палитры, поэтому при тех же типах они совпадают от сборки к сборке; `Other` — серый.

   Добавляется только недостающее: ваш фильтр, ваши группы и перекрашенные вами типы остаются, группы удалённых из конфига типов не удаляются, нечитаемый `graph.json` не трогается. Если хранилище открыто во время сборки, Obsidian может перезаписать файл своей копией настроек: тогда закройте и откройте хранилище и запустите `wiki build` ещё раз. Отключается `wiki.obsidian: false`; `reset` папку `.obsidian/` не трогает.

Что важно знать:

- **Не редактируйте `entities/*.md` вручную:** при следующем обновлении страницы LLM перепишет её на основе прежнего текста и новых фактов, ручные правки могут потеряться. Свои заметки кладите рядом, например в `wiki/notes/`. Их не трогает ни `wiki build`, ни `reset`. В заметках можно ссылаться на страницы сущностей: `[[transformer]]`.
- Страница создаётся для сущностей, которые упомянуты не менее чем в `wiki.min_mentions` чанках (по умолчанию 2) и для которых нашлись чанки-доказательства.
- Папка `wiki/` в `.gitignore` (это производные данные). Если команде нужно хранить wiki в git, уберите её оттуда.

4.4 Сравнение режимов (runs/compare.jsonl)

Каждый `compare` дописывает 4 строки JSON (по одной на режим) с полями:

Поле	Что это
`ts`	время прогона (UTC, одинаковое для 4 строк)
`question`, `mode`, `answer`	вопрос, режим, текст ответа
`alt_queries`	дополнительные формулировки для поиска (английский перевод вопроса; пусто, если перевод не нужен или не удался)
`sources`	список источников: `n`, `kind`, `ref`, `path`, `page`, `snippet`, `cited`
`latency_ms`	перевод запроса, поиск и генерация вместе
`tokens_in`, `tokens_out`	токены LLM на ответ (по данным `usage` эндпоинта)
`context_tokens`	оценка размера контекста (символы / 4)
`config_hash`	хеш настроек `llm`, `embedder`, `chunking`, `retrieval`, `graph`: сравнивайте прогоны только с одинаковым хешем

Примеры с `jq`:

```
# режим, задержка, токены и число процитированных источников
jq -r '[.mode, .latency_ms, .tokens_in, .tokens_out, ([.sources[] | select(.cited)] | length)]
| @tsv' runs/compare.jsonl

# все ответы на конкретный вопрос
jq 'select(.question | test("внимани")) | {mode, answer}' runs/compare.jsonl
```

4.5 Логи (logs/mnogobase.jsonl)

Команды пишут структурированный лог в `logs/mnogobase.jsonl` (JSON по строкам, ротация по 10 МБ, хранится 5 старых файлов). В консоль выводятся только предупреждения и ошибки. Трейсбеки пишутся без локальных переменных, чтобы в лог не попадали ключи и тексты документов.

Общие поля: `ts`, `level`, `event`, `run_id` (у `ingest` и `wiki build`), `doc_id` (внутри обработки файла). Основные события:

`event`	Поля	Когда
`stage_done` / `stage_failed`	`stage`, `duration_ms`, `path`; при ошибке `error_type`, `error`, `exception`	каждый этап каждого файла
`parsed`	`title`, `n_pages`, `n_images`	после Docling
`llm_call`	`task` (`extract`, `resolve`, `wiki`, `answer`, `query`), `model`, `tokens_in`, `tokens_out`, `retries`, `duration_ms`	каждый вызов LLM
`llm_invalid_json`	`task`, `error`	ответ LLM не прошёл схему, идёт повтор с текстом ошибки
`extract_failed` / `extract_partial`	`chunk_id`, `error` / `failed`, `total`	неудачные чанки на этапе `extract`
`entity_merged`	`name`, `into`, `score`, `method` (`vector`, `llm` или `llm_name` — кандидат по ключу имени, `score` пустой)	слияние сущностей
`wiki_built` / `wiki_page_failed`	счётчики / `entity_id`, `error`	сборка wiki
`ingest_done`, `ingest_file_failed`	счётчики / `path`, `error`	итог `ingest`
`resuming_interrupted_stages`, `resuming_document_removal`, `document_removed`, `document_repointed`, `document_needs_reingest`		восстановление, удаление старых версий, переименование
`entity_types_changed`	`detail`	типы сущностей поменялись
`reindex_done`, `reset_done`	счётчики	обслуживание
`embedder_signature_upgraded`	`old`, `new`	старая подпись индекса `model_id:dim` переписана в формате с хешем шаблонов

Примеры с `jq`:

```
# сколько секунд занял каждый этап в сумме
jq -s 'map(select(.event=="stage_done")) | group_by(.stage)
       | map({stage: .[0].stage, seconds: ((map(.duration_ms) | add) / 1000)})'
```

```
logs/mnogobase.jsonl

# все ошибки
jq 'select(.level=="error") | {ts, event, doc_id, stage, error_type, error}'
logs/mnogobase.jsonl

# вызовы и токены LLM по задачам
jq -s 'map(select(.event=="llm_call")) | group_by(.task)
       | map({task: .[0].task, calls: length, tokens_in: (map(.tokens_in) | add), tokens_out:
(map(.tokens_out) | add)})' logs/mnogobase.jsonl

# какие сущности слились и как
jq -c 'select(.event=="entity_merged") | {name, into, score, method}' logs/mnogobase.jsonl

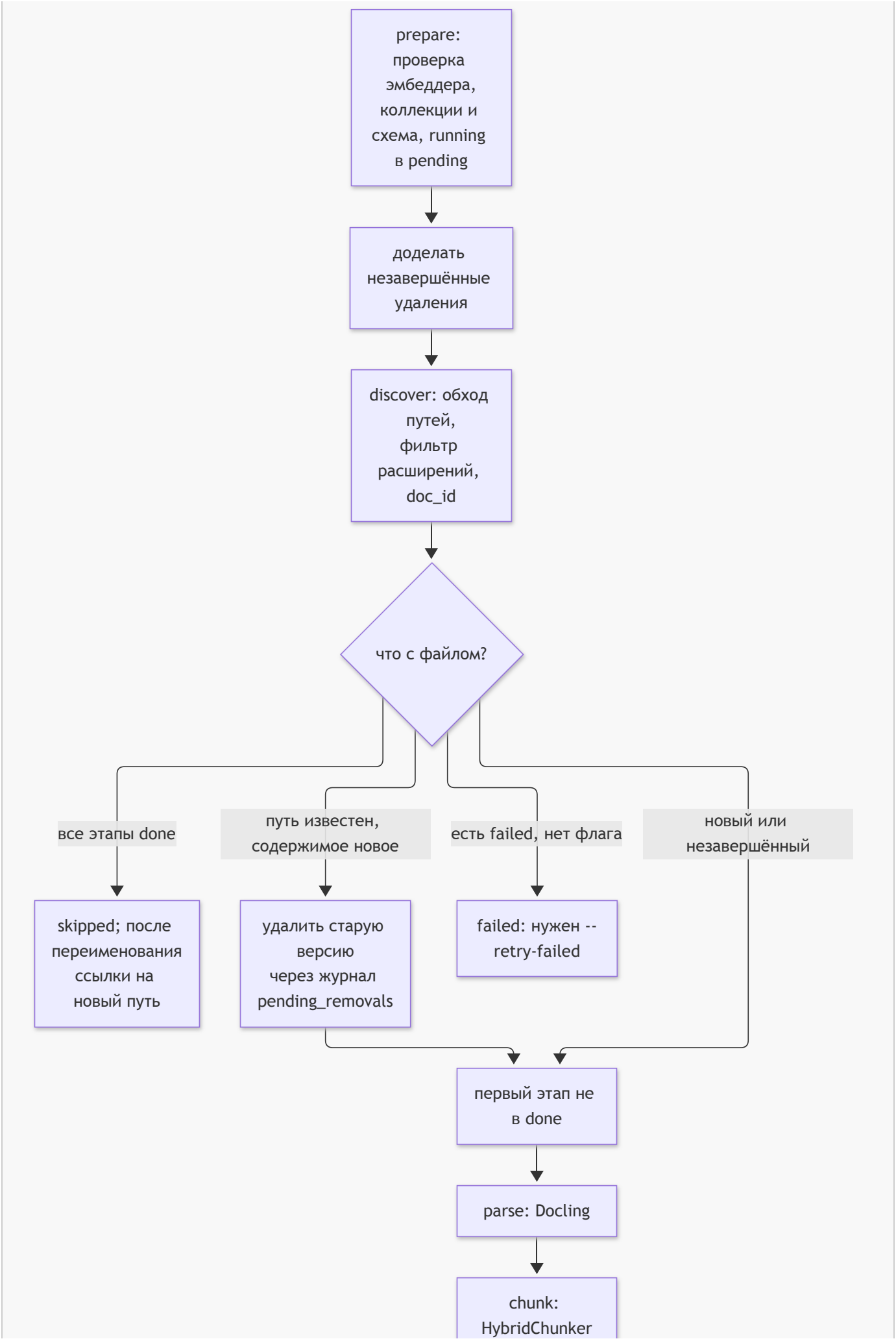
# один прогон (run_id печатается в конце ingest)
jq -c 'select(.run_id=="<run_id>")' logs/mnogobase.jsonl
```

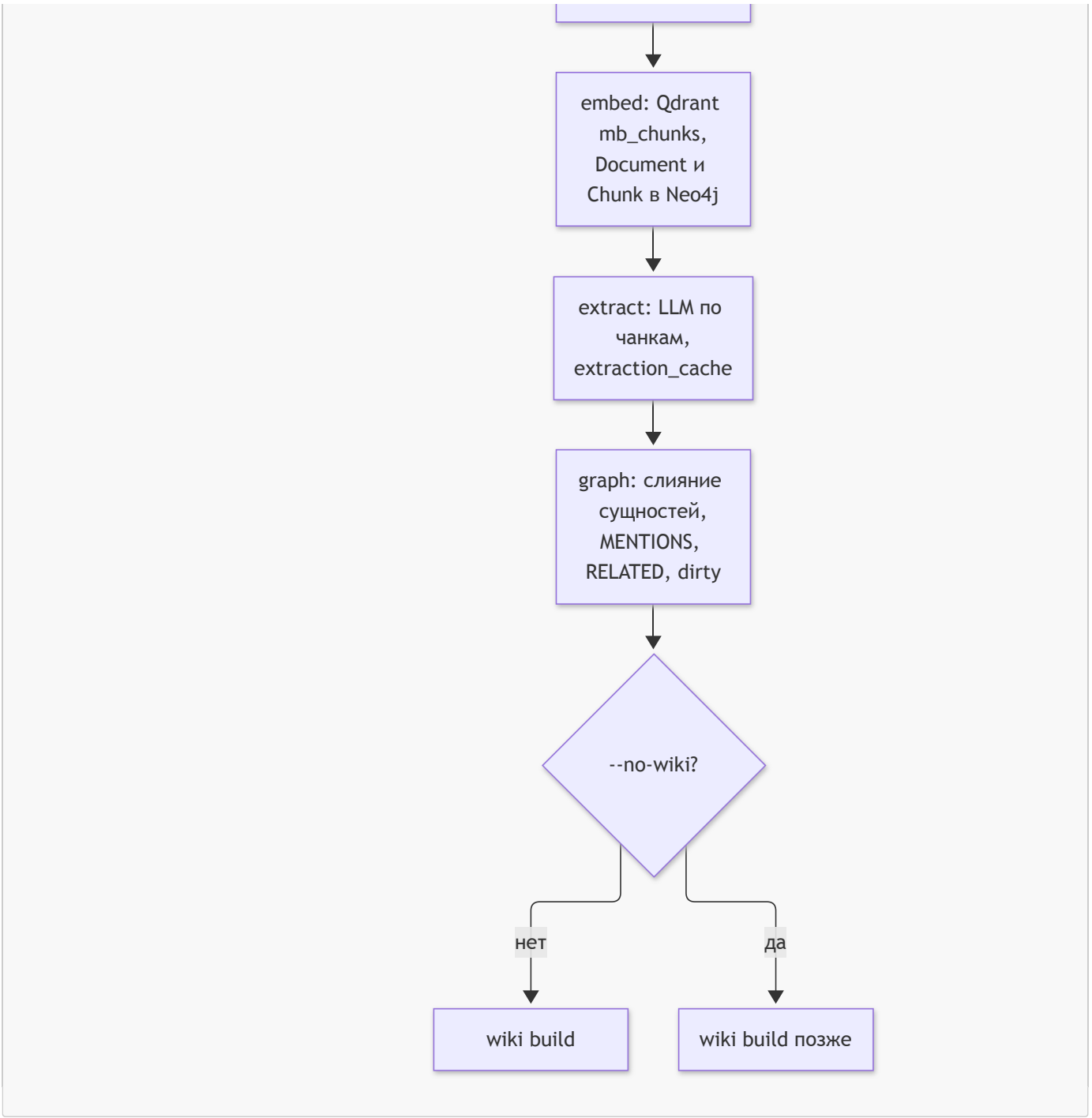
---

5. Как это устроено внутри

5.1 Конвейер ingest

Схема показывает путь каждого файла через этапы: подготовка, обнаружение, при необходимости удаление старой версии, пять этапов и сборка wiki.


prepare: проверка эмбеддера, коллекции и схема, running в pending
доделать незавершённые удаления
discover: обход путей, фильтр расширений, doc_id
что с файлом?
все этапы done
путь известен, содержимое новое
есть failed, нет флага
новый или незавершённый
skipped; после переименования ссылки на новый путь
удалить старую версию через журнал pending_removals
failed: нужен --retry-failed
первый этап не в done
parse: Docling
chunk: HybridChunker



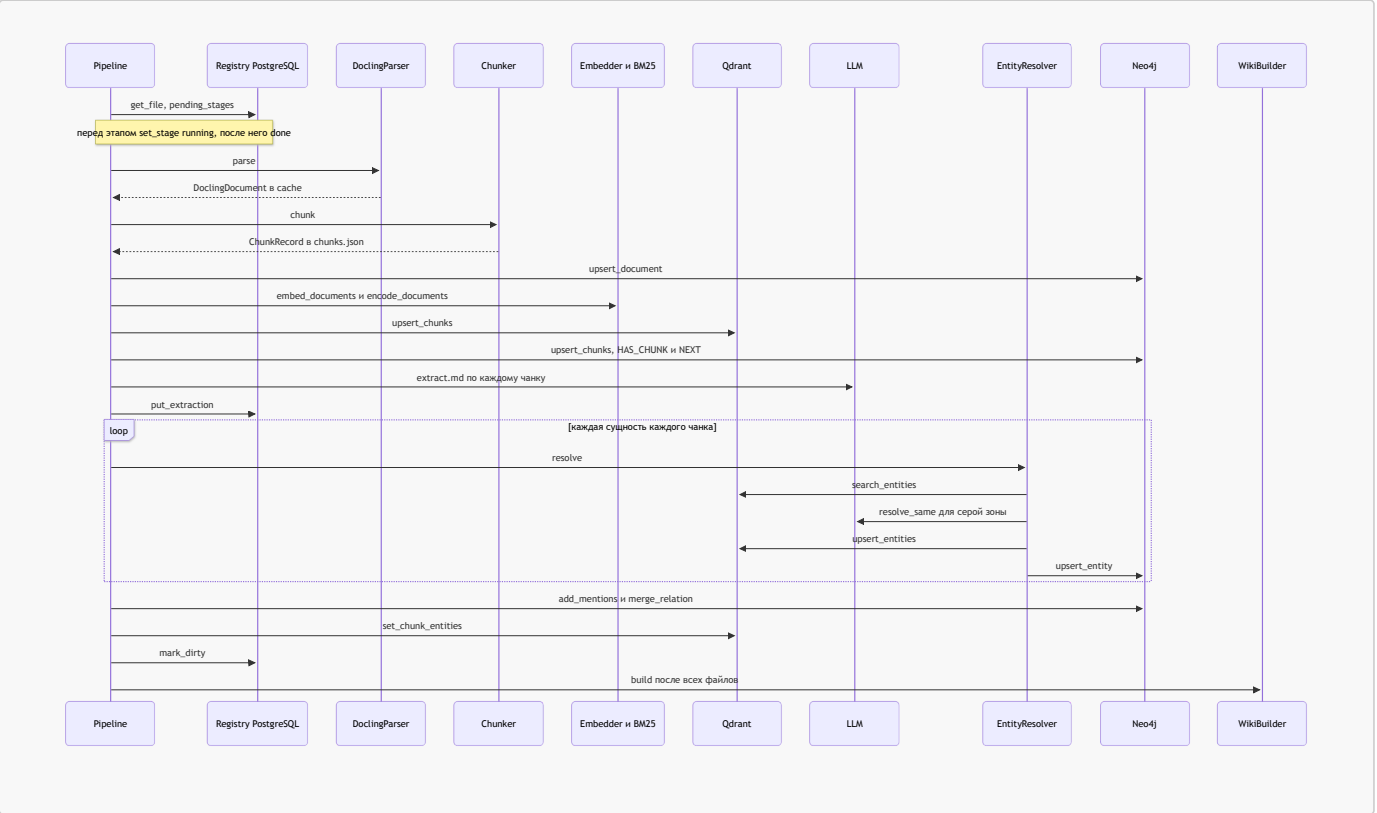
Детали этапов:

- **parse** — `DocumentConverter` с OCR из `parsing.ocr` / `parsing.ocr_lang` и ускорителем из `device` (эти опции заданы для PDF и картинок; на CUDA увеличены размеры батчей). При `vision.enabled` картинки PDF, DOCX и PPTX описывает VLM (раздел 7.16), а файл-картинка описывается ещё и целиком. Заголовок документа — первый элемент Docling с меткой `TITLE`, иначе имя файла.
- **chunk** — `HybridChunker(merge_peers=True)` с токенайзером из `chunking.tokenizer` (`chunking/tokenizer.py`) и пределом `chunking.max_tokens`. При `auto` у `serve: llamacpp` это `POST /tokenize` сервера эмбеддера (`ServerTokenizer` с кэшем): токенайзер зашит в GGUF, исходный HF-репозиторий не нужен (число токенов совпадает с HF-токенайзером, проверено на Qwen3-Embedding-0.6B); у `vllm` — HF-токенайзер `embedder.model`; у `api` — репозиторий из `root` в `/v1/models` (vLLM) или `POST /tokenize` сервера (llama.cpp). Для эмбеддинга используется `contextualize(chunk)` (заголовки раздела и текст), в payload и граф кладётся чистый текст, заголовки и страницы. Пустые чанки отбрасываются.
- **embed** — плотные векторы батчами по `embedder.batch_size` через `POST /embeddings` OpenAI-совместимого сервера, обрезка до `embedder.dim` (при `auto` — без обрезки) и нормировка; BM25 через fastembed. Шаблон задаёт `embedder.doc_template`; в текущем конфиге это `{text}`, где текст чанка уже дополнен заголовками раздела через `contextualize`.

- **extract** — для каждого чанка промпт `extract.md` и строгая JSON Schema `{entities: [{name, type, description, aliases}], relations: [{source, target, predicate, description, strength}]}`. Измерения, координаты, даты и безымянные гипотетические объекты («ring at 144 au», «an unseen planet») промпт просит не делать сущностями. Ответ чистится: неизвестный тип → `Other`, дубли имён сливаются, предикат → `snake_case`, `strength` ограничивается 1–10, связи с неизвестными концами отбрасываются. Результат кэшируется по `(chunk_id, prompt_version, model)`.
- **graph** — по каждому чанку: слияние каждой сущности (раздел 5.5), `MENTIONS` от чанка к сущностям, `entity_ids` в payload чанка, `RELATED` по `(src, dst, predicate)` с дописыванием `evidence` и `weight += strength`. Все затронутые сущности попадают в `dirty_entities`. Если у документа больше `max_failed_ratio` чанков без кэша извлечения, `graph` падает, а `extract` возвращается в `pending`.

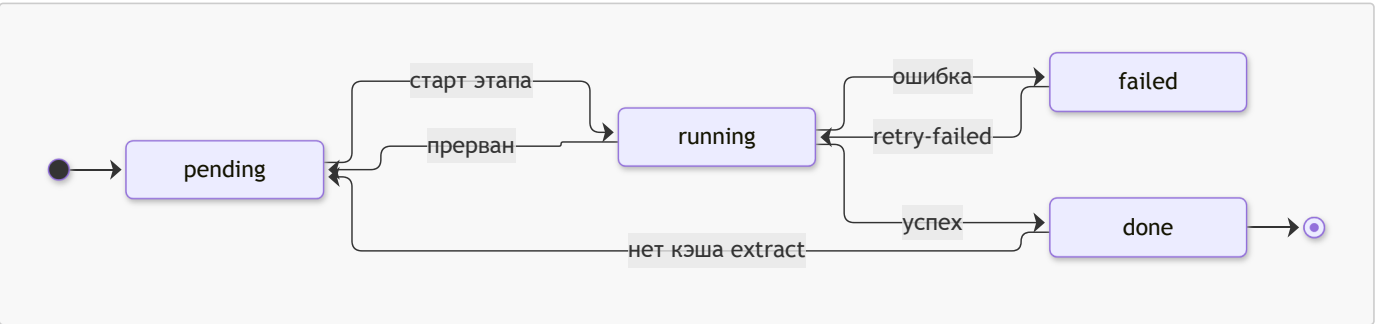
5.2 Загрузка одного файла

Диаграмма последовательности показывает, кто с кем общается при загрузке одного нового файла.



5.3 Состояния этапов

Каждый этап каждого документа хранится в таблице `stages` со статусом; диаграмма показывает переходы, включая возобновление после прерывания и повтор упавших.

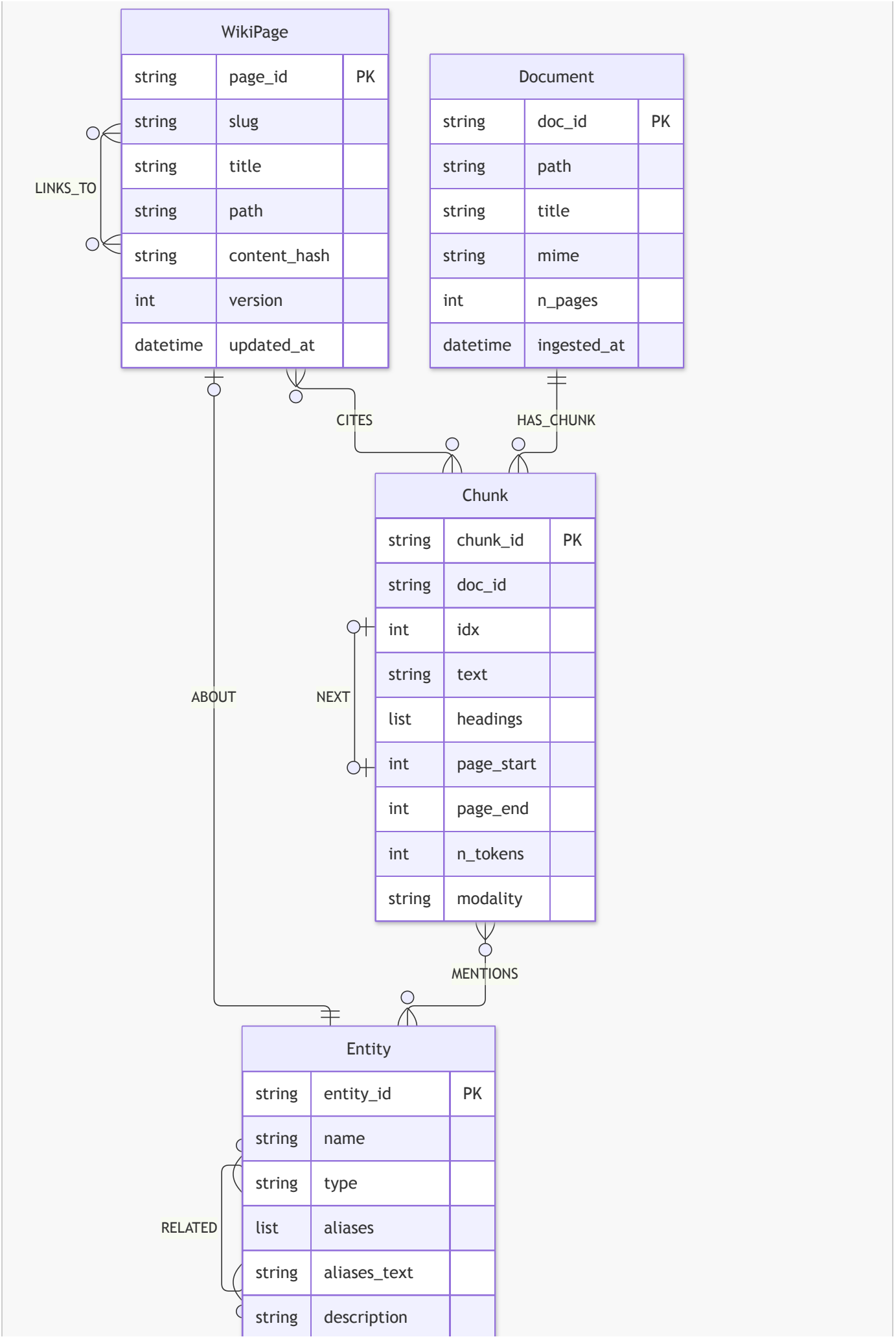


- Статуса нет в таблице → этап считается `pending`. Документ без незавершённых этапов пропускается (`skipped`).
- Этап в `running` при старте следующего `ingest` значит, что процесс прервали: он сбрасывается в `pending` (событие `resuming_interrupted_stages`), и работа продолжается с него.
- `failed` без `--retry-failed` не перезапускается: файл помечается `failed` с подсказкой. С флагом документ продолжается с первого не-`done` этапа; на `extract` повторно вызываются только чанки без кэша.

- Статус файла (`files.status`): `processing` во время работы, затем `done` или `failed`; `pending` — если файл нужно загрузить заново после удаления общей с ним старой версии.

5.4 Модель данных

Neo4j. Узлы и связи с ключевыми свойствами:


WikiPage
string page_id PK
string slug
string title
string path
string content_hash
int version
datetime updated_at
LINKS_TO
Document
string doc_id PK
string path
string title
string mime
int n_pages
datetime ingested_at
CITES
HAS_CHUNK
Chunk
string chunk_id PK
string doc_id
int idx
string text
list headings
int page_start
int page_end
int n_tokens
string modality
ABOUT
NEXT
MENTIONS
Entity
string entity_id PK
string name
string type
list aliases
string aliases_text
string description
RELATED

list	descriptions	
int	mention_count	

- `RELATED` — один тип связи, смысл в свойствах: `predicate` (snake_case), `description` (самое длинное из описаний), `weight` (сумма `strengths`), `evidence` (список `chunk_id`), `strengths` (сила из каждого чанка, параллельно `evidence`).
- `Entity.mention_count` — число чанков с `MENTIONS` на сущность; `descriptions` — накопленные описания, `description` — их склейка; `aliases_text` — алиасы через `|` для полнотекстового индекса.
- `WikiPage.page_id` равен `entity_id`, `path` — путь относительно `wiki/` (`entities/<slug>.md`).
- Ограничения уникальности: `Document.doc_id`, `Chunk.chunk_id`, `Entity.entity_id`, `WikiPage.page_id`. Индексы: `WikiPage.slug`, `Chunk.doc_id`, полнотекстовый `entity_names` по `Entity.name` и `Entity.aliases_text`. Схема создаётся `IF NOT EXISTS` при каждом запуске (`GraphStore.ensure_schema`).

Qdrant. Префикс коллекций — `qdrant.prefix` (`mb_`). Плотный вектор называется `dense` (косинус, размерность `embedder.dim` или, при `auto`, модели), разреженный — `bm25` (модификатор IDF). Все индексируемые поля payload — keyword.

Коллекция	Векторы	Индексы payload	Что эмбеддится	ID точки
`mb_chunks`	`dense`, `bm25`	`chunk_id`, `doc_id`, `entity_ids`, `modality`	заголовки раздела и текст чанка	`uuid5(chunk_id)`
`mb_entities`	`dense`	`entity_id`, `type`, `name_keys`	`"{name}: {description}"`	`uuid5(entity_id)`
`mb_wiki_pages`	`dense`, `bm25`	`page_id`, `entity_id`	раздел страницы без сносок, с заголовком `Название — Раздел`	`uuid5("{page_id}#{номер раздела}")`

Детерминированные ID (`src/mnogobase/ids.py`) — на них держатся идемпотентность и дедупликация:

Объект	ID
Документ	`doc_id = sha256(байты файла)[:16]` — одинаковое содержимое по разным путям даёт один документ
Чанк	`chunk_id = f"{doc_id}:{idx:05d}"`
Сущность	`entity_id = sha1(f"{type.casefold()}|{normalize_name(name)}")[:16]`; `normalize_name`: NFKC, casefold, все разделители кроме `+` и `#` → пробел, пробелы схлопываются
Wiki-страница	`page_id = entity_id`; `slug` = нормализованное имя через дефис (`#` → `sharp`, длинные имена обрезаются с хешем), при занятости `slug-<тип>`, затем `slug-<6 символов id>`
Точка Qdrant	`uuid5(NAMESPACE, ключ)`, ключ — `chunk_id`, `entity_id` или `page_id#i`

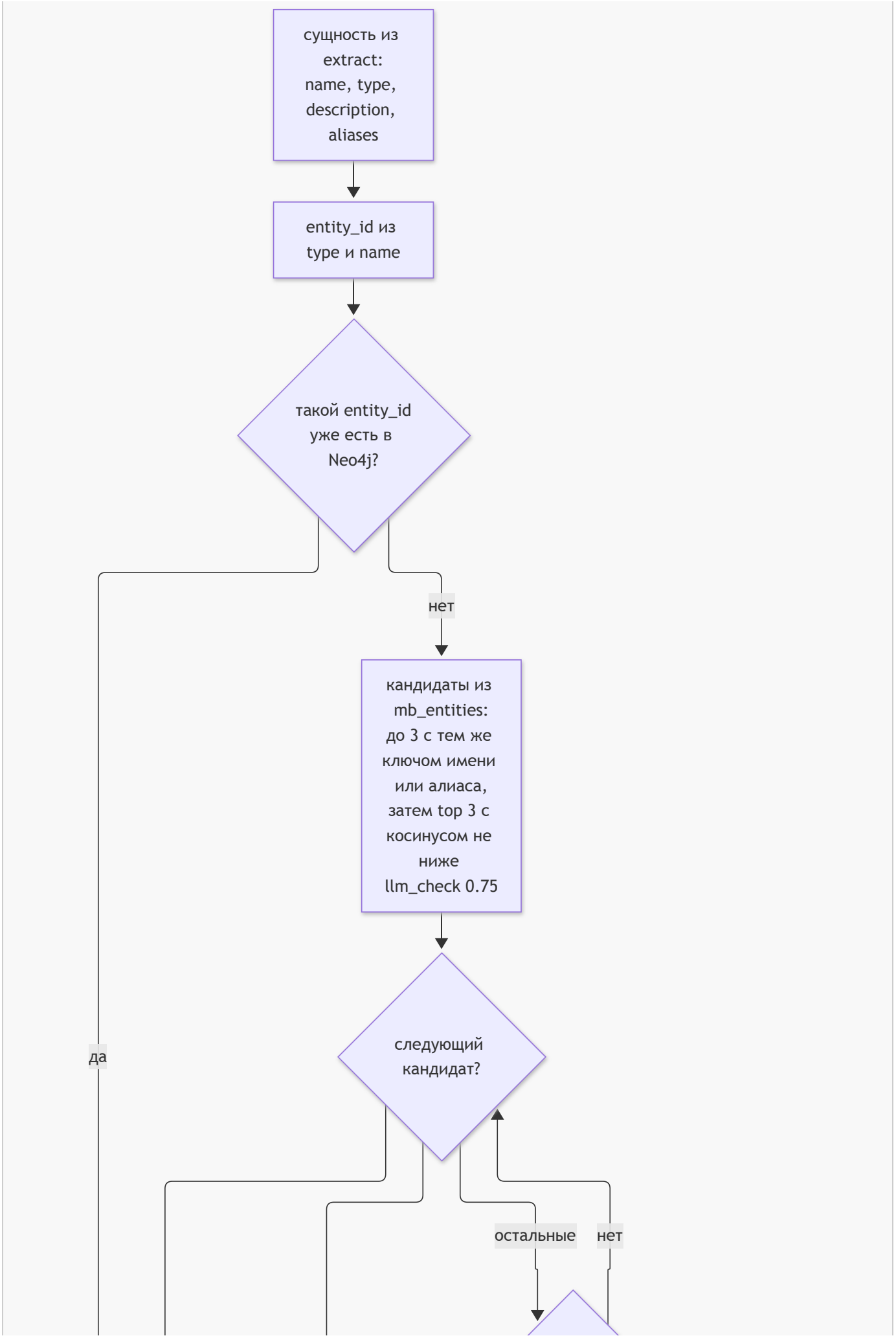
Registry PostgreSQL (схема `mnogobase_project_default`):

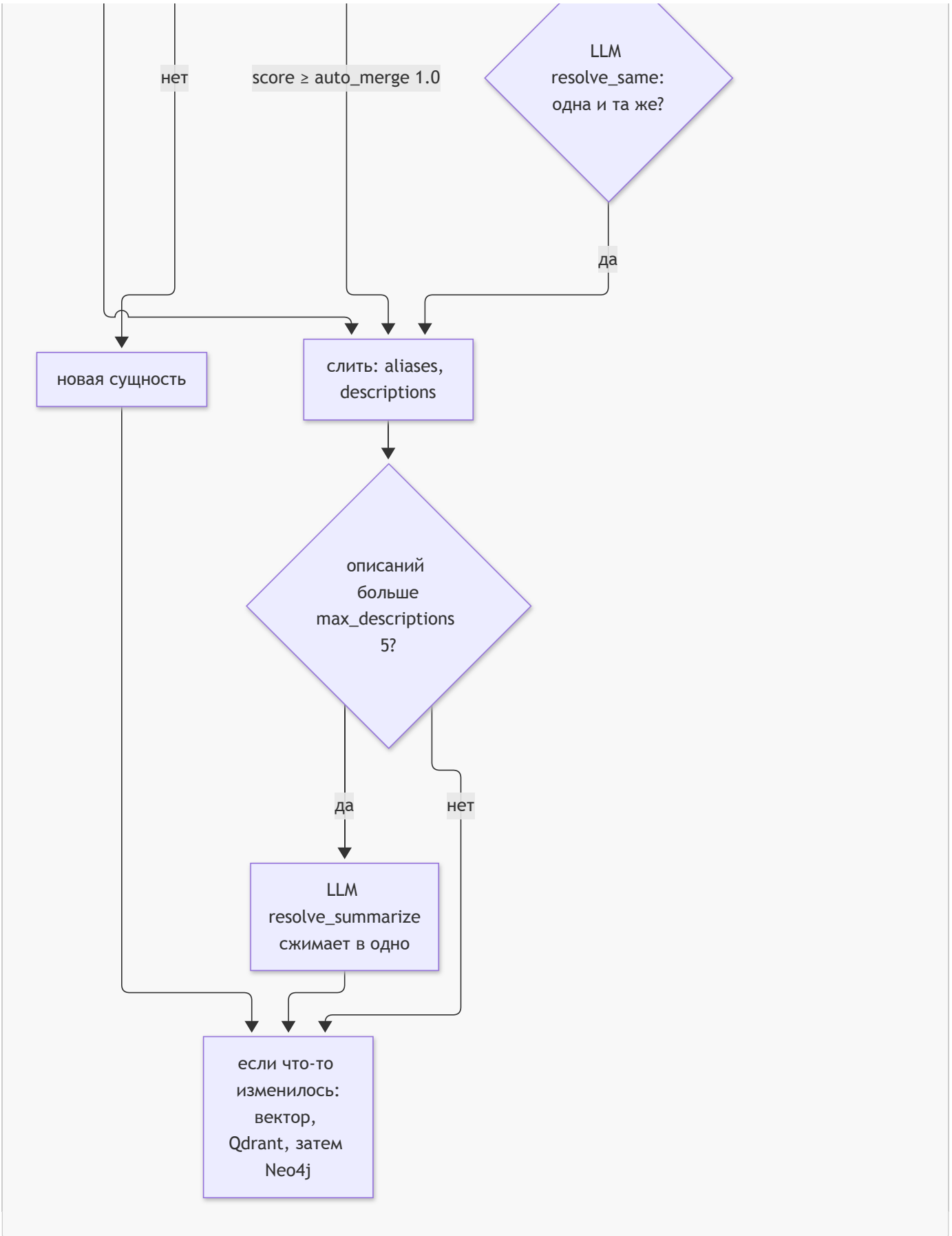
Таблица	Колонки	Зачем
`files`	`path` PK, `doc_id`, `size`, `mtime`, `status`, `updated_at`	какие пути загружены и с каким содержимым
`stages`	`doc_id`, `stage`, `status`, `attempts`, `error`, `updated_at`	состояние этапов `parse`, `chunk`, `embed`, `extract`, `graph`
`chunk_extract`	`chunk_id` PK, `status`, `attempts`, `error`	результат извлечения по чанкам

Таблица	Колонки	Зачем
`extraction_cache`	`chunk_id`, `prompt_version`, `model`, `result_json`	кэш ответов LLM на этапе `extract`
`dirty_entities`	`entity_id` PK, `marked_at`	очередь сущностей на обновление wiki
`meta`	`key` PK, `value`	подпись эмбеддера индекса (`embedder` = `<serve>:<веса>:<dim>:tpl-<хеш шаблонов>`, например `llamacpp:Qwen/Qwen3-Embedding-4B-GGUF:Q4_K_M:2560:tpl-...`) и служебные значения
`pending_removals`	`doc_id` PK, `plan_json`	журнал незавершённых удалений старых версий документов

5.5 Слияние сущностей (entity resolution)

Каждая извлечённая сущность либо совпадает с уже известной, либо сливается с похожей, либо создаётся заново:


сущность из extract: name, type, description, aliases
entity_id из type и name
такой entity_id уже есть в Neo4j?
нет
да
кандидаты из mb_entities: до 3 с тем же ключом имени или алиаса, затем top 3 с косинусом не ниже llm_check 0.75
следующий кандидат?
остальные
нет

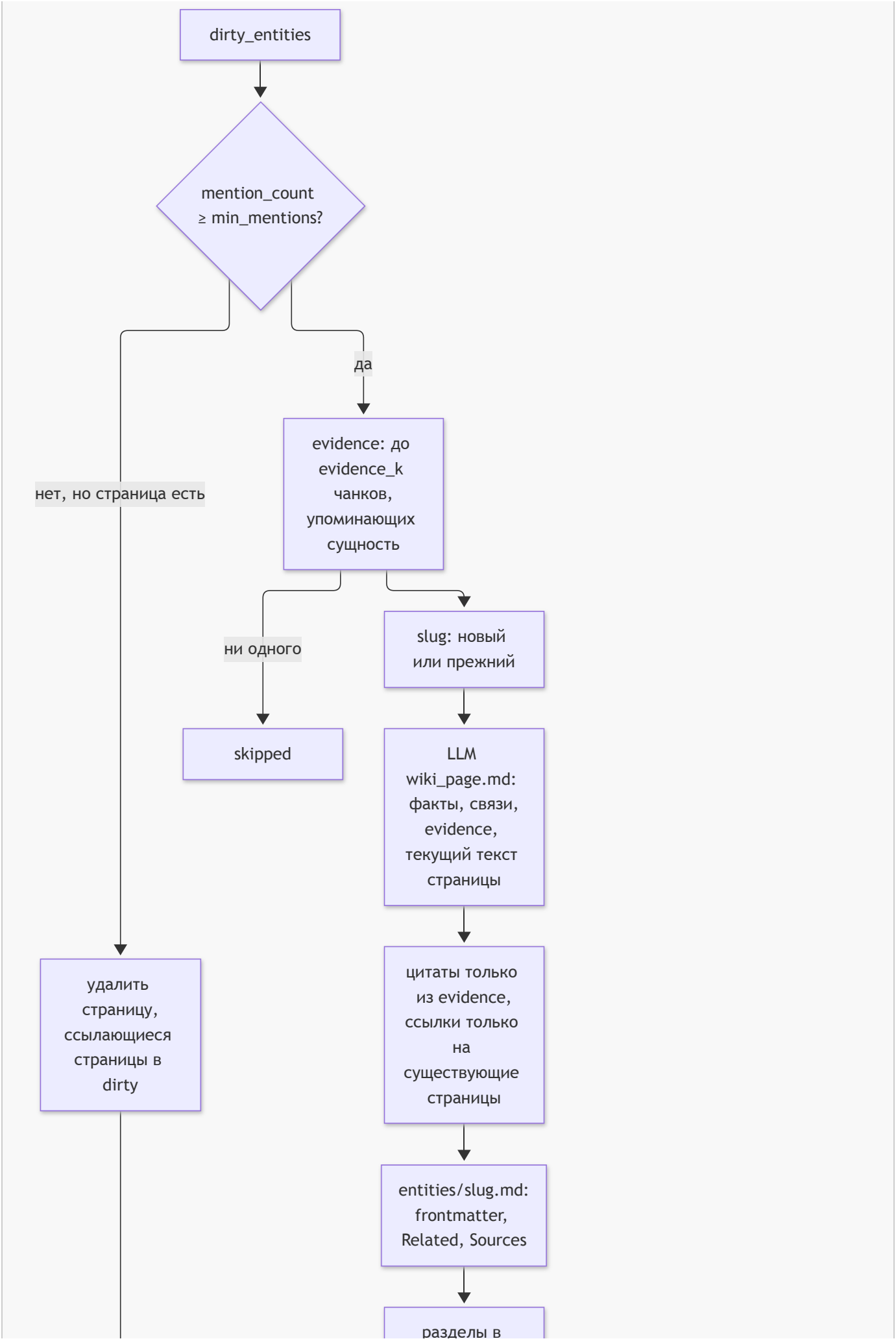


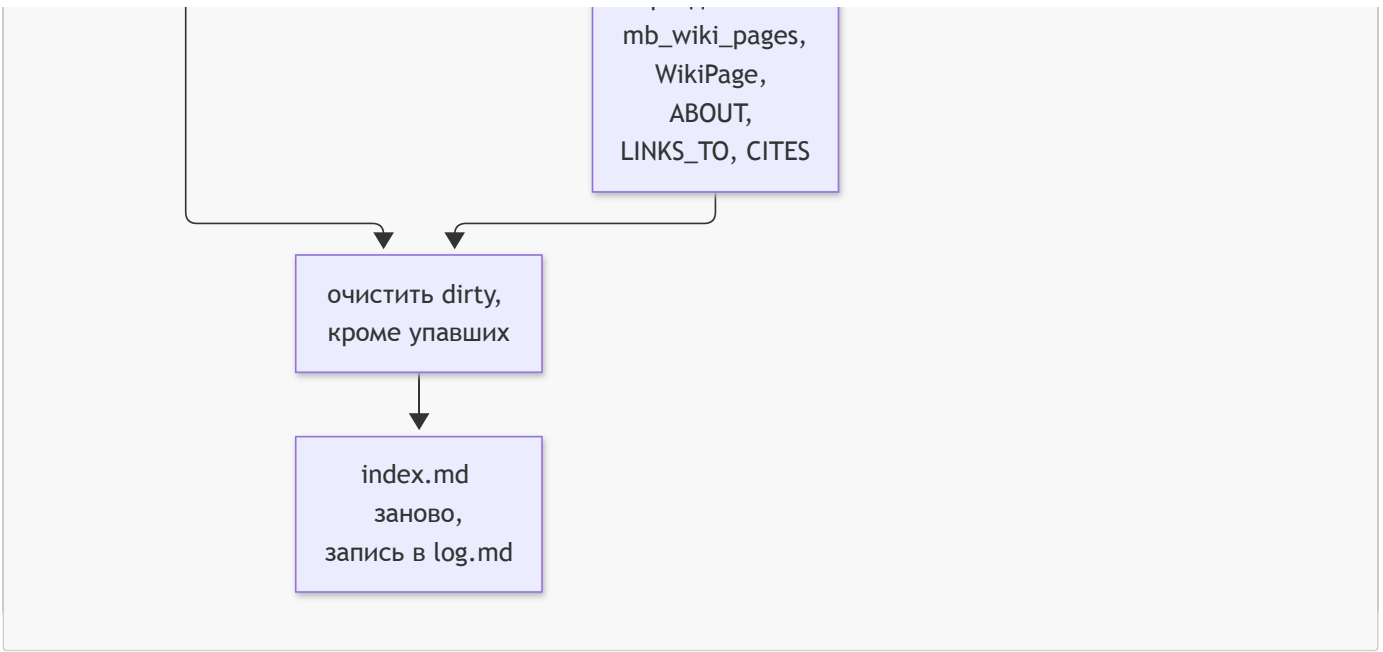
- Имена извлекаются как канонические английские, исходное написание уходит в `aliases` — поэтому русский и английский тексты обычно дают один `entity_id` сразу, а вектор и LLM ловят остальные случаи (синонимы, сокращения, варианты написания).
- При слиянии сохраняются имя и тип существующей сущности, новое имя становится алиасом. Сопоставление `entity_id` извлечённой сущности с итоговым запоминается на время процесса, чтобы не искать заново.

- Поиск идёт без фильтра по типу. Ключ имени — `normalize_name` без пробелов (`MagAO-X` = `MagAOX`, `Semi-major axis` = `Semimajor axis`); он считается от имени и всех алиасов и хранится в payload `name_keys`. Кандидаты по ключу проверяются LLM при любом косинусе: одна сущность под другим типом или с другим описанием (`Sun` как Location и как Other, `NASA` и `National Aeronautics and Space Administration`) часто даёт косинус 0.5–0.75.
- По умолчанию сущности не сливаются по одному косинусу (`auto_merge: 1.0`), см. раздел 7.12.

5.6 Инкрементальное обновление wiki

Сборка wiki обрабатывает только сущности из очереди `dirty_entities` (или все при `--all`):

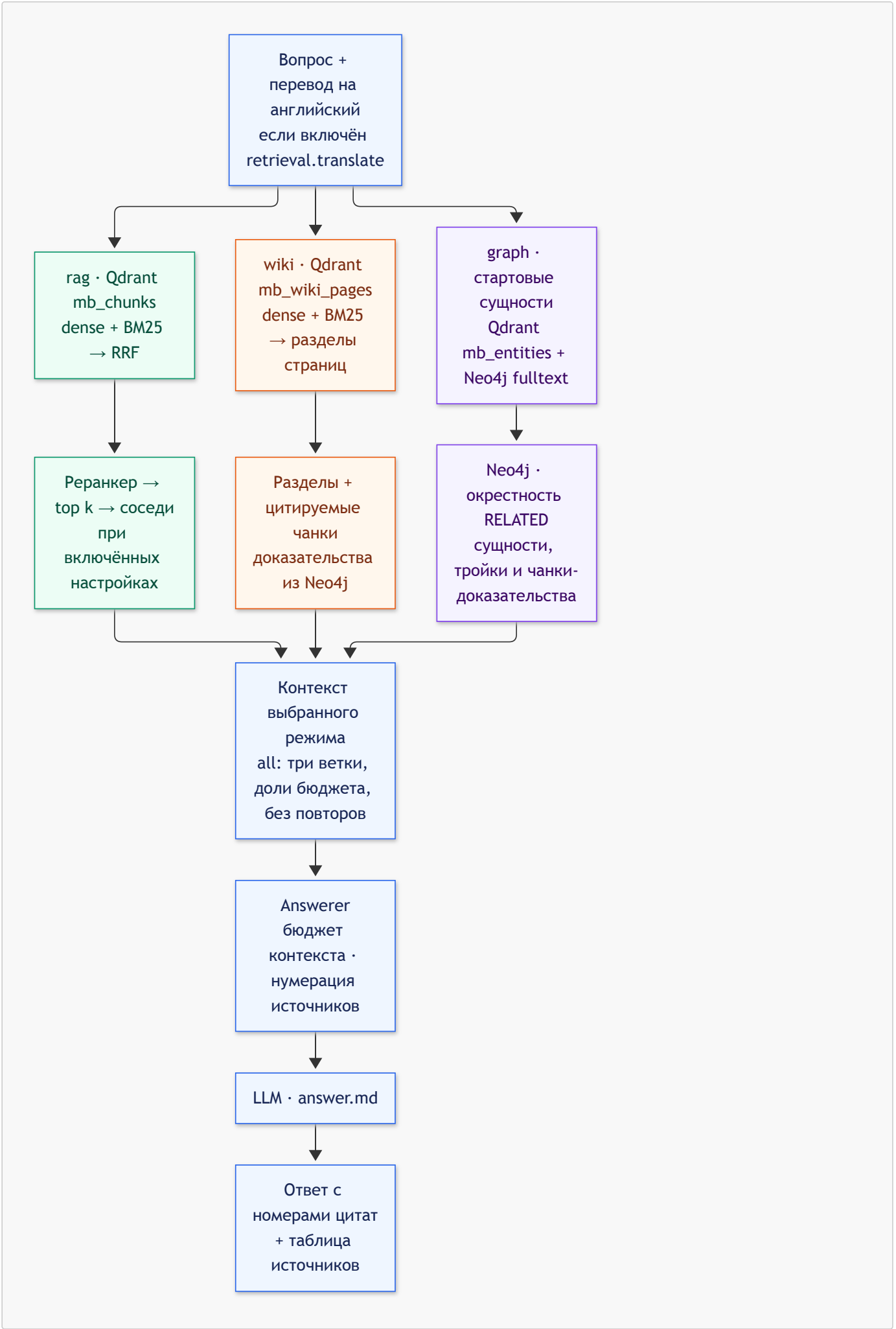

dirty_entities
mention_count ≥ min_mentions?
нет, но страница есть
да
evidence: до evidence_k чанков, упоминающих сущность
ни одного
slug: новый или прежний
skipped
LLM wiki_page.md: факты, связи, evidence, текущий текст страницы
удалить страницу, ссылающиеся страницы в dirty
цитаты только из evidence, ссылки только на существующие страницы
entities/slug.md: frontmatter, Related, Sources
разделы в



- **Evidence** — чанки, где упомянута сущность (фильтр `entity_ids` в `mb_chunks`), ранжированные по близости к вектору `"{name}: {description}"`, не больше `wiki.evidence_k` (12). При обновлении к ним добавляются чанки, которые уже цитирует текущая страница, если они существуют и всё ещё упоминают сущность: валидные цитаты не теряются, когда чанк выпал из свежего top-K.
- **Обновление, а не переписывание.** LLM получает текущий текст страницы и правило «сохранить подтверждённое, добавить новое, исправить противоречия».
- **Валидация.** Сноски `[^id]` на чанки вне evidence удаляются. `[[Name]]` превращается в `[[slug|Name]]`, только если у Name (по имени или алиасу) есть страница или она пишется в этом же прогоне, иначе остаётся текстом. Разделы `## Related` и `## Sources`, заголовок и определения сносок генерируются кодом, а не LLM.
- **Надёжность.** Ошибка одной страницы не валит сборку: сущность остаётся в очереди и повторяется в следующий раз. `index.md` пересобирается целиком, в `log.md` дописывается запись, если что-то изменилось или были обработаны документы.

5.7 Поиск и ответ

Четыре режима собирают контекст по-разному, а ответ формирует общий `Answerer`:


Вопрос + перевод на английский если включён retrieval.translate
rag · Qdrant mb_chunks dense + BM25 → RRF
wiki · Qdrant mb_wiki_pages dense + BM25 → разделы страниц
graph · стартовые сущности Qdrant mb_entities + Neo4j fulltext
Реранкер → top k → соседи при включённых настройках
Разделы + цитируемые чанки доказательства из Neo4j
Neo4j · окрестность RELATED сущности, тройки и чанки-доказательства
Контекст выбранного режима all: три ветки, доли бюджета, без повторов
Answerer бюджет контекста · нумерация источников
LLM · answer.md
Ответ с номерами цитат + таблица источников

- **Перевод запроса.** Если вопрос не на английском и `retrieval.translate: true`, `QueryTranslator` (`retrieval/translate.py`, промпт `translate_query.md`, задача `query`) один раз переводит его на английский; все режимы ищут и по оригиналу, и по переводу (`alt_queries`). Источники, wiki и имена сущностей в основном английские, а BM25 не сопоставляет русский запрос с английским текстом. Ошибка перевода не прерывает ответ: поиск идёт по оригиналу.
- **rag** — гибридный запрос Qdrant: по два prefetch по `k * 4` (`dense` и `bm25`) на каждую формулировку, слияние RRF, `k` результатов. Текст каждого чанка начинается с пути заголовков (`Глава > Раздел`). При `retrieval.neighbors: N > 0` попадание расширяется до N соседних чанков с каждой стороны, пока у них те же заголовки (тот же раздел); попадание, уже вошедшее в окно более сильного, пропускается. Цитата указывает на чанк-попадание.
- **Реранкер** (`rerank.enabled`, `retrieval/rerank.py`). В режиме rag гибридный поиск отдаёт `rerank.candidates` попаданий, Qwen3-Reranker оценивает каждое (путь заголовков + текст) против исходного вопроса, и остаются лучшие `k`; `ContextItem.score` — оценка реранкера (0–1). Все кандидаты уходят одним запросом `POST /rerank` (формат Jina/Cohere), промпт Qwen3-Reranker строит сервер: llama.cpp — из GGUF, vLLM — с `--chat-template`. Если реранкер недоступен, в лог пишется `rerank_failed` и используется порядок гибридного поиска.
- **wiki** — то же по разделам wiki; после каждого раздела идут чанки из его `chunk_ids` (всего не больше `k`), чтобы ответ ссылался на файл и страницу.
- **graph** — seeds: до `graph.seeds` сущностей по вектору вопроса с косинусом не ниже `graph.seed_threshold` плюс полнотекстовый поиск по именам и алиасам, по каждой формулировке вопроса (всего не больше `2 * seeds`; векторные seeds — по убыванию сходства). Окрестность на `graph.hops` шагов (в коде ограничено 1–3), до `graph.max_relations` связей; контекст — описания seed-сущностей, тройки `A —predicate→ B` и до `k` чанков-доказательств.
- **all** — режимы по очереди заполняют свою долю `retrieval.context_tokens` (`retrieval.budget`), повторы по `(kind, ref)` пропускаются.
- **Answerer** нумерует элементы `[1]..[n]`, пока не исчерпан `retrieval.context_tokens` (первый элемент берётся всегда; токены оцениваются как символы / 4), отправляет промпт `answer.md` и отмечает `cited` у источников, номера которых встречаются в ответе.

На схеме показаны возможные ветки: `ask -m rag/wiki/graph` выполняет выбранную, `all` объединяет три. Наличие номера цитаты означает, что модель сослалась на источник; соответствие утверждения источнику отдельно оценивается метриками и ручной проверкой.

5.8 APOC и GDS

Что это даёт пользователю. APOC — библиотека процедур Neo4j: в нашем поиске она управляет обходом связей вокруг найденных сущностей. Собственной численной оценки APOC в этом режиме не вычисляет. GDS (Graph Data Science) — библиотека графовых алгоритмов: PageRank оценивает центральность сущностей, Leiden объединяет их в сообщества по структуре связей.

Результат	Как понимать	Где посмотреть
`pagerank`	насколько сущность центральна в анализируемом графе; больше — выше центральность. Это не вероятность правильного ответа и не процент качества. Сравнивайте сущности внутри одного снимка анализа	`graph status`, поле `entities`; полный рейтинг — через Neo4j Browser ниже
`community`	ключ сообщества вида `community:…`; одинаковый ключ означает принадлежность к одной группе. Это не название темы и не оценка качества группы	`graph status`; состав и размеры сообществ — через Browser
`ready`	`true` — есть опубликованный актуальный снимок; `false` — снимка нет или он сброшен после изменения данных	`graph status`
`completed_at`	время последнего успешного анализа; при `ready: false` прежнее время само по себе не означает актуальность	`graph status`
`metadata.nodes`, `relationships`, `communities`	число сущностей, исходных связей и сообществ в последнем расчёте. При `ready: false` это могут быть исторические значения	`graph status`

APOC позволяет обходить `RELATED` в ширину: в проектном `config.yaml` включён `graph.traversal: apoc`. Глубина ограничена тремя шагами, список разрешённых узлов — только Entity текущего проекта/команды. При превышении

`graph.analysis.apoc_max_nodes` или отсутствии процедуры выполняется обычный Cypher-обход. Для самого APOC-запроса задан таймаут; ошибки соединения и прочие ошибки сервера не маскируются. Оба варианта возвращают исходные `evidence` и чанки для цитирования.

Как пользоваться. Из корня уже проиндексированного проекта выполните оценку памяти, затем анализ и просмотр результата. GDS запускается отдельно, без эмбеддера, Qdrant и LLM:

```
uv run mnogobase graph analyze --estimate  # оценить память проекции, без её создания
uv run mnogobase graph analyze             # PageRank + Leiden, сохранить результат
uv run mnogobase graph status              # актуальность, версии, топ сущностей и их
сообщества
uv run mnogobase graph clear --yes         # очистить только производную аналитику
# Для команды: глобальный --team идёт перед graph, как --config.
uv run mnogobase --team <team_id> graph analyze
uv run mnogobase --team <team_id> graph status
```

Команды требуют существующий проект. `analyze` и `clear` используют общую блокировку GDS и существующую блокировку писателя корпуса. Проекция включает только Entity и RELATED данного scope, как взвешенный ненаправленный граф. PageRank оценивает структурную центральность, Leiden — сообщества; числовые номера сообществ заменяются хешами их состава. Поле `community` у сущности в `status` позволяет увидеть её группу; вывод ограничен 20 центральными сущностями.

Как читать оценки. `graph status` печатает JSON. Ниже сокращённый учебный пример; имена, числа и ключи иллюстративные:

```
{
  "ready": true,
  "completed_at": "2026-11-02T20:00:00Z",
  "metadata": {
    "nodes": 12,
    "relationships": 18,
    "communities": 3
  },
  "entities": [
    {
      "entity_id": "example-transformer",
      "name": "Transformer",
      "pagerank": 2.31,
      "community": "community:0a1b2c3d4e5f678901234567"
    },
    {
      "entity_id": "example-attention",
      "name": "Attention",
      "pagerank": 1.42,
      "community": "community:0a1b2c3d4e5f678901234567"
    }
  ]
}
```

В примере Transformer центральнее Attention; обе сущности попали в одно сообщество. По этим числам нельзя заключить, что сведения о Transformer точнее или полезнее для любого вопроса. CLI показывает только топ-20, поэтому отсутствие сущности в списке не означает, что её нет в графе или что она не получила оценку. Версии плагинов в `metadata.plugins` относятся к последнему расчёту; текущие установленные версии проверяет `uv run mnogobase doctor`.

Как посмотреть больше в Neo4j Browser. Откройте http://localhost:7474 и подключитесь к базе, как в разделе 4.1. Следующие запросы предназначены для обычного проекта с labels `Entity` и `AnalysisState`. Для проекта команды замените **оба** label на `mb_team_<team_id>_Entity` и `mb_team_<team_id>_AnalysisState`, подставив действительный ID команды. В CLI нужный scope выбирается через глобальный `--team`.

Топ-100 сущностей с оценками:

```
MATCH (s:AnalysisState {key: 'graph'})
WHERE s.run_id IS NOT NULL
MATCH (e:Entity)
WHERE e.analytics_run_id = s.run_id
RETURN e.entity_id AS entity_id, e.name AS name,
       e.analytics_pagerank AS pagerank, e.analytics_community AS community
ORDER BY pagerank DESC, entity_id
LIMIT 100;
```

Крупнейшие сообщества, их размеры и примеры участников:

```
MATCH (s:AnalysisState {key: 'graph'})
WHERE s.run_id IS NOT NULL
MATCH (e:Entity)
WHERE e.analytics_run_id = s.run_id
RETURN e.analytics_community AS community, count(*) AS members,
       collect(e.name)[..10] AS examples
ORDER BY members DESC, community
LIMIT 20;
```

Участники конкретного сообщества — замените ключ в кавычках значением из предыдущего запроса:

```
MATCH (s:AnalysisState {key: 'graph'})
WHERE s.run_id IS NOT NULL
MATCH (e:Entity)
WHERE e.analytics_run_id = s.run_id
  AND e.analytics_community = 'community:0a1b2c3d4e5f678901234567'
RETURN e.entity_id AS entity_id, e.name AS name, e.analytics_pagerank AS pagerank
ORDER BY pagerank DESC, entity_id
LIMIT 100;
```

Эти запросы сверяют оценки с опубликованным снимком. Если результат пуст и `ready: false`, повторите `graph analyze`: у отдельных узлов после изменения корпуса могут оставаться старые свойства аналитики, но поиск их не использует. `graph clear --yes` удаляет производные оценки и временные проекции этого scope; документы, связи, wiki и векторы остаются.

Как оценки влияют на ответ. Чтобы использовать графовый поиск, выполните `uv run mnogobase ask "Ваш вопрос" -m graph` или `-m all`. Для ответа нужны настроенные серверы моделей/API, как и до подключения GDS.

В `graph`/`all` при `graph.use_gds: true` готовый PageRank разрывает равенство весов **уже найденных** связей по сумме оценок концов. Он не добавляет кандидатов вне окрестности и не доказывает истинность утверждения. Если снимка нет или он устарел, сохраняется исходный порядок. Изменения документов, чанков, сущностей, упоминаний и связей, а также удаление документа инвалидируют анализ в той же Neo4j-транзакции. Публикация нового снимка атомарна и проверяет ревизию исходного графа. Алгоритмы не запускаются при `ask`; после изменений корпуса повторите `graph analyze`.

Как Leiden помогает генерации. В проекте включён `graph.community.enabled: true`. После поиска стартовых сущностей `GraphRetriever` выбирает их сообщества из актуального анализа и добирает исходные чанки, упоминающие участников этих групп. Так LLM получает сведения за пределами обычного `graph.hops`, а цитаты продолжают вести к документам и страницам. Одинаковое сообщество означает структурную близость, поэтому дополнительный фрагмент может оказаться менее полезным для конкретного вопроса; центральность не заменяет проверку фактов по тексту источника.

По умолчанию берутся до 3 сообществ по порядку стартовых сущностей, до 30 участников каждого по PageRank и до 3 дополнительных чанков. Внутри сообщества чанки упорядочены по числу найденных участников и их центральности; повторы исключаются. Общий лимит исходных чанков остаётся `retrieval.k`: если есть локальные подтверждения связей, расширению доступно не больше половины этого лимита, остальные места заполняются локальными. При `k=1` и локальном подтверждении дополнительного чанка не будет. Если связей нет, актуальное сообщество всё равно может дать исходный

текст через упоминания сущностей. При добавлении таких источников чанки идут перед описаниями сущностей и связей, чтобы они имели приоритет в контекстном бюджете.

Запрос ограничен `graph.community.timeout_s` (5 секунд). При его таймауте или отсутствии актуального анализа остаётся обычный графовый контекст. В логах событие `leiden_context` показывает число добавленных чанков; `leiden_context_timeout` означает пропуск расширения. Источники ответа содержат путь и страницу исходного фрагмента. В режиме `all` дополнительно действует общий бюджет графовой части, поэтому не каждый найденный чанк попадёт в prompt.

Чтобы отключить только расширение сообществ, установите `graph.community.enabled: false`. `graph.use_gds: false` отключает и PageRank, и Leiden при поиске; для обычного Cypher-обхода также задайте `graph.traversal: cypher`. Переиндексация не нужна.

Leiden не пишет текст самостоятельно: готовые группы помогают собрать контекст, после чего настроенная LLM генерирует ответ. Дополнительных LLM-вызовов для сводок здесь нет. Обзоры сообществ для глобальных вопросов остаются отдельным возможным расширением — см. GraphRAG Global Search. Реализация описана в плане Leiden, результаты запусков — в отчёте проверки.

Начальные лимиты: 100 000 узлов, 500 000 исходных рёбер, 1536 МиБ оценки памяти GDS, 120 секунд на запрос. Перед проекцией проверяется её оценка памяти, после создания — оценки алгоритмов; учитываются проекция и наибольшая рабочая память последовательных алгоритмов. При исключении временная проекция удаляется; после аварийного завершения её убирает следующий `graph analyze` или `graph clear` этого scope. Оценки памяти не гарантируют защиту от OOM.

`doctor` отдельно проверяет версии APOC/GDS и необходимые процедуры (`graph_plugins`); отсутствие плагинов даёт предупреждение, обычные режимы остаются доступны. `graph analyze` требует GDS и сообщает ошибку при его отсутствии. HTTP healthcheck контейнера не проверяет загрузку плагинов. Runtime-установка GDS требует доступа к `graphdatascience.ninja`; при сбое загрузки Neo4j может стартовать без GDS. Для production фиксируйте совместимые Neo4j/GDS и сохраняйте проверенные JAR в собственном образе или каталоге плагинов.

Автоматические wiki-обзоры сообществ — отдельный будущий этап; текущая реализация аналитики дополнительных LLM-запросов не делает.

5.9 Гарантии согласованности

- **Идемпотентность.** Все записи — upsert/`MERGE` по детерминированным ID, поэтому повтор этапа после падения даёт тот же результат. `merge_relation` не добавляет `evidence` из того же чанка дважды (вес не удваивается), `mention_count` пересчитывается подсчётом связей.
- **Порядок записи.** Этап отмечается `done` только после всех записей. На `embed`: узел `Document` → точки Qdrant → узлы `Chunk`. При слиянии сущности: вектор → Qdrant → Neo4j, чтобы сбой эмбеддера или Qdrant не оставил граф «впереди».
- **Изменённый файл.** План удаления старой версии вычисляется только чтением и записывается в журнал `pending_removals`. Затем: одна транзакция Neo4j (чанки, документ, `evidence` из его чанков, связи без `evidence`, сущности без упоминаний и их страницы) → очередь wiki → Qdrant и файлы wiki → registry → запись журнала удаляется последней. Граф чистится первым намеренно: если шаг Qdrant не удался, режимы `rag`/`wiki`/`all` до следующего `ingest` или `wiki build` (они доводят журнал) могут вернуть текст удалённой версии, режим `graph` уже чист. Незавершённое удаление перепроверяется по графу, а если довести его не удаётся, `ingest` и `wiki build` завершаются с кодом 2.
- **Одинаковое содержимое по двум путям** — один документ. Второй путь пропускается; изменение одного из файлов не удаляет данные, пока другой путь хранит то же содержимое (ссылки в источниках переводятся на живой путь). Живой путь — тот, где файл существует и его хеш совпадает с `doc_id`: строка registry для переименованного или удалённого файла копией не считается.
- **Переименование.** Файл с новым путём и прежним содержимым пропускается, а если записанный путь документа (кэш чанков или `Document.path` в Neo4j; кэша может не быть) больше не хранит это содержимое, документ переводится на новый путь: сначала `path` в payload чанков Qdrant, затем `Document.path`, кэш чанков последним, поэтому прерванный перевод доделывает следующий `ingest`. Wiki-страницы, которые цитируют чанки документа, помечаются на обновление: раздел `## Sources` получит новое имя файла при ближайшей сборке wiki. Строки registry с этим содержимым, файла которых больше нет, удаляются, а строки существующих файлов (копий или изменённых) остаются. Если после этого `ingest` изменить переименованный файл, старая версия удаляется как обычно.

Переименование и правка между двумя запусками `ingest` выглядят как удалённый старый файл плюс новый: старая версия остаётся (см. ограничения).

- **Блокировка.** PostgreSQL advisory lock корпуса не даёт параллельно запускать пишущие команды; соединение удерживается на всё время операции.
- **Повторы сетевых вызовов.** LLM: до 5 попыток при ошибках соединения и таймаутах, 429 и 5xx (экспоненциальная пауза до 30 с); невалидный JSON — ещё до 2 попыток с текстом ошибки валидации. В повтор передаются исходные сообщения, первые 2000 символов последнего невалидного ответа и его ошибка; длинные ответы и история предыдущих исправлений не накапливаются. Эмбеддер и реранкер: до 5 попыток при сетевых ошибках и 5xx. Qdrant: до 5 попыток при сетевых ошибках. Neo4j — средствами драйвера.
- **Подпись эмбеддера.** В `meta` хранится `model_id:dim:tpl-<хеш>`, где хеш (8 hex-символов sha256) покрывает `embedder.doc_template` и `query_template`. Смена модели, размерности или любого из шаблонов останавливает `ingest`, `wiki build`, `ask` и `compare` до `reindex`. Старая подпись без хеша (`model_id:dim`) принимается, если модель и размерность совпадают, и переписывается в новом формате при следующем запуске (только если значение в `meta` не изменилось за это время: идущий параллельно `reindex` не затирается). Такая подпись не знает, какими шаблонами построен индекс: если вы меняли `doc_template`/`query_template` до этой версии и не запускали после этого `reindex`, выполните `uv run mnogobase reindex` один раз.
- **Ограничения.** Удалённые с диска файлы из индекса не убираются (строки registry для них остаются, если то же содержимое не нашлось по другому пути). Также остаётся старая версия файла, который переименовали и изменили без `ingest` между этими шагами.

---

6. Справочник настроек

Все несекретные параметры — в `config.yaml`, секреты — в `.env`. Описание всех полей с типами — в `src/mnogobase/config.py`; значения по умолчанию в коде совпадают с `config.yaml`.

Приоритет источников: переменные `MNOGOBASE_*` (в том числе из `.env`) → файл конфига → значения по умолчанию. Ошибка в конфиге (неверный YAML, неизвестное значение, некорректный тип сущности) останавливает любую команду с `Invalid configuration: ...` и кодом 2; явно указанный, но отсутствующий файл — `Config file not found: ...`. Относительные пути (`data_dir`, `logs_dir`, `runs_dir`, `wiki.dir`) считаются от текущей папки, а не от файла конфига.

6.1 Все ключи config.yaml

Колонка «По умолчанию» описывает классы настроек в `src/mnogobase/config.py`. Значения в поставляемом `config.yaml` имеют более высокий приоритет; различия, включая модели 0.6B, устройство MPS и контекст 20000, собраны в разделе 2.6.

Ключ	По умолчанию	Что делает	После изменения
`device`	`auto`	`auto`, `cuda`, `mps`, `cpu`: устройство для Docling (и BM25 при `cuda`); `cpu` также выбирает CPU-серверы моделей, в режиме `--runner auto` — контейнеры Docker	сразу для Docling; `models up` для серверов
`data_dir`	`.mnogobase`	локальный маркер проекта, кэши, измерения	PostgreSQL namespace задаётся `database.project`
`logs_dir`	`logs`	папка `mnogobase.jsonl`	сразу
`runs_dir`	`runs`	папка `compare.jsonl`	сразу

Ключ	По умолчанию	Что делает	После изменения
`llm.model`	`gpt-6-luna`	что запускается и как модель называется на сервере: у `llamacpp` — GGUF `repo:квант`, у `vllm` — HF-репозиторий исходной модели, у `api` — имя на сервере (раздел 7.15)	новые вызовы; новая модель `extract` — новый ключ кэша
`llm.serve`	`api`	где работает модель: `llamacpp`, `vllm` (поднимает `mnogobase models up` на порту 8000) или `api` (готовый сервер или API по `base_url`)	новые вызовы; новая модель `extract` — новый ключ кэша
`llm.gguf`	не задан	только `llamacpp`, необязательно: веса `repo:квант` отдельно от имени — тогда запускается `gguf`, а `model` остаётся именем на сервере (`--alias`)	новые вызовы; новая модель `extract` — новый ключ кэша
`llm.base_url`	`https://codex.sale/v1`	адрес OpenAI-API до `/v1` включительно; обязателен для `api`, для локальных бэкендов по умолчанию `http://localhost:8000/v1` (задайте, если модель поднята на другой машине)	сразу
`llm.server`	ctx `32768`, gpu_memory `0.6`, args `[]`	как `models up` запускает сервер: окно контекста (`--ctx-size` / `--max-model-len`), доля видеопамяти vLLM, дополнительные флаги	`models up`
`llm.api_key_env`	`LLM_API_KEY`	имя переменной с ключом	сразу
`llm.concurrency`	`4`	максимум одновременных запросов к LLM	сразу
`llm.temperature`	`0`	температура генерации	сразу
`llm.timeout_s`	`120`	таймаут одного запроса, с	сразу
`llm.overrides`	`{extract: null, resolve: null, wiki: null, answer: null, query: null}`	своя модель для задачи, `null` — `llm.model`	как `llm.model`
`embedder.model`	`Qwen/Qwen3-Embedding-4B-GGUF:Q4_K_M`	что запускается и имя на сервере: `llamacpp` — GGUF `repo:квант`, `vllm` — HF-репозиторий, `api` — имя на сервере	`reindex`
`embedder.serve`	`llamacpp`	где работает модель: `llamacpp`, `vllm` (поднимает `mnogobase models up` на порту 8001) или `api` (готовый сервер или API по `base_url`)	`reindex`
`embedder.gguf`	не задан	как `llm.gguf`	`reindex`

Ключ	По умолчанию	Что делает	После изменения
`embedder.base_url`	не задан	адрес OpenAI-API до `/v1` включительно; обязателен для `api`, для локальных бэкендов по умолчанию `http://localhost:8001/v1` (задайте, если модель поднята на другой машине)	сразу
`embedder.server`	ctx `8192`, gpu_memory `0.15`, args `[]`	как `models up` запускает сервер: окно контекста (`--ctx-size` / `--max-model-len`), доля видеопамяти vLLM, дополнительные флаги	`models up`
`embedder.api_key_env`	не задан	имя переменной с Bearer-ключом сервера (`--api-key` у vLLM и llama.cpp)	сразу
`embedder.dim`	`auto`	размерность векторов. `auto` — полная размерность модели: её сообщает пробный запрос `/embeddings`, когда она впервые нужна (создание коллекций, подпись индекса). Число — вектор модели обрезается до него и нормируется (Matryoshka)	`reindex`, см. 7.4
`embedder.batch_size`	`32`	текстов в одном запросе к серверу	сразу
`embedder.doc_template`	`{text}`	шаблон документа; `{title}` — заголовок документа, имя сущности или страницы (`none`, если нет)	`reindex`
`embedder.query_template`	`Instruct: Given a question, retrieve passages that answer it\nQuery: {query}`	шаблон запроса (инструкция Qwen3-Embedding; согласуйте с doc_template)	`reindex`
`sparse.model`	`Qdrant/bm25`	модель fastembed для BM25	`reindex`
`parsing.ocr`	`true`	OCR в PDF и картинках	новые и изменённые файлы
`parsing.ocr_lang`	`["iso:ru", "iso:en"]`	языки OCR (BCP-47 с префиксом `iso:`); RapidOCR берёт первый, кириллическая модель читает и латиницу. Пустой список — китайская модель RapidOCR, кириллицу она не читает	новые и изменённые файлы
`parsing.extensions`	`[pdf, docx, pptx, xlsx, html, htm, md, adoc, csv, txt, png, jpg, jpeg, tif, tiff, bmp, webp]`	какие файлы берёт `ingest`	сразу

Ключ	По умолчанию	Что делает	После изменения
`chunking.tokenizer`	`auto`	токенайзер для подсчёта токенов чанка: `auto` — токенайзер самой модели эмбеддера (у llama.cpp — из GGUF через `/tokenize`), иначе HF-репозиторий (например `Qwen/Qwen3-Embedding-4B`)	новые и изменённые файлы
`chunking.max_tokens`	`512`	максимальный размер чанка	новые и изменённые файлы
`extract.entity_types`	18 типов (6.3)	типы сущностей и их описания для промпта	новые и изменённые файлы, предупреждение
`extract.max_failed_ratio`	`0.2`	допустимая доля чанков без извлечения на документ	сразу
`resolve.auto_merge`	`1.0`	косинус, с которого сущности сливаются без LLM (1.0 — никогда)	новые слияния
`resolve.llm_check`	`0.75`	косинус, с которого LLM спрашивают «одна ли это сущность» (кандидаты по ключу имени — всегда)	новые слияния
`resolve.max_descriptions`	`5`	после скольких описаний LLM сжимает их в одно	новые слияния
`wiki.dir`	`wiki`	папка wiki	`wiki build --all`
`wiki.language`	`en`	язык страниц: `en`, `ru` или название языка	`wiki build --all`
`wiki.min_mentions`	`2`	минимум чанков с упоминанием для страницы	`wiki build --all`
`wiki.evidence_k`	`12`	чанков-доказательств на страницу	`wiki build --all`
`wiki.obsidian`	`true`	дописывать в `wiki/.obsidian/graph.json` фильтр служебных страниц и цвета по типам (4.3)	следующий `wiki build`
`retrieval.k`	`8`	результатов на поиск (по умолчанию для `--k`)	сразу
`retrieval.context_tokens`	`6000`	бюджет контекста ответа (символы / 4)	сразу
`retrieval.translate`	`true`	искать и по английскому переводу вопроса не на английском (один вызов LLM `query` на вопрос)	сразу
`retrieval.answer_language`	`auto` (в `config.yaml` — `ru`)	язык ответа: код (`ru`, `en`, …) или `auto` — язык вопроса	сразу
`retrieval.neighbors`	`0`	соседних чанков того же раздела с каждой стороны	сразу

Ключ	По умолчанию	Что делает	После изменения
		попадания в режиме rag (и в его доле `all`)	
`retrieval.budget`	`{rag: 0.4, wiki: 0.3, graph: 0.3}`	доли бюджета в режиме `all`	сразу
`rerank.enabled`	`false` (в `config.yaml` — `true`)	пересортировать попадания rag кросс-энкодером	сразу
`rerank.model`	`gorse-io/Qwen3-Reranker-4B-GGUF:Q4_K_M`	как `embedder.model`; у `llamacpp` нужен GGUF с классификатором Qwen3-Reranker (раздел 7.15)	сразу
`rerank.serve`	`llamacpp`	где работает модель: `llamacpp`, `vllm` (поднимает `mnogobase models up` на порту 8002) или `api` (готовый сервер или API по `base_url`)	сразу
`rerank.gguf`	не задан	как `llm.gguf`	сразу
`rerank.base_url`	не задан	адрес OpenAI-API до `/v1` включительно; обязателен для `api`, для локальных бэкендов по умолчанию `http://localhost:8002/v1` (задайте, если модель поднята на другой машине)	сразу
`rerank.server`	ctx `8192`, gpu_memory `0.15`, args `[]`	как `models up` запускает сервер: окно контекста (`--ctx-size` / `--max-model-len`), доля видеопамяти vLLM, дополнительные флаги	`models up`
`rerank.api_key_env`	не задан	имя переменной с Bearer-ключом сервера	сразу
`rerank.candidates`	`30`	сколько попаданий гибридного поиска оценивает реранкер; остаются лучшие `retrieval.k`	сразу
`rerank.max_chars`	`4000`	до скольких символов обрезается текст чанка в запросе к реранкеру	сразу
`vision.enabled`	`false` (в `config.yaml` — `true`)	описывать картинки документов VLM-моделью при разборе	новые и изменённые файлы
`vision.model`	`prithivMLmods/Xiaomi-OCR-0-GGUF:Q6_K`	как `embedder.model`; у `llamacpp` в GGUF-репозитории должен лежать проектор картинок `mmproj-*.gguf`, его `-hf` качает сам	новые и изменённые файлы
`vision.serve`	`llamacpp`	`llamacpp`, `vllm` (`models up` на порту 8003) или `api` (любой мультимодальный OpenAI-API по `base_url`)	новые и изменённые файлы

Ключ	По умолчанию	Что делает	После изменения
`vision.gguf`	не задан	как `llm.gguf`	новые и изменённые файлы
`vision.base_url`, `vision.api_key_env`, `vision.server`	как у других ролей	адрес (по умолчанию `http://localhost:8003/v1`), ключ, параметры сервера	—
`vision.prompt`	описать картинку, у графика — оси, ключевые значения и тренды, текст переписать дословно	промпт VLM для каждой картинки	новые и изменённые файлы
`vision.max_tokens`	`400`	предел длины описания	новые и изменённые файлы
`vision.params`	`{temperature: 0.0, presence_penalty: 1.5}`	дополнительные поля запроса `/chat/completions`; штраф за повторы не даёт маленькой VLM зацикливаться	новые и изменённые файлы
`vision.min_area`	`0.05`	картинки меньше этой доли площади страницы не описываются	новые и изменённые файлы
`vision.skip`	`[logo, icon, qr_code, bar_code, signature, stamp]`	классы картинок (классификатор Docling, вероятность класса ≥ 0.5), которые не описываются; `[]` — без классификатора	новые и изменённые файлы
`vision.concurrency`	`4`	параллельных запросов к VLM	сразу
`vision.timeout_s`	`120`	таймаут одного запроса	сразу
`graph.hops`	`2`	глубина обхода в режиме `graph` (1–3)	сразу
`graph.max_relations`	`30`	максимум связей в контексте `graph`	сразу
`graph.seeds`	`5`	число стартовых сущностей из каждого поиска	сразу
`graph.seed_threshold`	`0.3`	минимальный косинус вопроса к сущности	сразу
`graph.traversal`	`cypher` (в `config.yaml` — `apoc`)	`cypher` или ограниченный APOC BFS с возвратом к Cypher при отсутствии процедуры/превышении node budget	сразу
`graph.use_gds`	`false` (в `config.yaml` — `true`)	учитывать готовый PageRank и, при `community.enabled`, сообщества Leiden в graph/all	сразу
`graph.community.enabled`	`false` (в `config.yaml` — `true`)	добавлять исходные чанки сообществ стартовых сущностей; требует `use_gds` и актуальный анализ	сразу
`graph.community.max_communities`	`3` (1–20)	максимум сообществ по порядку стартовых сущностей	сразу

Ключ	По умолчанию	Что делает	После изменения
`graph.community.max_entities`	`30` (1–500)	максимум участников каждого сообщества по PageRank	сразу
`graph.community.max_chunks`	`3` (1–100)	максимум дополнительных чанков внутри `retrieval.k`; локальные подтверждения сохраняют не менее половины мест	сразу
`graph.community.timeout_s`	`5`	таймаут запроса расширения; при превышении используется обычный контекст	сразу
`graph.analysis.max_nodes`, `max_relationships`	`100000`, `500000`	предел размера анализируемого графа	следующий `graph analyze`
`graph.analysis.memory_mb`, `timeout_s`	`1536`, `120`	бюджет оценки памяти проекции + алгоритма; таймаут одного GDS-запроса	следующий `graph analyze`
`graph.analysis.apoc_max_nodes`, `apoc_timeout_s`	`20000`, `5`	предел списка разрешённых узлов APOC и таймаут обхода	сразу
`qdrant.url`	`http://localhost:6333`	адрес Qdrant	сразу
`qdrant.prefix`	`mb_`	префикс коллекций	`reindex` (или отдельный `data_dir`)
`neo4j.uri`	`bolt://localhost:7687`	адрес Neo4j	это другой граф
`neo4j.user`	`neo4j`	пользователь	сразу
`neo4j.password_env`	`NEO4J_PASSWORD`	имя переменной с паролем	сразу
`evaluation.base_url`	`null` → `llm.url`	адрес OpenAI-совместимого API судьи	новый каталог оценки при смене судьи
`evaluation.model`	`null` → модель LLM для `answer`	модель-судья	новый каталог оценки
`evaluation.api_key_env`	`null` → `llm.api_key_env`	имя переменной ключа судьи; `""` — без ключа	новые вызовы; имя входит в манифест судьи
`evaluation.embedding_url`	`null` → `embedder.url`	адрес embeddings для `answer_relevancy`	новый каталог оценки
`evaluation.embedding_model`	`null` → `embedder.model`	модель embeddings судьи	новый каталог оценки
`evaluation.embedding_api_key_env`	`null` → `embedder.api_key_env`	имя переменной ключа embeddings; `""` — без ключа	новые вызовы; имя входит в манифест судьи
`evaluation.timeout_s`	`360`	таймаут запросов судьи	новый каталог оценки
`evaluation.temperature`	`0`	температура судьи	новый каталог оценки
`evaluation.strictness`	`3`	число вопросов для `answer_relevancy`	новый каталог оценки

«Новые и изменённые файлы» значит: уже загруженные документы не пересчитываются сами (их этапы `done`). Чтобы применить настройку ко всем, нужен `uv run mnogobase reset` и повторный `ingest` (LLM будет вызвана заново для всех чанков).

6.2 Переопределение через переменные окружения

Любой ключ можно задать переменной с префиксом `MNOGOBASE_`, вложенность — через `__`. Переменные можно положить и в `.env`.

```
MNOGOBASE_LLM__MODEL=gpt-6-luna-mini uv run mnogobase ask "..."
MNOGOBASE_EMBEDDER__SERVE=vllm                  # роль на другом бэкенде
MNOGOBASE_EMBEDDER__DIM=512
MNOGOBASE_LLM__OVERRIDES__ANSWER=gpt-6-luna
MNOGOBASE_RETRIEVAL__BUDGET__RAG=0.5            # остальные доли берутся из config.yaml
MNOGOBASE_PARSING__EXTENSIONS='["pdf", "md"]'   # списки и словари — в JSON
MNOGOBASE_CONFIG=other.yaml                     # другой файл конфига (как --config)
```

Типы сущностей из переменной задаются **JSON-объектом или JSON-списком** и **заменяют** набор из YAML целиком, а не дополняют его:

```
MNOGOBASE_EXTRACT__ENTITY_TYPES='{"Gene": "A named gene or protein.", "Other": "Anything
else."}'
MNOGOBASE_EXTRACT__ENTITY_TYPES='["Gene", "Condition", "Other"]'
```

Значение не в JSON (например, `Person,Other`) — ошибка конфига с кодом 2.

6.3 Типы сущностей

Типы задаются в `extract.entity_types`: имя типа → одна строка описания. Описания попадают в промпт извлечения и помогают LLM различать похожие типы. По умолчанию 18 крупных типов для документов из любых областей:

`Person`, `Organization`, `Location`, `Event`, `Project`, `Product`, `Equipment`, `Software`, `Technology`, `Method`, `Concept`, `Work`, `Metric`, `Regulation`, `Substance`, `Condition`, `Organism`, `Other`.

Набор нарочно небольшой. Тип входит в `entity_id`, поэтому каждая размытая граница между типами даёт дубли (одна сущность под двумя типами), а поиск по типу не фильтрует — тип нужен для подписи в режиме graph, frontmatter wiki и цветов. Близкие типы объединены: обученные модели — `Software`, дисциплины и природные процессы — `Concept`, датасеты и журналы — `Work`, небесные тела — `Location`, эпохи — `Event`. Узкие типы под свой корпус (`Gene`, `Contract`, `Vulnerability`, …) добавляйте в `config.yaml`.

Чтобы добавить или изменить тип, отредактируйте список в `config.yaml` (порядок сохраняется, `Other` оставляйте последним: в него попадает всё, что LLM отнесла к неизвестному типу; если `Other` нет, неизвестный тип становится последним в списке):

```
extract:
  entity_types:
    Person: A real or fictional individual, named or clearly identified.
    Gene: A named gene or protein; not the disease it causes (Condition).
    Condition: A disease, disorder, symptom or other medical or psychological condition.
    Other: Anything meaningful that fits none of the types above.
```

Можно указать и просто список имён без описаний: `entity_types: [Person, Gene, Other]`, или список вида `- Person: описание` / `- Gene`.

Правила проверки: список не пустой; имя не пустое и не содержит `:` или перевода строки; имена не повторяются без учёта регистра (`Person` и `person` — дубль).

Главное правило: **типы не должны пересекаться**. Тип входит в идентификатор сущности (`entity_id`), поэтому если одну и ту же вещь можно отнести к двум типам, она может расколоться на две разные сущности (например, `PyTorch` как `Software` и как `Technology`). В описании полезно прямо писать, чем тип отличается от соседних.

Что происходит после изменения типов:

- изменением считается любая правка списка: новое или удалённое имя, другое описание и даже другой порядок типов (меняется промпт);
- новые и изменённые документы сразу извлекаются с новыми типами (ключ кэша извлечения включает подпись набора типов);
- уже загруженные документы сохраняют старые типы. `status`, `doctor` и `ingest` предупреждают об этом, пока остаётся хотя бы один документ, извлечённый со старыми типами;
- чтобы переизвлечь всё с новыми типами: `uv run mnogobase reset`, затем `uv run mnogobase ingest documents/` (это заново вызывает LLM для всех чанков).

Для раскраски новых типов в Neo4j добавьте строку вида `node.Gene { color: #...; border-color: #...; }` в `docs/neo4j/mnogobase.grass` (раздел 4.1).

6.4 PostgreSQL и перенос существующего проекта

Единственный рабочий SQL backend — `database.backend: postgresql`. Значение `sqlite` отклоняется при загрузке конфигурации. DSN берётся из переменной, названной `database.url_env` (по умолчанию `DATABASE_URL`); файловые базы и автоматический fallback не поддерживаются. SQLite используется только командой импорта старых баз `database migrate-sqlite`. Переход и очистка текущих данных описаны в плане и отчёте проверки.

```
database:
  backend: postgresql
  url_env: DATABASE_URL
  schema_prefix: mnogobase
  project: default
  connect_timeout: 5
```

Пароль и URL находятся в `.env` (пример в `.env.example`). `POSTGRES_PASSWORD` задаёт пароль при первом создании Compose volume; `DATABASE_URL` задаёт подключение Python. Смена `POSTGRES_PASSWORD` после инициализации volume сама не меняет пароль роли в БД. Пароль с особыми символами в URL необходимо percent-encode.

В одной базе используются схемы `mnogobase_service` (пользователи, команды, права, сессии, личные диалоги), `mnogobase_project_default` (registry, очередь и uploads) и `mnogobase_project_team_<id>` для каждого командного корпуса. Для другого самостоятельного проекта задайте другое `database.project`, а также отдельные Qdrant prefix, Neo4j и файловые каталоги. PostgreSQL namespace не зависит от cwd. `database.schema_prefix` — `[a-z][a-z0-9_]{0,15}`, `database.project` — `[a-z][a-z0-9_]{0,36}`; составные имена укладываются в лимит PostgreSQL.

Существующий проект переносится так:

```
# Сначала остановите API, worker, watcher и остальные пишущие процессы проекта.
# При существующем .venv сохраняем дополнительные пакеты (например, ipykernel VS Code).
uv sync --inexact
docker compose up -d postgres
# Заполните DATABASE_URL в .env и установите database.backend: postgresql.
uv run mnogobase database migrate-sqlite
uv run mnogobase status
uv run mnogobase doctor
```

Команда переносит текущие `state.db`, `jobs.db`, `uploads.db`, общий `service.db` и базы зарегистрированных команд. Она резервирует SQLite источники от записи, блокирует целевые таблицы и импортирует все строки в одной транзакции PostgreSQL. До commit сравниваются количество и SHA-256 содержимого каждой таблицы; сохраняются ID, password hashes, session hashes/сроки, источники сообщений, порядок кэша, сигнатуры embedder и незавершённые операции.

Исходные SQLite-файлы сохраняются. Непустая целевая таблица останавливает перенос: команда не перезаписывает имеющиеся данные. Другие экспериментальные проекты в подпапках переносятся отдельно с их конфигом.

При ingest и миграции создаётся локальный `.mnogobase/postgresql-project.json` с идентификатором подключения без пароля. Он сохраняет защиту читающих команд и `reset --yes` от запуска из чужого каталога. Старые eval run.json после смены backend/настроек продолжают хранить старый снимок: начинайте новый каталог опыта.

Для резервного копирования PostgreSQL (каталог backups создайте заранее):

```
docker compose exec -T postgres pg_dump -U mnogobase -d mnogobase -Fc > backups/mnogobase.dump
```

Откат через `database.backend: sqlite` больше не поддерживается. Старые SQLite-файлы служат источником одностороннего импорта. Для восстановления используйте PostgreSQL backup; wiki, источники документов, Qdrant и Neo4j резервируются вместе с БД как части одного проекта.

PostgreSQL не меняет текущую модель обработки: один worker сервиса и один писатель корпуса. Wiki, документы и кэши остаются на файловой системе. Для нескольких машин потребуются согласованное файловое хранилище и отдельная доработка worker.

7. Как поменять типовые вещи

7.1 Локальная LLM

Для закрытых документов — ничего не уходит наружу. В `config.yaml`:

```
llm:
  model: unsloth/Qwen3-30B-A3B-Instruct-2507-GGUF:Q4_K_M     # llama.cpp: GGUF с квантом, ~18
ГБ
  serve: llamacpp
  base_url: null  # использовать localhost:8000/v1 вместо адреса внешней LLM
  # vLLM (только Linux + NVIDIA): serve: vllm, model: Qwen/Qwen3-30B-A3B-Instruct-2507-FP8
```

Для LLM задайте `base_url: null` явно: если просто удалить поле, вернётся внешний адрес из значений по умолчанию класса настроек. Через `.env` эквивалент — `MNOGOBASE_LLM__BASE_URL=`.

Затем `uv run mnogobase models up` поднимет и LLM. Ключ не нужен. Проверка: `uv run mnogobase doctor` (строка `llm`). Последствия: модель входит в ключ кэша извлечения, поэтому новые и изменённые документы извлекаются новой моделью, а уже загруженные остаются как есть; для полного переизвлечения — `reset` и `ingest`. Параллелизм ограничивают и `llm.concurrency`, и число слотов сервера (у llama.cpp — `--parallel`, задаётся в `llm.server.args`). `llm.overrides` с другими именами моделей работает только с `api`: локальный сервер держит одну модель.

7.2 Другой OpenAI-совместимый эндпоинт

vLLM, llama.cpp server, LM Studio, OpenRouter, OpenAI или корпоративный прокси подключаются одинаково:

```
llm:
  base_url: http://localhost:8000/v1    # адрес до /v1 включительно
  model: <имя модели на этом сервере>
  api_key_env: MY_PROVIDER_KEY          # и MY_PROVIDER_KEY=... в .env
```

Требование к серверу: `POST /chat/completions` с `response_format` типа `json_schema` и `strict: true` — так работают извлечение и слияние сущностей. Если сервер это не поддерживает, этап `extract` будет падать (см. раздел 8.3, «Новый провайдер LLM»). Блоки `<think>...</think>` из ответов модели вырезаются автоматически, а у JSON-ответов — и обёртка блоком кода.

7.3 Разные модели под задачи

```
llm:
  model: gpt-6-luna
  overrides: {extract: <дешёвая модель>, resolve: null, wiki: null, answer: <сильная модель>}
```

Задачи: `extract` (сущности и связи по чанкам — больше всего вызовов), `resolve` (проверка «одна ли сущность» и сжатие описаний), `wiki` (страницы), `answer` (ответы), `query` (перевод вопроса). Override меняет только имя модели: адрес и ключ общие (`llm.base_url`). Смена модели `extract` меняет ключ кэша, как в 7.1.

7.4 Сменить эмбеддер или размерность

Пример: Qwen3-Embedding-0.6B (1024 измерения) на llama.cpp. В `config.yaml`:

```
embedder:
  model: Qwen/Qwen3-Embedding-0.6B-GGUF:Q8_0
```

`embedder.dim` по умолчанию `auto`: размерность новой модели берётся у сервера, править её не нужно.

1. `uv run mnogobase models up` — перезапустит эмбеддер с новой моделью (остальные серверы не трогает).
2. `uv run mnogobase reindex` — удалит коллекции Qdrant, создаст их заново с новой размерностью и пересчитает все векторы. Ничего удалять вручную не нужно.

`chunking.tokenizer: auto` сам подхватит токенайзер новой модели. У всех Qwen3-Embedding он один и тот же (отличаются только служебные `<think>`/`<tool_response>`), так что границы чанков при переходе между размерами модели не меняются.

Подпись индекса — это бэкенд, веса (у llama.cpp — GGUF с квантом), размерность и шаблоны. Пока `reindex` не выполнен, `ingest`, `wiki build`, `ask` и `compare` отказываются работать: при новой размерности предварительная проверка пишет `mb_chunks has dim 2560, embedder 1024: run mnogobase reindex` (с явным числом в `embedder.dim` — `config 1024`), при других весах или бэкенде — `index was built with llamacpp:Qwen/Qwen3-Embedding-4B-GGUF:Q4_K_M:2560:tpl-..., config now uses ...; run mnogobase reindex`. Поэтому перевод эмбеддера с llama.cpp на vLLM (Q4 → bf16) или смена кванта тоже требуют `reindex`, а после правки одних `doc_template`/`query_template` в сообщении будет `embedder templates changed`. Индексы, построенные до `serve` (подписи `ollama:...` и `openai:...`), тоже требуют `reindex`.

Matryoshka (MRL): Qwen3-Embedding можно использовать в меньшей размерности — поставьте `embedder.dim: 1024` (или 512) без смены модели и выполните `reindex`. Переход с явной полной размерности на `auto` (и обратно) у той же модели подпись индекса не меняет и `reindex` не требует. Векторы меньше, поиск быстрее, качество немного ниже. Если размерность больше той, что выдаёт модель, `reindex` остановится до изменения индекса: `embedder check failed: model returned N dims, config expects M; nothing was changed`.

Токенайзер (`chunking.tokenizer: auto` следует за моделью эмбеддера) влияет только на чанкинг новых и изменённых файлов.

7.5 Размер чанка

`chunking.max_tokens` (512 по умолчанию). Меньше — точнее поиск и больше вызовов LLM на `extract`; больше — меньше вызовов, но грубее цитаты. Применяется к новым и изменённым файлам; для всего корпуса — `reset` и `ingest`. Сервер эмбеддера не обрезает слишком длинный вход, а отклоняет его (окно — `embedder.server.ctx`, по умолчанию 8192 токена), поэтому не ставьте предел выше окна эмбеддера.

7.6 OCR

`parsing.ocr: false` заметно ускоряет разбор «цифровых» PDF (с текстовым слоем), но сканы и скриншоты тогда дадут пустой текст. OCR работает только там, где у PDF нет текстового слоя (сканы, растровые области), и на файлах-картинках.

Языки — `parsing.ocr_lang` (по умолчанию `["iso:ru", "iso:en"]`). Docling сам выбирает движок (на Mac обычно RapidOCR), а RapidOCR работает с одним языком — первым из списка; его кириллическая модель читает и латиницу. Без `ocr_lang` RapidOCR берёт китайскую модель, и русский текст не распознаётся совсем. Модели OCR качаются при первом разборе (несколько МБ). Настройки действуют на новые и изменённые файлы.

7.7 Новые форматы файлов

Если формат поддерживает Docling, достаточно добавить расширение в `parsing.extensions`, например `[..., odt, epub, rtf, tex, xhtml]`. Список форматов установленного Docling:

```
uv run python -c "from docling.datamodel.base_models import FormatToExtensions as F;
[print(k.value, v) for k, v in F.items()]"
```

Картинки (`png`, `jpg`, ...) уже в списке: они идут тем же конвейером, что и PDF (разметка, OCR, описание VLM). Некоторым форматам нужны дополнительные зависимости Docling — проверьте на одном файле. Форматы, которых нет в Docling, требуют кода (раздел 8.3).

7.8 Типы сущностей

См. раздел 6.3.

7.9 Wiki: язык, порог, число доказательств

```
wiki:
  language: ru        # en и ru превращаются в English / Russian, другое значение передаётся
как есть
  min_mentions: 3
  evidence_k: 16
```

Затем `uv run mnogobase wiki build --all`. Имена сущностей, заголовки страниц и служебные разделы `Related`/`Sources` остаются английскими (имена извлекаются как канонические английские). Если `min_mentions` повысить, страницы сущностей ниже нового порога удаляются, только когда сущность снова попадёт в очередь (новое упоминание или удаление документа).

7.10 Поиск и ответы

`retrieval.k` (или `--k` в команде), `retrieval.context_tokens` (сколько контекста получает LLM — влияет на стоимость и качество), `retrieval.budget` (доли режимов в `all`, сумма не обязана быть 1). Действует сразу; `config_hash` в `compare.jsonl` меняется, так что прогоны с разными настройками легко различить.

7.11 Режим graph

`graph.hops` (1–3: больше — шире контекст и шум), `graph.seeds` (сколько стартовых сущностей брать из векторного и полнотекстового поиска), `graph.seed_threshold` (понизьте, если `graph` часто отвечает «нет источников»), `graph.max_relations`. Действует сразу. Страницы wiki всегда берут до 30 связей сущности независимо от `graph.max_relations`.

7.12 Слияние сущностей

`resolve.auto_merge` ниже 1.0 — часть слияний без LLM (дешевле) и риск склеить разные вещи; `resolve.llm_check` ниже — больше вопросов к LLM и меньше пропущенных дублей. Значения по умолчанию выбраны замером на 361 размеченной вручную паре сущностей из статьи по астрономии и брошюры о Солнечной системе (вердикты — LLM `resolve` из `config.yaml`):

Политика	Слито	Ошибочно	Пропущено
`auto_merge 0.92`, `llm_check 0.80`, без ключа имени (было)	32	7	31 из 56
`auto_merge 0.97`, `llm_check 0.80`	27	2	31 из 56
`auto_merge 1.0`, `llm_check 0.80`, с ключом имени	42	1	15 из 56
`auto_merge 1.0`, `llm_check 0.75`, с ключом имени (сейчас)	58	2	0 из 56

Вектор ошибается и при очень высоком косинусе: `Gap depth method` → `Gap width method` (0.988), `WMT 2014 English-to-French` → `English-to-German` (0.968), `X. Zhang` → `H. Zhang` (0.962), а верные пары начинаются с 0.5. В журнале 111 прошлых слияний (статьи о Transformer и BERT) по одному вектору верны 16 из 47, после проверки LLM — 52 из 54 однозначных. Пары ниже 0.75 размечены только среди найденных по ключу имени, поэтому «0 пропущенных» — оценка сверху. Действует на новые слияния; уже слитые сущности не разделяются (для пересборки графа — `reset` и `ingest`). Результаты видны в логе (`entity_merged`, раздел 4.5).

7.13 Промпты

Промпты лежат в `src/mnogobase/llm/prompts/` и подставляются через `string.Template` (`$name`; буквальный `$` пишется как `$$`). Пакет установлен в режиме редактирования, поэтому правки действуют без переустановки.

Файл	Задача	Плейсхолдеры	Когда действует
`extract.md`	сущности и связи из чанка	`$entity_types`, `$title`, `$headings`, `$text`	см. ниже
`resolve_same.md`	одна ли это сущность	`$a_name`, `$a_type`, `$a_description`, `$b_name`, `$b_type`, `$b_description`	новые слияния
`resolve_summarize.md`	сжать описания сущности	`$name`, `$type`, `$descriptions`	новые слияния
`wiki_page.md`	тело wiki-страницы	`$language`, `$name`, `$type`, `$description`, `$aliases`, `$relations`, `$evidence`, `$example_id`, `$existing`	`wiki build --all`
`answer.md`	ответ с цитатами `[n]`	`$question`, `$context`, `$language`	сразу
`translate_query.md`	перевод вопроса на английский для поиска	`$question`	сразу

- **Кэш извлечения.** Ключ кэша — `(chunk_id, prompt_version, model)`, где `prompt_version` = `PROMPT_VERSION` из `src/mnogobase/extraction/extractor.py` (сейчас `extract-v4`) плюс подпись набора типов. **Меняете смысл `extract.md` — увеличьте `PROMPT_VERSION`**, иначе чанки с кэшем получат старые результаты. Уже загруженные документы сами не переизвлекаются; для всех — `reset` и `ingest`.
- Новый плейсхолдер нужно передать из кода (`render(...)` в месте вызова) и добавить в `tests/unit/test_templates.py`: тест проверяет, что в промпте не осталось `$`.
- Поля JSON-ответов задаются схемами в коде (`ExtractionResult` в `models.py`, `SameEntity` и `MergedDescription` в `extraction/resolver.py`), промптом их не поменять.

7.14 Отдельные проекты

Чтобы вести несколько независимых баз, создайте для каждой свой конфиг и запускайте команды с `-c`:

```
# project-b.yaml — заменяет config.yaml целиком: не указанные ключи берутся из значений по умолчанию
data_dir: .mnogobase-b
logs_dir: logs-b
runs_dir: runs-b
wiki: {dir: wiki-b}
qdrant: {prefix: pb_}
neo4j: {uri: bolt://localhost:7688}
```

```
uv run mnogobase -c project-b.yaml ingest documents-b/
```

Qdrant можно делить между проектами через `qdrant.prefix`. **Neo4j делить нельзя:** код пишет в базу по умолчанию, а `reset` удаляет в ней все узлы. Поднимите второй экземпляр, например `docker run -d --name neo4j-b -p 7475:7474 -p 7688:7687 -e NEO4J_AUTH=neo4j/mnogobase-dev neo4j:5.26-community`.

7.15 Где работают модели (serve)

У каждой роли — эмбеддера, реранкера, VLM (`vision`) и LLM — своё поле `serve`:

`serve`	Что это	Кто запускает
`llamacpp`	llama.cpp server с квантизованным GGUF из `model` (`repo:квант`); мало видеопамяти, работает и на Mac (Metal)	`mnogobase models up`
`vllm`	vLLM с исходными весами `model`; быстрее на CUDA под нагрузкой, только Linux + NVIDIA	`mnogobase models up` (в Docker)
`api`	уже работающий сервер или облачный API по `base_url`	вы

Роли независимы, их можно смешивать. Например, эмбеддер и реранкер на llama.cpp (мало памяти), LLM на vLLM:

```
embedder: {model: Qwen/Qwen3-Embedding-4B-GGUF:Q4_K_M, serve: llamacpp}
rerank:   {enabled: true, model: gorse-io/Qwen3-Reranker-4B-GGUF:Q4_K_M, serve: llamacpp}
llm:      {model: Qwen/Qwen3-30B-A3B-Instruct-2507-FP8, serve: vllm, base_url: null}
```

Или эмбеддер на vLLM, а LLM по API: `embedder: {serve: vllm, model: Qwen/Qwen3-Embedding-4B}`, `llm: {serve: api, base_url: ...}`.

Что писать в `model` зависит от `serve`. Под этим же именем модель видна на сервере (llama.cpp получает его как `--alias`), по нему `doctor` проверяет `/v1/models`.

- `llamacpp` — GGUF-репозиторий с квантом, `repo:квант`. Варианты — на странице GGUF-репозитория на Hugging Face (`*-GGUF`, файлы `*-Q4_K_M.gguf`, `*-Q8_0.gguf` и т. д.). Токенайзер зашит в GGUF, исходный HF-репозиторий модели не нужен.
- `vllm` — HF-репозиторий исходной модели (safetensors, FP8, AWQ и т. п.). GGUF у vLLM — экспериментальный плагин, которому всё равно нужен токенайзер исходной модели, поэтому `model` с `:квант` или заданный `gguf` при `vllm` — ошибка конфига.
- `api` — имя модели на сервере, как его отдаёт `/v1/models`.

Поэтому при переводе роли между `llamacpp` и `vllm` меняется и `model`, а не только `serve`. `gguf` — необязательное поле для `llamacpp`: если модель на сервере нужно назвать иначе, чем веса, `model` остаётся именем, а запускается `gguf` (`repo:квант`). Подпись индекса зависит только от весов, поэтому перенос того же GGUF из `gguf` в `model` не требует `reindex`.

Команды (по `config.yaml`):

```
uv run mnogobase models up               # поднять все роли с serve: llamacpp | vllm и
дождаться /health
uv run mnogobase models up embedder      # только одну роль
uv run mnogobase models status           # роль, бэкенд, веса, адрес, готовность
uv run mnogobase models logs rerank -f   # лог сервера роли
uv run mnogobase models down             # остановить (скачанные модели остаются в кэше)
```

`models up` сначала печатает команду каждого сервера (`llama-server ...` или `vllm serve ...`), затем, пока серверы поднимаются, раз в 15 с — состояние каждого не готового по его `/health`:

```
embedder → http://localhost:8001/v1 (native): llama-server -hf Qwen/Qwen3-Embedding-4B-
GGUF:Q4_K_M --alias ...
[0:00] embedder: starting or downloading the model (server log: `mnogobase models logs embedder
-f`)
[0:45] embedder: loading the model (server log: ...)
[0:52] embedder: ready
[0:52] All model servers are ready.
```

«starting or downloading» — сервер ещё не отвечает (запускается или качает веса; размер загрузки сервер не сообщает), «loading the model» — `/health` отвечает 503, веса грузятся в память.

`models up` не пересоздаёт работающий сервер с той же моделью, устройством и флагами; после правки `config.yaml` перезапускается только изменившаяся роль, а сервер роли, ставшей `api` или отключённой, останавливается. Где запускать, выбирает `--runner`: `native` (по умолчанию на macOS при `device` не `cpu`) — процессы `llama-server` (`brew install llama.cpp`), состояние в `.mnogobase/models/`, логи в `logs/models-<роль>.log`; `docker` (по умолчанию на Linux/Windows и при `device: cpu` на любой ОС) — сервисы из `docker-compose.models.yml` в compose-проекте `mnogobase-models` (раздел 11). Явный `--runner native` сохраняет нативный запуск, в том числе llama.cpp на CPU. vLLM запускается только в Docker.

CPU и запасной запуск после ошибки GPU — только llama.cpp. При `device: cpu` команда сразу выбирает сервисы `<роль>-llamacpp-cpu`: CPU-образ llama.cpp с `--n-gpu-layers 0`. Модели загружаются из тех же GGUF-репозиториев, имя API-модели и порты не меняются; веса остаются в общем томе `llama_models`. Для такого переключения `reindex` не нужен. Роли с `serve: api` и выключенные роли не запускаются. **vLLM работает исключительно на CUDA:** активная роль с `serve: vllm` при `device: cpu` вызывает понятную ошибку до изменения контейнеров. Для CPU выберите `serve: llamacpp` и совместимые GGUF-веса; автоматической подмены бэкенда или модели нет.

В Docker роли запускаются по очереди. Если CUDA/NVIDIA у **llama.cpp** не инициализируется, отсутствует GPU-драйвер контейнеров или сервер сообщает CUDA OOM, `models up` печатает причину, останавливает неудачный GPU-контейнер и один раз пробует CPU-контейнер **этой роли**. Уже поднятые роли продолжают работать. vLLM после ошибки GPU на CPU не переключается. Ошибки скачивания, авторизации, конфигурации и занятых портов остаются ошибками; обычное ожидание загрузки модели не вызывает переключения. Если CPU-запуск тоже завершается ошибкой, команда выходит с кодом 2 и останавливает неудачный контейнер, сохранив его логи. `models logs <роль>` находит CPU-контейнер после переключения, `models down` останавливает оба варианта, оставляя веса в кэше. При следующем `models up` с `device`, отличным от `cpu`, GPU будет проверена заново; для постоянного CPU-режима задайте `device: cpu`.

CPU-контейнер llama.cpp по умолчанию ограничен **8 ГиБ RAM без swap и 2 CPU**. Значения задаются прямо в блоке `x-cpu` файла `docker-compose.models.yml`: `mem_limit`, `memswap_limit` и `cpus`. Чтобы запретить swap, сохраняйте `memswap_limit` равным `mem_limit`. Это лимиты **каждого контейнера**, суммарный потолок равен числу CPU-серверов, умноженному на лимит. CPU-образ поддерживает x86-64 и ARM64. Нативный Metal-запуск на Mac не переключается автоматически на Docker: для CPU-контейнеров задайте `device: cpu`, оставив `--runner auto` или указав `--runner docker`. При смене между `native` и `docker` сначала остановите прежний раннер: `models down --runner native` или `models down --runner docker`.

Особенности:

- **llama.cpp, реранкер.** Нужен GGUF с классификатором Qwen3-Reranker (`gorse-io/Qwen3-Reranker-4B-GGUF` по умолчанию, есть и 0.6B); старые конвертации (`dengcao`, `mradermacher`, `QuantFactory`) загружаются, но дают бессмысленные оценки.
- **vLLM — только CUDA.** Каждый сервер получает долю **всей** видеопамяти (`<роль>.server.gpu_memory`: 0.15 для эмбеддера, реранкера и vision, 0.6 для LLM); на одной карте сумма долей должна быть меньше 1 с запасом для прочих процессов. Для Qwen3-Reranker `models up` сам добавляет `--hf-overrides` и шаблон промпта из образа. vLLM проверен по документации и модульным тестам, не вживую.
- **Окно контекста** — `<роль>.server.ctx`. Длинный вход сервер не обрезает, а отклоняет: окно эмбеддера должно вмещать `chunking.max_tokens` плюс шаблон.
- **Дополнительные флаги** сервера — `<роль>.server.args`, например `["--parallel", "8"]` для llama.cpp.
- **Ключ API.** Если сервер (`api`) требует ключ, задайте `<роль>.api_key_env` — имя переменной из `.env`.
- **VLM на llama.cpp.** Нужен GGUF-репозиторий с проектором картинок (`mmproj-*.gguf`): `-hf` скачивает и подключает его сам, в логе сервера — `loaded multimodal model`.

7.16 Картинки и графики (vision)

OCR только извлекает текст, и подписи внутри графика или схемы обычно не попадают в чанк: Docling считает такую область картинкой. При `vision.enabled` каждую картинку описывает VLM (по умолчанию Xiaomi-OCR-0 Q6_K на llama.cpp, порт 8003): что изображено, оси и ключевые значения графика, дословный текст. Описание встаёт в документ рядом с картинкой (после подписи `Figure ...`) и дальше работает как обычный текст: эмбеддинг и поиск, извлечение сущностей в граф, цитаты в ответах и wiki со ссылкой на документ и страницу. Модель выбирается через `vision.model`; OCR Docling настраивается отдельно через `parsing.ocr` и `parsing.ocr_lang`.

Для Xiaomi-OCR-0 в `config.yaml` задано `vision.server.args: ["--image-min-tokens", "1024"]`. Это минимум визуальных токенов llama.cpp: в локальной проверке без него маленькая схема, выделенная из PDF, получила выдуманное описание постороннего графика. С этим параметром та же схема описана правильно, в том числе при повторном разборе PDF.

Как Docling отбирает картинки:

1. Модель разметки находит на странице области «картинка» (в PDF, DOCX, PPTX и файлах-картинках).
2. Картинки меньше `vision.min_area` площади страницы пропускаются.
3. Классификатор Docling определяет тип (графики, схемы, фото, логотипы, ...); классы из `vision.skip` с вероятностью ≥ 0.5 пропускаются.
4. Остальные уходят на `POST <vision.url>/chat/completions` (`vision.concurrency` параллельно).

Файл-картинка (скриншот) описывается ещё и **целиком**: разметка находит в скриншоте текст и таблицы, а не картинку. Описание VLM идёт первым, за ним текст OCR. В локальной проверке Xiaomi-OCR-0 сохранила русские названия товаров и все числа контрольной таблицы, но добавила неверный английский перевод слова «Ручка» — «Booklet».

Включение:

```
# config.yaml: vision.enabled: true (модель и сервер — ключи vision.*, раздел 6.1)
uv run mnogobase models up vision    # ~0.84 ГБ: Xiaomi-OCR-0 Q6_K и проектор картинок
uv run mnogobase doctor              # строка models: vision ... available
```

Особенности:

- **Применяется к новым и изменённым файлам.** Уже загруженные документы разобраны без описаний; чтобы описать их картинки — `reset` и `ingest` (кэш извлечения привязан к `chunk_id`, поэтому частичный повторный разбор не годится).
- **Скорость.** Три прямых запроса к Xiaomi-OCR-0 Q6_K на Mac с `--image-min-tokens 1024` заняли 0.89–1.22 с на картинку. Это контрольные таблица, график и схема; время загрузки модели и разбора Docling сюда не входит.
- **Ошибки VLM не проходят молча.** Docling при ошибке API оставляет описание пустым; `mnogobase` в этом случае завершает этап `parse` ошибкой (`VisionError`), и файл повторяется через `ingest --retry-failed`.
- **Качество.** Локально проверены реальные ответы Xiaomi-OCR-0 и разбор PNG, DOCX и PDF: описания сохраняются в кэше и попадают в чанки, у PDF сохраняются номера страниц. Все контрольные числа и подписи присутствовали. При этом в описании выделенного графика модель ошибочно назвала разные столбцы одинаковыми по высоте; совпадение чисел не гарантирует верность выводов. Другие варианты — `vision.model: unsloth/Qwen3-VL-8B-Instruct-GGUF:Q4_K_M` (~6 ГБ; на vLLM — `Qwen/Qwen3-VL-8B-Instruct`) или мультимодальный API: `vision: {serve: api, base_url: ..., model: ..., api_key_env: ...}`.
- **Язык описания** — английский (промпт на английском), как и канонические имена сущностей; текст с картинки переписывается на его языке.

---

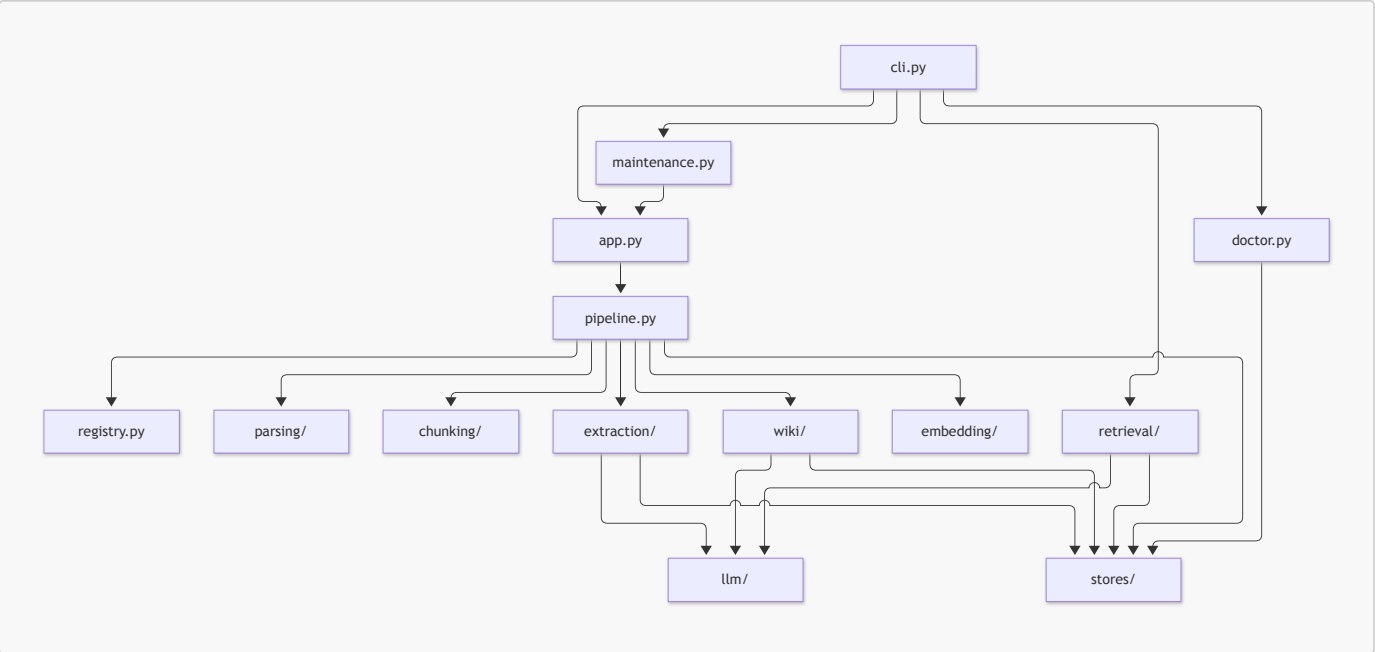
8. Как расширять код

8.1 Карта модулей

Модуль	Отвечает за
`cli.py`	команды Typer, предварительные проверки, блокировка, коды выхода, вывод
`app.py`	`build_app`: создаёт все компоненты и связывает их (точка подмены реализаций)
`pipeline.py`	`Pipeline`: подготовка, обнаружение файлов, этапы `ingest`, `remove`, восстановление удалений
`removal.py`	выбор зарегистрированных источников по путям и `--missing`, `RemovalReport`
`database.py`	подключения PostgreSQL, bind-параметры, namespace, advisory locks и маркер проекта
`database_migration.py` / `database_cli.py`	транзакционный перенос SQLite, сверка данных и команда `database migrate-sqlite`

Модуль	Отвечает за
`registry.py`	`Registry` (PostgreSQL), `STAGES`, `ingest_lock`
`config.py`	`Settings` (pydantic-settings), типы сущностей по умолчанию, загрузка YAML и env
`models.py`	доменные модели: `DocumentRecord`, `ChunkRecord`, `EntityRecord`, `ExtractionResult`, `ContextItem`, `Answer` и др.
`ids.py`	детерминированные ID, `normalize_name`, `slugify`
`log.py`	structlog, `run_id`, `log_stage`
`device.py`	выбор `cuda` / `mps` / `cpu`
`parsing/docling_parser.py`	`DoclingParser`: Docling, кэш, картинки; OCR-языки, описание картинок VLM (`_describe_pictures`, `_describe_whole_image`, `VisionError`)
`chunking/hybrid.py`	`Chunker`: HybridChunker с токенайзером из `chunking/tokenizer.py`
`chunking/tokenizer.py`	`load_tokenizer`: HF-репозиторий или (`auto`) `/tokenize` сервера у llama.cpp (`ServerTokenizer`), HF-токенайзер `embedder.model` у vLLM; у `api` — по `root` из `/v1/models` или `/tokenize` сервера
`embedding/`	протоколы `Embedder`, `SparseEncoder` и помощники (`base.py`), `OpenAICompatEmbedder` (`openai_compat.py`), общий HTTP-код (`http.py`), `BM25Encoder`
`llm/`	протокол `LLMClient`, `OpenAICompatLLM` (`client.py`), `render` / `render_messages` (`templates.py`), промпты
`extraction/`	`Extractor` (кэш, очистка ответа, `PROMPT_VERSION`), `EntityResolver`
`stores/`	`QdrantStore` и `GraphStore` — весь код Qdrant и весь Cypher
`wiki/`	`WikiBuilder` (сборка), `render.py` (Markdown, index, log), `validate.py` (цитаты и ссылки)
`retrieval/`	протокол `Retriever`, 4 режима, `Answerer`, `compare`
`doctor.py`	проверки окружения и предварительные проверки команд
`serving.py`	`mnogobase models`: роли с `serve`, команды llama.cpp и vLLM (`server_args`), запуск в Docker (`DockerRunner`) и нативно (`NativeRunner`), ожидание `/health`
`maintenance.py`	`reindex`, `reset`
`background/`	`queue.py` — очередь PostgreSQL; `watch.py` — стабильность и обнаружение файлов; `worker.py` — обработка/повторы; `cli.py` — команды jobs/worker
`processing.py`	измерения этапов, успешных токенов LLM и снимок индексированного корпуса
`evaluation/`	CLI run/score/report/benchmark, артефакты и возобновление, Ragas-судья, сводные и парные сравнения

Зависимости между пакетами (общие `config`, `models`, `ids`, `log` используются всеми и на схеме опущены):



8.2 Узкие интерфейсы

Сменные части спрятаны за протоколами (`typing.Protocol`, реализация не наследуется — достаточно тех же атрибутов и методов):

```python
# embedding/base.py
class Embedder(Protocol):
    model_id: str
    dim: int
    templates: tuple[str, str]  # (document, query) templates the embedder applies

    def embed_documents(self, items: Sequence[EmbedInput]) -> list[list[float]]: ...
    def embed_query(self, query: str) -> list[float]: ...


class SparseEncoder(Protocol):
    def encode_documents(self, texts: Sequence[str]) -> list[qm.SparseVector]: ...
    def encode_query(self, text: str) -> qm.SparseVector: ...


# llm/client.py
class LLMClient(Protocol):
    usage: Usage  # накопленные tokens_in / tokens_out

    def model_for(self, task: str) -> str: ...
    async def complete(self, messages: list[Message], *, task: str) -> str: ...
    async def structured(
        self, messages: list[Message], schema: type[T], *, task: str, max_repairs: int = 2
    ) -> T: ...


# retrieval/base.py
class Retriever(Protocol):
    def retrieve(
        self, query: str, k: int, alt_queries: Sequence[str] = ()
    ) -> list[ContextItem]: ...
```

`EmbedInput` (`models.py`) — `modality` (`text`; `image` и `audio` зарезервированы), `text`, `path`, `title`. `task` у LLM — одно из `extract`, `resolve`, `wiki`, `answer`, `query`: выбирает модель через `llm.overrides` и попадает в логи.

Хранилища (`QdrantStore`, `GraphStore`), `Registry`, `DoclingParser` и `Chunker` — конкретные классы без протоколов.

Связывание происходит в `app.py`:

```
def build_app(settings, *, embedder=None, sparse=None, llm=None, vectors=None, graph=None) ->
App
```

Всё, что не передано, создаётся по настройкам: `OpenAICompatEmbedder`, `ApiReranker` (при `rerank.enabled`), `BM25Encoder`, `QdrantStore.from_settings`, `GraphStore.from_settings`, `OpenAICompatLLM`. Тесты передают сюда фейки. Режимы поиска собирает `retrieval/__init__.py: build_retrievers(...)` — словарь `{режим: Retriever}`.

8.3 Рецепты расширения

Новый эмбеддер (например, sentence-transformers или облачный API):

1. Сначала проверьте, нельзя ли обойтись без кода: любой сервер с OpenAI-совместимым `/embeddings` (LM Studio, TEI, облачный API) подключается как `embedder.serve: api` с `base_url` и `model`. Новый **бэкенд запуска** (как llama.cpp и vLLM) добавляется в `serving.py`: значение `Serve` в `config.py`, ветка в `server_args`, сервисы в `docker-compose.models.yml`.
2. Иначе — класс в `src/mnogobase/embedding/<name>.py` с атрибутами `model_id`, `dim`, `templates` и методами `embed_documents`, `embed_query` (образец — `OpenAICompatEmbedder`). Используйте помощники из `embedding/base.py`: `format_document(item, template)`, `format_query(query, template)` и `truncate_normalize(vec, dim)` (обрезка Matryoshka и L2-нормировка), а для HTTP — `retryable` и `auth_headers` из `embedding/http.py`.
3. `model_id` должен быть уникальным с префиксом провайдера (как `openai:<model>`): подпись `model_id:dim:tpl-<хеш templates>` (`embedder_signature`) сохраняется в `meta` и защищает индекс от смешивания векторов. Если провайдер не использует шаблоны, задайте `templates` постоянными.
4. `config.py`: поле в `EmbedderSettings`, по которому выбирается класс; выбор — в `build_app` (`app.py`) и в проверке `index` (`doctor.py`). Проверка `models` и `chunking.tokenizer: auto` рассчитаны на OpenAI-совместимый `/v1/models`; для другого API адаптируйте `_served_models` (`doctor.py`) или задайте HF-репозиторий в `chunking.tokenizer`. Набор проверок команд задан в `cli.py` (`PREFLIGHT_CHECKS`, `INGEST_CHECKS`, `REINDEX_CHECKS`).
5. Тесты по образцу `tests/unit/test_embedding.py` (HTTP через `httpx.MockTransport`). После переключения — `reindex` (раздел 7.4).

Новый провайдер LLM. Почти всегда хватает `OpenAICompatLLM` (раздел 7.2). Свой клиент нужен, если API не OpenAI-совместимый или не умеет `json_schema`:

1. Класс с `usage: Usage`, `model_for(task)`, `async complete(...)`, `async structured(...)`. Переиспользуйте `strict_json_schema(schema)`, `clean_json(text)`, `strip_think(text)` из `llm/client.py`; `structured` должен вернуть провалидированную pydantic-модель и при ошибке повторить запрос с текстом ошибки (как в `OpenAICompatLLM`).
2. Ограничьте параллелизм семафором на `llm.concurrency` (извлечение и wiki вызывают LLM параллельно) и копите токены в `usage` — их читает `compare`.
3. Выбор класса — в `build_app` (новое поле в `LLMSettings`); проверку `llm` в `doctor.py` адаптируйте к API.

Новый режим поиска:

1. Класс с `retrieve(self, query: str, k: int, alt_queries: Sequence[str] = ()) -> list[ContextItem]` (`alt_queries` — другие формулировки вопроса, например английский перевод; ищите и по ним) в `src/mnogobase/retrieval/<name>.py`. `ContextItem.kind` — одно из `chunk`, `wiki`, `relation`, `entity` (`models.py`); для цитат с файлом и страницей отдавайте элементы `chunk` с `path` и `page`.
2. Добавьте значение в `Mode` и в словарь `build_retrievers` (`retrieval/__init__.py`). `ask --mode` и `compare` подхватят режим автоматически (`compare` проходит по всем ретриверам).
3. Чтобы режим участвовал в `all`, добавьте его в `parts` `CombinedRetriever` и долю в `retrieval.budget`.
4. Тесты — по образцу `tests/unit/test_retrieval.py` (Qdrant `:memory:`) или интеграционные для графа.

Новая команда CLI. Команды — функции с `@app.command()` в `cli.py` (подкоманды wiki — `@wiki_app.command(...)`). Соглашения: `_settings(project=True)` для команд, которым нужен существующий проект; `_preflight(settings, checks)` перед обращением к сервисам; `_lock(settings)` для всего, что пишет; `build_app` и `application.close()` в

`finally`; известные ошибки — через `_fail_cleanly` (код 2); текст из документов печатать через `escape`. Пример команды только для чтения:

```
from mnogobase.stores.graph_store import GraphStore  # в начало cli.py


@app.command()
def top(limit: Annotated[int, typer.Option("--limit", help="How many entities.")] = 20) ->
None:
    """Show the most mentioned entities."""
    settings = _settings(project=True)
    _preflight(settings, ("neo4j",))
    graph = GraphStore.from_settings(settings.neo4j)
    try:
        entities = graph.entities(min_mentions=1)
    finally:
        graph.close()
    table = Table("name", "type", "mentions")
    for e in sorted(entities, key=lambda e: -e.mention_count)[:limit]:
        table.add_row(escape(e.name), e.type, str(e.mention_count))
    console.print(table)
```

Тесты CLI — `tests/unit/test_cli.py`: `typer.testing.CliRunner`, `run_checks` и `configure_logging` подменяются фикстурой.

Новый формат файлов.

- Формат есть в Docling — только `parsing.extensions` (раздел 7.7).
- Формата нет в Docling — расширьте `DoclingParser.parse` (`parsing/docling_parser.py`): по расширению превратите файл в Markdown или HTML и передайте конвертеру поток `DocumentStream(name="<имя>.md", stream=BytesIO(...))` из `docling.datamodel.base_models` (метод `convert` принимает путь или поток). Остальной конвейер работает с `DoclingDocument`, поэтому `parse` должен по-прежнему сохранять JSON в кэш и возвращать `ParsedDocument`. Не забудьте добавить расширение в `parsing.extensions` и тест в `tests/unit/test_parsing_chunking.py`.

Изменение схемы графа. Весь Cypher — в `stores/graph_store.py`:

1. Ограничения и индексы — в список `_SCHEMA` (только `IF NOT EXISTS`: схема применяется при каждом запуске).
2. Новое свойство — в модель (`models.py`) и в соответствующий `upsert_*`; новая связь или метка — новый метод `GraphStore` и вызов из `pipeline.py` или `wiki/builder.py`.
3. Поддержите каскадное удаление: `_delete_document_tx` и `document_deletion_plan` должны убирать новые данные документа согласованно (план вычисляется до транзакции). Если данные нужны `reindex`, проверьте `doc_chunks` и `maintenance.py`.
4. Уже загруженные данные сами не мигрируют: разовый Cypher-скрипт или `reset` и `ingest`.
5. Обновите интеграционные тесты (`tests/integration/test_graph_store.py`, `test_pipeline.py`), запросы в разделе 4.1 и стиль `docs/neo4j/mnogobase.grass` для новых меток.

8.4 Тесты, стиль и коммиты

- **SQL-хранилища проверяются на настоящем PostgreSQL**, без подмены SQLite. Соответствующие проверки registry, очереди, service и CLI отмечены `integration`, даже если файл исторически лежит в `tests/unit/`. Каждый тест получает отдельный namespace. `MNOGOBASE_TEST_DATABASE_URL` допускает выделенный тестовый PostgreSQL вместо временного контейнера; рабочий `DATABASE_URL` тесты не используют.
- **Маркеры** (`pyproject.toml`): без маркера — модульные тесты, `integration` — нужен Docker (testcontainers), `e2e` — Docker, модели из `config.yaml` (`mnogobase models up`) и LLM. `addopts = "-m 'not integration and not e2e'"`, поэтому `uv run pytest` запускает только модульные.
- **Фейки** (`tests/fakes.py`): `FakeLLM(handler)` — детерминированный `LLMClient`, `handler(task, prompt) -> текст ответа`, вызовы копятся в `calls`; `scripted_llm_handler` отвечает по ключевым словам (`transformer`, `attention`, `внимани`, `softmax`, `vaswani`); `FakeEmbedder` — хешированный мешок слов (тексты с общими словами близки); `FakeSparse` — такой же sparse.

- **Хранилища в тестах.** Qdrant в модульных тестах — `QdrantClient(":memory:")`. In-memory фейка Neo4j нет: всё, что касается графа, — интеграционные тесты. Фикстуры `tests/conftest.py`: `qdrant_client` и `neo4j_container` (на сессию, образы `qdrant/qdrant:v1.19.2` и `neo4j:5.26-community`), `graph` (чистый `GraphStore` со схемой на каждый тест).
- **Шаблон интеграционного теста конвейера** (`tests/integration/test_pipeline.py`): `build_app(settings, embedder=FakeEmbedder(), sparse=FakeSparse(), llm=FakeLLM(scripted_llm_handler), vectors=QdrantStore(QdrantClient(":memory:"), "t_", 64), graph=graph)` с `data_dir`, `wiki.dir` и `logs_dir` во временной папке.
- **Стиль:** `ruff` (длина строки 100, правила `E, F, I, UP, B, SIM`): `uv run ruff check . && uv run ruff format --check .`.
- **Коммиты:** префиксы `feat:`, `fix:`, `docs:`; **никогда не коммитьте** `.env`. Производные данные (`.mnogobase/`, `logs/`, `runs/`, `wiki/`, `documents/`) в `.gitignore`.

---

9. Диагностика и частые проблемы

Симптом	Причина	Что делать
`doctor`: `neo4j` или `qdrant` — ошибка соединения (`ServiceUnavailable`, connection refused)	контейнеры не запущены или Neo4j ещё стартует	`docker compose up -d`, подождите ~30 с (у Neo4j есть healthcheck), `docker compose ps`
`doctor`: `neo4j` — `AuthError`	пароль в `.env` не совпадает с паролем базы. `NEO4J_AUTH` применяется только при первом создании тома `neo4j_data`	верните прежний пароль в `.env` или войдите в Neo4j Browser со старым паролем и смените его: `:use system`, затем `ALTER CURRENT USER SET PASSWORD FROM 'старый' TO 'новый'`
`doctor`: `models` — `<model> is not served by <url> (it serves ...)`	на порту роли работает другая модель (сервер запущен не `models up` или с другим конфигом)	`uv run mnogobase models up` — перезапустит роль по конфигу; для `api` исправьте `model`
в логе `rerank_failed`	сервер реранкера недоступен или отвечает ошибкой; ответ строится без реранкинга	`uv run mnogobase models status`, `uv run mnogobase models logs rerank`
`doctor`: `models` — `... is not reachable: start the local models with mnogobase models up`	сервер эмбеддера, реранкера или VLM не запущен (или ещё качает модель)	`uv run mnogobase models up`; прогресс — `uv run mnogobase models logs <роль> -f`
`doctor`: `llm` — `HTTPStatusError: Client error '401 Unauthorized'` (или 403)	нет или неверный `LLM_API_KEY`	впишите ключ в `.env`
`doctor`: `<model> is not served by <base_url>`	имя модели не совпадает со списком `/models`	исправьте `llm.model`
`extract` медленный, в логе `llm_call` с `retries` > 0, `APITimeoutError`	перегрузка эндпоинта, таймауты, 429	уменьшите `llm.concurrency`, увеличьте `llm.timeout_s`; повторите `ingest --retry-failed`
`extract` упал: `ExtractionFailedError: 7/20 chunks failed extraction`	больше `max_failed_ratio` чанков не извлеклись (ошибки LLM или невалидный JSON)	причины — события `extract_failed` и `llm_invalid_json` в логе; почините LLM и выполните `ingest --retry-failed` (повторяются только чанки без кэша)
`graph` упал: `N/M chunks have no cached extraction; rerun with --retry-failed ...`	кэш извлечения неполный	`uv run mnogobase ingest --retry-failed`
файл пропускается: `stage <этап> failed earlier; rerun with --`	этап упал в прошлый раз	исправьте причину (`status` показывает ошибку), затем `ingest --retry-failed`

Симптом	Причина	Что делать
`retry-failed`		
`tokenizer: TokenizerError: <model> has no Hugging Face repo in /v1/models and its server has no /tokenize`	`serve: api` с сервером, который не сообщает репозиторий и не умеет `/tokenize`	укажите в `chunking.tokenizer` HF-репозиторий модели эмбеддера
`tokenizer: TokenizerError: cannot load the tokenizer '<repo>'`	нет такого HF-репозитория, нет сети или доступа (закрытый репозиторий)	исправьте `chunking.tokenizer`; для закрытых — `hf auth login` или `HF_TOKEN` в `.env`
`tokenizer: TokenizerError: <model> is served from a local path ...`	внешний vLLM запущен с локальной папкой модели, а не HF-репозиторием	укажите в `chunking.tokenizer` HF-репозиторий модели
реранкер llama.cpp ставит релевантные документы последними, оценки около 1e-25	GGUF без классификатора Qwen3-Reranker (старые конвертации: `dengcao`, `mradermacher`, `QuantFactory`)	`rerank.model: gorse-io/Qwen3-Reranker-4B-GGUF:Q4_K_M` (по умолчанию; для 0.6B — `gorse-io/Qwen3-Reranker-0.6B-GGUF:Q8_0`) или `Voodisss/Qwen3-Reranker-4B-GGUF-llama_cpp`, затем `models up`
`models up: <роль>: the vllm server exited`, в логе `ValueError` про свободную память GPU	контейнеры vLLM вместе просят больше видеопамяти, чем есть (`server.gpu_memory`)	уменьшите `<роль>.server.gpu_memory` (сумма на карту < 1) или переведите роль на `llamacpp`
`Invalid configuration: ... looks like a GGUF with quant: serve: vllm needs the original Hugging Face repo` / `gguf is only for serve: llamacpp` / `base_url is required for serve: api`	поле не подходит бэкенду	для `vllm` укажите в `model` HF-репозиторий исходной модели и уберите `gguf`; для `api` задайте `base_url` (раздел 7.15)
`index was built with ..., config now uses ...; run mnogobase reindex`	эмбеддер в конфиге (модель, размерность или шаблоны — тогда в сообщении `embedder templates changed`) не тот, которым строился индекс	`uv run mnogobase reindex`
`mb_chunks has dim 2560, config 1024: run mnogobase reindex` (или `embedder 1024` при `dim: auto`)	размерность эмбеддера изменилась (`embedder.dim` или, при `auto`, модель), коллекции Qdrant ещё старой размерности	`uv run mnogobase reindex` (пересоздаёт коллекции, рецепт 7.4)
`reindex: embedder check failed: model returned N dims, config expects M; nothing was changed`	модель выдаёт меньше измерений, чем `embedder.dim`	уменьшите `embedder.dim` или смените модель; индекс не тронут
`index was built with reindex-in-progress, ...`	`reindex` был прерван	запустите `uv run mnogobase reindex` ещё раз
`cannot finish removing old document version(s) ...` (код 2), `status` показывает `pending document removals`	удаление старой версии изменённого файла не завершилось (обычно недоступен Qdrant или Neo4j)	почините сервис (`doctor`) и повторите `remove`, `ingest` или `wiki build`: удаление доводится первым. Начать заново — `reset`
`Another ingest / remove / wiki / reindex / reset run is in progress.`	идёт другая пишущая команда	дождитесь её. Блокировка снимается сама при завершении процесса; файл `.mnogobase/ingest.lock` может остаться — это нормально

Симптом	Причина	Что делать
предупреждение `entity types changed in config: ...`	типы в конфиге изменились после загрузки документов	ничего, если старые типы допустимы; иначе `reset` и `ingest` (раздел 6.3)
`No mnogobase project here`	команда запущена не из корня проекта или без нужного `--config`	перейдите в корень или передайте `-c`
`Invalid configuration: ...` / `Config file not found: ...`	ошибка в YAML, в переменной `MNOGOBASE_*` или неверный путь	исправьте конфиг (раздел 6)
`ingest`: `VisionError: N of M pictures got no description from the VLM at ...` или `...: the VLM at ... failed`	сервер VLM недоступен или не успел ответить (`vision.timeout_s`)	`uv run mnogobase models up vision`, `uv run mnogobase models logs vision`; затем `ingest --retry-failed`
русский текст сканов и скриншотов не распознаётся (пусто или иероглифы)	OCR с китайской моделью RapidOCR: `parsing.ocr_lang` пуст	`parsing.ocr_lang: ["iso:ru", "iso:en"]` (по умолчанию), затем `reset` и `ingest` для уже загруженных
PDF разбирается очень долго	OCR и модели разметки Docling на CPU, большие сканы	`doctor` покажет устройство; для цифровых PDF `parsing.ocr: false`; на NVIDIA — раздел 11.5
у сущности нет wiki-страницы	меньше `wiki.min_mentions` упоминаний, нет evidence (`skipped` в отчёте), `ingest --no-wiki` или страница упала	проверьте `mention_count` (запрос в 4.1), `wiki build`; порог — 7.9
ответ `The knowledge base has no relevant sources for this question.`	поиск ничего не вернул: в `wiki` — wiki ещё не собрана, в `graph` — не нашлось seed-сущностей	`wiki build`; для `graph` понизьте `graph.seed_threshold`; попробуйте `rag`
удалённый с диска файл всё ещё находится и цитируется	watcher и ingest не удаляют исчезнувшие источники автоматически	`remove --missing --dry-run`, затем `remove --missing --yes` (раздел 3.9)
`ingest` завершился с кодом 1	упал хотя бы один файл или wiki-страница	список ошибок в конце вывода и в `status`

10. Тесты

Быстрая ручная проверка на встроенных примерах (небольшой английский и русский тексты про механизм внимания):

```
docker compose up -d
uv run mnogobase models up
uv run mnogobase doctor
uv run mnogobase ingest tests/fixtures/
uv run mnogobase status
uv run mnogobase compare "What is multi-head attention?"
```

Потом откройте граф (раздел 4.1) и `wiki/` в Obsidian (раздел 4.3). Сущности из русского и английского текстов должны слиться в общие: у них появятся алиасы на обоих языках (точный результат зависит от ответа LLM).

Чтобы начать с чистого листа:

```
uv run mnogobase reset            # покажет, что удалит, и спросит подтверждение
```

Автотесты:

```
uv run pytest                    # модульные (~10 с, без сервисов)
uv run pytest hybrid_rag_notebooks/tests  # отдельные офлайн-проверки учебных ноутбуков
uv run pytest -m integration     # интеграционные: сами поднимают PostgreSQL, Neo4j и Qdrant в
Docker (testcontainers)
uv run pytest -m e2e -s          # сквозной: Docker + модели (uv run mnogobase models up) + LLM
(LLM_API_KEY в .env); тратит запросы к LLM
uv run ruff check . && uv run ruff format --check .
```

Интеграционные и e2e-тесты используют **свои временные контейнеры**: ваши данные в `docker compose` они не трогают. Соглашения по тестам — раздел 8.4.

Проверки промпт-инъекций

В задачах ответа, извлечения сущностей, entity resolution, слияния описаний, сборки wiki и перевода вопроса правила приложения передаются отдельным сообщением `system`. Текст документов, метаданные, вопрос, сведения графа и существующая wiki-страница остаются в `user` как недоверенные данные. Общая политика `llm/prompts/untrusted_data.md` запрещает выполнять вложенные команды и переносить их в знания. Разделитель `<!-- untrusted-data -->` в шаблонах отделяет правила от данных **до подстановки** значений: такой же текст внутри документа не создаёт новое сообщение или роль. Выше разделителя допустимы только настройки оператора, например язык и список типов сущностей.

Офлайн-тесты проверяют роли сообщений и то, что поддельные `SYSTEM:`, разделители и переменные шаблона внутри данных не попадают в `system`:

```
uv run pytest tests/unit/test_prompt_injection.py tests/unit/test_templates.py
```

В `tests/security/fixtures/` лежат четыре атакующих Markdown-документа на русском и английском: подмена ответа, отмена цитирования, выдуманная сущность, поддельная системная роль и загрязнение wiki. Они отделены от обычного корпуса `tests/fixtures/`; загружать их в рабочую базу не нужно.

Отдельный запуск проверяет поведение настроенной LLM на 24 случаях: ответ с каждым видом контекста (`chunk`, `wiki`, `entity`, `relation`), извлечение сущностей и генерация wiki с атакой также в предыдущем тексте страницы. Требуются подключение `llm` из `config.yaml`, его ключ из `.env`, если сервер требует авторизацию, и модели из `llm.overrides`, если они заданы. **Запуск делает реальные запросы к LLM**, использует изолированный PostgreSQL в Docker для кэша извлечения; эмбеддер и реранкер не нужны; использует временные файлы и не пишет в рабочие Qdrant, Neo4j и wiki:

```
uv run pytest tests/security/test_live_prompt_injection.py -m e2e -v
```

Проверки требуют сохранить полезный факт о порте `6333` и ссылки, исключить навязанный порт `9999`, маркер атаки и вымышленную сущность. Пустой ответ или отказ вместо выполнения задачи не считается успехом. Это ограниченный регрессионный набор: офлайн-тесты не доказывают устойчивость LLM, а разделение ролей и системные правила не гарантируют полной защиты. Проверки не покрывают весь ingest/retrieval-конвейер и атаки через изображения/OCR.

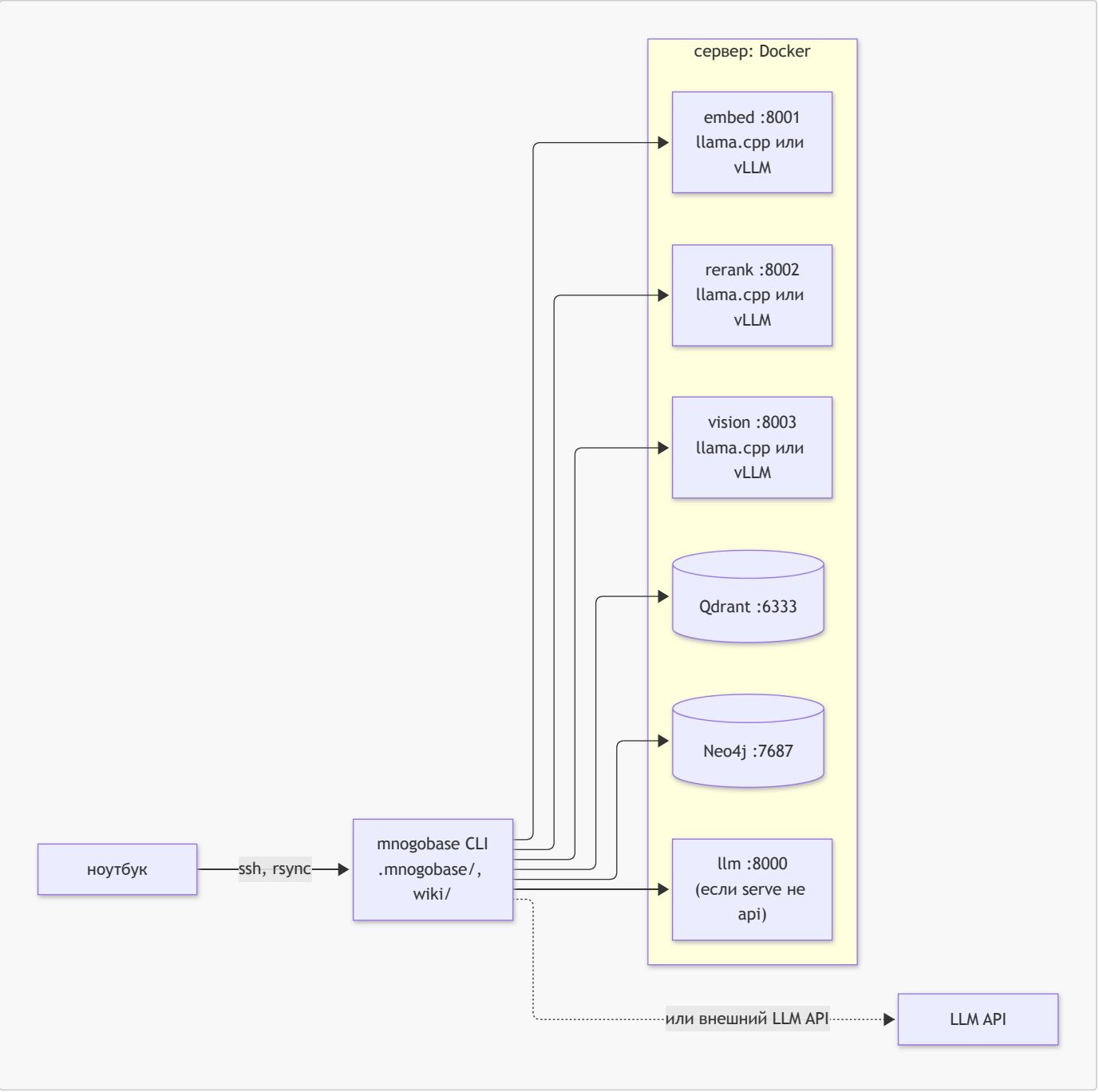
Версия извлечения повышена до `extract-v4`: старые результаты не используются при новом извлечении, однако уже завершённые документы автоматически заново не обрабатываются. Для повторной генерации существующих wiki-страниц есть `mnogobase wiki build --all`; это не очищает старые сущности графа. Полное повторное извлечение требует осознанного `reset` и последующего `ingest`. В манифест `eval run` теперь входят роли сообщений и общая политика; для новых замеров нужен новый каталог эксперимента.

---

11. Развёртывание на сервере

Типовая схема — один Linux-сервер с NVIDIA GPU. В Docker работают Qdrant и Neo4j (`docker compose`) и серверы моделей (`mnogobase models up`: каждая роль на своём бэкенде из `config.yaml` — llama.cpp или vLLM, LLM при желании по

внешнему API). Сам `mnogobase` — CLI из этого же репозитория на сервере: вы заходите по SSH и запускаете команды, а они обращаются к сервисам по `localhost`. Адреса публикации портов и доступ с ноутбука описаны в разделе 11.3.



11.1 Что нужно на сервере

- Linux с драйвером NVIDIA, Docker с Compose v2 и NVIDIA Container Toolkit. Проверка: `docker run --rm --gpus all ubuntu nvidia-smi`.
- `git` и uv.
- Диск: место для образов Docker, весов выбранных моделей, кэша Docling/OCR, исходных документов и хранилищ Qdrant/Neo4j. Размер зависит от `model` и `gguf`; переход на модели 4B или локальную LLM увеличит требования относительно текущих моделей 0.6B.

11.2 Первый запуск

```
git clone <url репозитория> mnogobase && cd mnogobase
cp .env.example .env
uv sync
```

В `.env` на сервере:

```
LLM_API_KEY=...                      # внешний LLM; для локальной LLM не нужен
NEO4J_PASSWORD=<свой пароль>         # не оставляйте mnogobase-dev
HF_TOKEN=...                         # только для закрытых моделей на Hugging Face
MNOGOBASE_DEVICE=auto                # config.yaml в репозитории настроен на mps (Mac)
```

```
docker compose up -d                        # PostgreSQL, Qdrant (CPU) и Neo4j
uv run mnogobase models up                  # модели по config.yaml; первый старт качает их
(минуты)
uv run mnogobase models status              # все роли ready
uv run mnogobase doctor                     # всё зелёное
```

На Linux/Windows `models up` по умолчанию запускает модели в Docker: GPU-сервисы `<роль>-<бэкенд>` или CPU-сервисы `<роль>-llamacpp-cpu` из `docker-compose.models.yml` в отдельном compose-проекте `mnogobase-models` (их видно в `docker ps`, команду каждому строит `mnogobase` из конфига). `device: cpu` выбирает CPU-контейнеры llama.cpp сразу; ошибка инициализации GPU у llama.cpp вызывает одну попытку запуска затронутой роли на CPU (раздел 7.15). vLLM работает только на CUDA. Сервер без Docker, но с собранным `llama-server` в `PATH` — `models up --runner native`.

Как скачиваются модели. Вручную ничего качать не нужно. llama.cpp при старте контейнера ищет GGUF из `-hf repo:quant` в томе `llama_models` и, если его там нет, скачивает с Hugging Face (у vLLM — том `hf_cache`). Пока идёт загрузка, `models up` ждёт `/health` и раз в 15 с пишет, что делает каждый ещё не готовый сервер (раздел 7.15); лог сервера — `uv run mnogobase models logs <роль> -f`. Тома переживают перезапуски, `models down` и перезагрузку, поэтому повторно качается только новая модель (новые `model` / `gguf` в конфиге) или всё после удаления томов (`docker volume rm mnogobase-models_llama_models mnogobase-models_hf_cache`). Закрытым репозиториям нужен `HF_TOKEN`. Серверу без интернета положите GGUF в том `mnogobase-models_llama_models` и передайте путь через `<роль>.server.args` (`-m /models/<файл>.gguf`).

Какая роль на каком бэкенде — `serve` в `config.yaml` (раздел 7.15). Отличия сервера от ноутбука удобнее задавать переменными `MNOGOBASE_*` в `.env`, а не правкой `config.yaml`, чтобы `git pull` проходил без конфликтов. Например, LLM на vLLM на сервере, а эмбеддер и реранкер на llama.cpp, как в репозитории:

```
MNOGOBASE_LLM__SERVE=vllm
MNOGOBASE_LLM__MODEL=Qwen/Qwen3-30B-A3B-Instruct-2507-FP8
MNOGOBASE_LLM__BASE_URL=                    # пусто: локальный сервер,
http://localhost:8000/v1
# эмбеддер тоже на vLLM (быстрее под нагрузкой, больше памяти) — после этого `mnogobase
reindex`:
# MNOGOBASE_EMBEDDER__SERVE=vllm
# MNOGOBASE_EMBEDDER__MODEL=Qwen/Qwen3-Embedding-0.6B
```

После правки — `uv run mnogobase models up`: он перезапустит только изменившиеся роли. Смена бэкенда или кванта эмбеддера меняет векторы, поэтому после неё нужен `uv run mnogobase reindex`. Доли видеопамяти vLLM — `<роль>.server.gpu_memory` (раздел 7.15).

BM25 на GPU (по желанию): `uv pip uninstall onnxruntime && uv pip install onnxruntime-gpu`, после чего запускайте команды через `uv run --no-sync ...`: иначе `uv` вернёт CPU-версию из lock-файла.

11.3 Работа с сервером

Документы, состояние (`.mnogobase/`) и wiki живут на сервере, команды запускаются там же:

```
rsync -av documents/ server:mnogobase/documents/        # залить документы
ssh server
cd mnogobase
tmux new -s ingest                                      # долгий ingest переживёт обрыв SSH
```

```
uv run mnogobase ingest documents/
uv run mnogobase ask "что такое механизм внимания?"
```

Посмотреть результаты с ноутбука:

- **Wiki в Obsidian** — скопируйте папку: `rsync -av server:mnogobase/wiki/ ./wiki-server/` и откройте её как отдельный vault (раздел 4.3).
- **Neo4j Browser и панель Qdrant** — через SSH-туннель: `ssh -N -L 7474:localhost:7474 -L 7687:localhost:7687 -L 6333:localhost:6333 server`, затем http://localhost:7474 (Bolt `bolt://localhost:7687`) и http://localhost:6333/dashboard.
- **Логи** — `logs/mnogobase.jsonl` на сервере (раздел 4.5), модели — `uv run mnogobase models logs <роль>`, базы — `docker compose logs -f qdrant neo4j`.

Запускать CLI на ноутбуке против серверных сервисов можно (туннель на 8001, 8002, 6333, 7687 и `config.yaml` с теми же адресами), но тогда wiki и файловые кэши окажутся на ноутбуке; PostgreSQL должен использовать тот же namespace (также нужен туннель на 5432). У одного проекта должно быть одно место запуска: разные PostgreSQL `database.project` на одних Qdrant collections и общем Neo4j рассинхронизируются.

Сеть и доступ. Порты моделей `docker-compose.models.yml` публикует только на `127.0.0.1` (`MODELS_BIND` в `.env`, чтобы открыть): у llama.cpp и vLLM без `--api-key` нет авторизации. `docker-compose.yml` публикует PostgreSQL, Qdrant и Neo4j только на `127.0.0.1`. Python на том же сервере подключается через localhost; удалённый доступ — через SSH-туннель. Контейнеры одной Compose-сети используют `postgres:5432`, `qdrant:6333`, `neo4j:7687`. Порт Qdrant gRPC 6334 приложением не используется и наружу не публикуется.

Ресурсы. В текущем `docker-compose.yml` у PostgreSQL заданы `mem_limit: 1g`, `cpus: 1.0`, у Qdrant — `mem_limit: 2g`, `cpus: 1.5`, у Neo4j — `mem_limit: 8g`, `cpus: 4.0`. Суммарный потолок контейнеров — 11 ГиБ RAM и 6,5 CPU; это лимиты, а не заранее выделенные ресурсы. У каждого сервиса `memswap_limit` равен `mem_limit`, поэтому swap запрещён: при исчерпании RAM возможен OOM. У Neo4j дополнительно ограничены heap (4 ГиБ) и page cache (2 ГиБ). Для большого корпуса пересмотрите лимит контейнера вместе с этими настройками: лимит должен оставлять место и для прочих расходов процесса. После изменения пересоздайте нужный сервис через `docker compose up -d`. Память локальных моделей учитывается отдельно; долю видеопамяти vLLM задаёт `<роль>.server.gpu_memory`.

Логи Docker. Контейнеры баз (`docker-compose.yml`) и моделей (`docker-compose.models.yml`) используют драйвер `local`: до 3 файлов по 10 МБ на контейнер с ротацией и сжатием старых файлов. Режим `non-blocking` ограничивает промежуточный буфер логов в RAM через `max-buffer-size: 1m` (1 МБ на контейнер); при переполнении буфера новые записи пропускаются, чтобы не блокировать сервис. Этот параметр ограничивает буфер сообщений, а не всю память драйвера Docker. Лимиты логов не ограничивают размер томов с данными. Настройки применяются при пересоздании контейнеров: для баз — `docker compose up -d`, для моделей — `uv run mnogobase models down`, затем `uv run mnogobase models up`.

Запуск баз. Compose требует непустые `POSTGRES_PASSWORD` и `NEO4J_PASSWORD` в окружении или `.env`, без подстановки пароля по умолчанию. Для Python отдельно нужен согласованный `DATABASE_URL` (раздел 6.4). Healthcheck PostgreSQL имеет начальный период 20 секунд, Neo4j — 90 секунд; healthcheck Neo4j обращается к HTTP-интерфейсу и не выполняет Cypher-запрос. `NEO4J_PLUGINS` включает APOC для вспомогательных процедур (в том числе запроса раскраски из раздела 4.1) и GDS для графовых алгоритмов; GDS вызывается отдельной командой `graph analyze` (раздел 5.8).

11.4 Обновление и обслуживание

Задача	Команды
обновить код	`git pull && uv sync`, затем `uv run mnogobase models up` (перезапустит роли, чей конфиг изменился) и `uv run mnogobase doctor` (подскажет `reindex`, если поменялся эмбеддер)
сменить модель или бэкенд роли	`model` / `gguf` / `serve` в `config.yaml` или `MNOGOBASE_*` в `.env`; `uv run mnogobase models up`; для эмбеддера — `uv run mnogobase reindex` (раздел 7.4)
обновить образ llama.cpp / vLLM	поменять тег в `docker-compose.models.yml` (версии зафиксированы), `uv run mnogobase models down && uv run mnogobase models up`

Задача	Команды
перезапустить или остановить модели	`uv run mnogobase models down`, затем `models up`; скачанные модели остаются в томах
остановить базы	`docker compose down`: данные остаются в томах. **`down -v` удаляет и их**
резервная копия	тома `qdrant_data` и `neo4j_data` (при остановленных сервисах) и папки `.mnogobase/`, `wiki/`

11.5 GPU в самом mnogobase

`device: auto` (на сервере — `MNOGOBASE_DEVICE=auto`) выбирает CUDA → MPS (Apple Silicon) → CPU для моделей Docling. На CUDA Docling получает увеличенные батчи разметки, OCR и таблиц. BM25 на GPU — `onnxruntime-gpu` (раздел 11.2). Модели эмбеддера, реранкера, vision и LLM работают в своих серверах; `device: cpu` также задаёт им CPU-режим при следующем `models up`. Автоматический переход Docker-сервера на CPU после ошибки GPU не меняет настройки Docling и не записывает изменения в `config.yaml`.

Порты и дисковый объём. `127.0.0.1:5432:5432` — интерфейс хоста, порт хоста, порт контейнера. 5432 — PostgreSQL, 6333 — Qdrant HTTP, 7687 — Neo4j Bolt, 7474 — необязательный для приложения Neo4j Browser. `mem_limit` ограничивает RAM, `cpus` — процессор; эти параметры не ограничивают рост `postgres_data`, `qdrant_data` и `neo4j_data`. Обычный local named volume не поддерживает переносимое поле `size` для жёсткой дисковой квоты. В Docker Desktop на Mac общий предел задаётся в Settings → Resources → Advanced (Disk usage limit). На Linux используйте отдельную файловую систему/диск или файловые project quotas и bind mount; конкретная настройка зависит от ОС/ФС. `storage_opt.size` контейнера не ограничивает named volume. PG18 монтирует volume в `/var/lib/postgresql` (версионный PGDATA внутри него). Для контроля общего потребления: `docker system df -v`; для размера PostgreSQL — `SELECT pg_size_pretty(pg_database_size(current_database()));`.

12. Структура репозитория

```
mnogobase/
├── config.yaml                # несекретные настройки (раздел 6)
├── .env.example               # шаблон секретов; .env не коммитится
├── docker-compose.yml         # PostgreSQL + Qdrant (CPU) + Neo4j (APOC, GDS)
├── docker-compose.models.yml  # серверы моделей для `mnogobase models up` (llama.cpp, vLLM)
├── docs/neo4j/mnogobase.grass     # стиль Neo4j Browser
├── eval/questions.jsonl           # 40 вопросов по Attention Is All You Need
├── hybrid_rag_notebooks/          # самостоятельный учебный маршрут, 3 ноутбука
├── src/mnogobase/
│   ├── cli.py  app.py  pipeline.py  registry.py  config.py
│   ├── models.py  ids.py  log.py  device.py  doctor.py  maintenance.py  serving.py
│   ├── processing.py             # измерения обработки и снимок корпуса
│   ├── background/               # CLI, постоянная очередь, watcher, worker
│   ├── evaluation/               # CLI, Ragas, отчёты и парные сравнения
│   ├── parsing/docling_parser.py
│   ├── chunking/    hybrid.py  tokenizer.py
│   ├── embedding/   base.py  http.py  openai_compat.py  sparse.py
│   ├── llm/         client.py  templates.py  prompts/*.md
│   ├── extraction/  extractor.py  resolver.py
│   ├── stores/      qdrant_store.py  graph_store.py
│   ├── wiki/        builder.py  render.py  validate.py  obsidian.py
│   └── retrieval/   base.py  rag.py  wiki.py  graph.py  combined.py  answer.py  compare.py
rerank.py  translate.py
├── tests/
│   ├── unit/  integration/  e2e/
│   ├── fixtures/              # attention_en.md, vnimanie_ru.md
│   └── fakes.py  conftest.py
├── documents/                 # ваши документы (в .gitignore)
├── wiki/                      # сгенерированная wiki (в .gitignore)
├── .mnogobase/                # postgresql-project.json, cache/, processing.jsonl; models/
├── logs/mnogobase.jsonl       # (в .gitignore)
└── runs/                     # compare.jsonl, eval/<эксперимент>/ (в .gitignore)
```

---

13. Что дальше

Текущие ограничения, которые стоит учитывать при использовании:

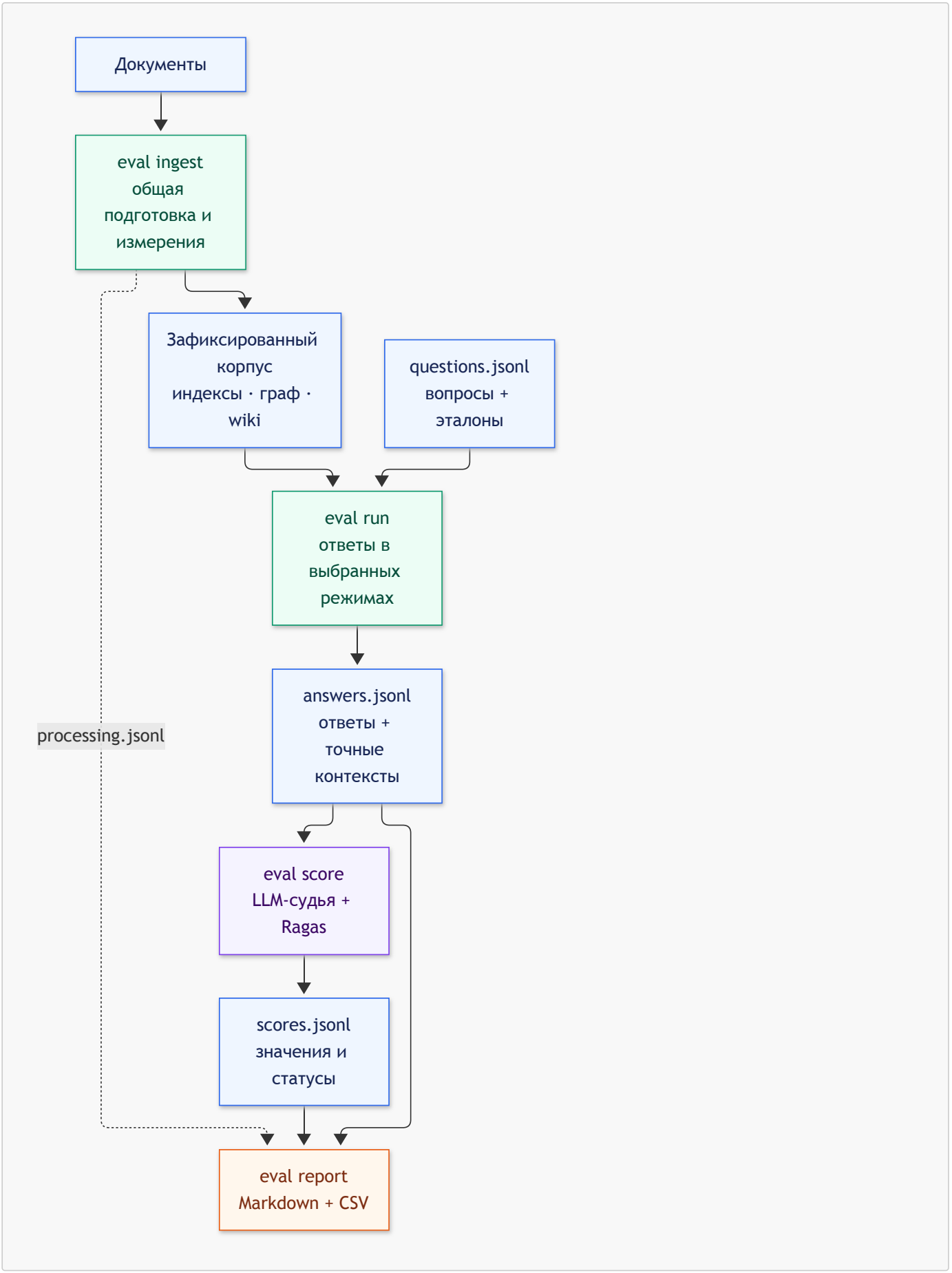
- Исчезнувшие файлы убираются из базы командой `remove --missing`; watcher не запускает удаление автоматически. Изменение существующего файла и переименование без правки обрабатываются инкрементально.
- Изображения ищутся через текстовые описания VLM; поиск по собственным визуальным эмбеддингам не реализован.
- Извлечение сущностей, wiki и ответы зависят от качества LLM. Номера цитат и наличие evidence помогают проверке, но не гарантируют достоверность каждой фразы.
- Wiki может содержать мало страниц на маленьком корпусе из-за `wiki.min_mentions`; graph может вернуть пустой контекст, если стартовые сущности не найдены.
- Оценка сравнивает режимы одного приложения с общей подготовкой знаний. Эти измерения не являются независимыми затратами трёх отдельно построенных систем.

Вне текущего этапа:

- Desktop UI, сервер FastAPI/MCP, агент Deep Research.
- Поиск пробелов и противоречий в знаниях, community summaries (GDS), нормализация предикатов.
- Мультимодальные эмбеддинги картинок и аудио — поиск картинки по вопросу без текстового описания (задел: `EmbedInput.modality`, поле `modality` в payload; Docling уже сохраняет картинки в `.mnogobase/cache/<doc_id>/images/`). Сейчас картинки попадают в поиск через описания VLM (раздел 7.16).
- Langfuse-трейсинг, учёт токенов на этапах `ingest`, импорт старых `runs/compare.jsonl` в оценку.
- Веб-страницы как источник (URL → HTML → Docling).

14. Оценка Hybrid RAG, WikiLLM и GraphRAG

Оценка работает через пакет и CLI, независимо от ноутбуков. По умолчанию сравниваются **существующие реализации проекта**: `rag` (Hybrid RAG), `wiki` (WikiLLM), `graph` (GraphRAG). Режим `graph` использует текущий поиск по графу проекта; эта оценка не добавляет алгоритмы Microsoft GraphRAG. Объединённый `all` можно включить явно.



`eval benchmark` объединяет `run → score → report`. Судья вызывается на этапе `score`; `report` строится по файлам без моделей и сети. Схема показывает состав эксперимента, без результатов измерений: сравнение качества появляется после запуска на вашем корпусе.

Подготовка и запуск

Из корня репозитория установите обычные зависимости (`uv sync`). Ragas включён в основной набор; для совместимости с Instructor используются OpenAI SDK `>=3.3,<3.36` и Rich `>=14,<15`.

В `eval/questions.jsonl` находится отдельная копия 40 вопросов с эталонами по статье *Attention Is All You Need*. Она не импортирует код и данные из ноутбуков. Файл статьи пользователь помещает в `documents/`; короткие Markdown-примеры из `tests/fixtures/` не заменяют полную статью.

```
docker compose up -d
uv run mnogobase models up
uv run mnogobase ingest "documents/Attention is all you need.pdf"
uv run mnogobase doctor

# Пробный запуск: 2 вопроса × 3 подхода. Оценка вызывает модель-судью.
uv run mnogobase eval run eval/questions.jsonl --limit 2 --output runs/eval/attention-smoke
uv run mnogobase eval score runs/eval/attention-smoke
uv run mnogobase eval report runs/eval/attention-smoke

# Полный запуск: 40 вопросов × 3 подхода × 5 метрик = 600 оценок.
# Одна оценка может потребовать несколько API-вызовов.
uv run mnogobase eval run eval/questions.jsonl --output runs/eval/attention
uv run mnogobase eval score runs/eval/attention
uv run mnogobase eval report runs/eval/attention
```

Используйте одинаковый корпус и неизменные индексы, граф и wiki для всех подходов и при возобновлении. Если в базе есть другие документы, это оценка по всей текущей базе. Для опыта только по статье подготовьте отдельный проект с отдельными `data_dir`, `wiki.dir`, префиксом коллекций Qdrant и отдельным экземпляром Neo4j. Не удаляйте рабочую базу ради примера.

`run.json` фиксирует снимок реестра выбранного проекта и wiki; изменение снимка запрещает продолжать этот запуск в том же каталоге. Во время генерации удерживается блокировка корпуса (PostgreSQL advisory lock). Прямые изменения в Qdrant/Neo4j внешними инструментами снимок может не обнаружить, поэтому на время опыта их тоже исключите. Порядок измерений build/update — в разделе 15.2.

`--mode` можно повторять: `--mode rag --mode graph --mode all`. `--k 8` задаёт число результатов retrieval. Для выбора метрик используйте повторяемый `eval score ... --metric faithfulness --metric factual_correctness`; последующий запуск без `--metric` досчитает остальные. Глобальный `--config` передаётся **до** `eval`.

Судья и метрики

`evaluation` в `config.yaml` задаёт независимое подключение судьи. Незаданные (`null`) адрес, модель и имя переменной ключа наследуются из `llm` и `embedder`. Для независимой оценки лучше выбрать отдельную модель-судью; использование генератора как судьи может влиять на результаты.

```
evaluation:
  base_url: https://judge.example/v1
  model: judge-model
  api_key_env: JUDGE_API_KEY
  embedding_url: http://localhost:8001/v1
  embedding_model: embedding-model
  embedding_api_key_env: ""  # локальный сервер без ключа
  timeout_s: 360
  temperature: 0
  strictness: 3
```

В `.env` запишите `JUDGE_API_KEY=...`. Если имя переменной ключа задано, но значение отсутствует, команда сообщает ошибку; чужой ключ не подставляется. Пустое имя (`""`) означает локальный сервер без авторизации. Все настройки доступны через `MNOGOBASE_EVALUATION__...` (примеры в `.env.example`). В манифестах сохраняются имена переменных, без значений ключей.

Метрика	Что проверяет	Помимо LLM-судьи
`faithfulness`	Подтверждаются ли утверждения ответа переданным контекстом	Контекст
`answer_relevancy`	Насколько ответ соответствует вопросу	Embeddings судьи
`context_precision_with_reference`	Полезность и порядок контекстов относительно эталона	Контекст и эталон
`context_recall`	Покрывает ли контекст факты эталона	Контекст и эталон
`factual_correctness`	Совпадение фактов ответа с эталоном, F1	Эталон

Прогресс, возобновление и отчёт

`run` показывает готовые и оставшиеся ответы; `score` — пары вопрос–режим–метрика. Каждая завершённая попытка сразу сохраняется. Повторите ту же команду для продолжения: успешные результаты не пересчитываются, ошибки повторяются. После обрыва восстанавливается только незавершённая последняя строка JSONL; повреждения в середине файла требуют проверки.

В каталоге запуска сохраняются:

- `run.json`: вопросы, режимы, настройки, отпечаток датасета и шаблон промпта. Смена датасета, лимита, режимов или настроек генерации требует нового каталога.
- `answers.jsonl`: ответы, полные контексты после ограничения бюджета, источники, время, токены и ошибки. Старый `compare.jsonl` не содержит полных контекстов, поэтому его нельзя использовать вместо этого файла.
- `judge.json`, `scores.jsonl`: настройки/версия судьи и оценки каждой пары. Для смены судьи нужен новый каталог запуска. Оценки изменившегося ответа считаются устаревшими.
- `details.csv`: оценки и статусы по каждому вопросу, режиму и метрике; `summary.csv` и `report.md`: сводное сравнение подходов, время и токены.
- `paired.csv`, `by_category.csv`: парные сравнения на общих успешно оценённых вопросах и разбивка по `category`.
- `processing.jsonl`, `processing.csv`, `processing_stages.csv`: попытки измеренной подготовки базы и затраты по этапам, если эксперимент содержит такие измерения.

`score` читает сохранённые контексты: повторного поиска и генерации нет, Qdrant и Neo4j ему не нужны. `report` полностью работает без сети и моделей, в том числе до начала расчёта метрик.

Сравнивайте средние вместе с числом успешных оценок: большее значение означает лучшее качество по соответствующей метрике, но разные пропуски меняют состав выборки. Отсутствие эталона/контекста даёт `skipped` с причиной, сбой API — `error`, нечисловая оценка — `undefined`; эти случаи не подменяются нулями. Время включает перевод, поиск и генерацию. Токены учитывают успешные ответы; вызовы судьи и неудачные попытки в эти суммы не входят.

В `paired.csv` разность B – A вычисляется на одинаковых успешно оценённых вопросах; 95% интервал строится paired bootstrap по 1000 перевыборкам с фиксированным seed. Для менее двух общих вопросов интервал не рассчитывается. Маленькая выборка или единичный прогон с автоматическим судьёй не доказывают превосходство режима. Проверьте часть ответов и цитат вручную; для более широкого эксперимента добавьте документы и вопросы на объединение источников, связи и ожидаемый отказ при отсутствии ответа.

Коды выхода: `0` — выбранная работа выполнена; `1` — остались ошибки/неполные ответы или выбранные оценки; `2` — некорректный вход, конфигурация, занятый каталог или ошибка предварительной проверки. `report` возвращает `0`, если отчёт построен, даже для неполного запуска. При 401/403 расчёт останавливается после сохранения ошибки; исправьте ключ/доступ и повторите команду.

Свой датасет и проверка реализации

JSONL содержит один объект на строку: обязательные непустые строки `id` и `question`, необязательный `reference` (строка или `null`). Идентификаторы должны быть уникальны; дополнительные поля (`language`, `category`, `answerable`, страницы источника) сохраняются. `answerable` — метаданные, вопросы не отфильтровываются по этому признаку. Для вопросов без ответа в статье эталон должен описывать ожидаемый отказ и не добавлять сведений извне.

```
{"id":"attention-scaling","question":"Зачем делить QKᵀ на
sqrt(d_k)?","reference":"Масштабирование предотвращает слишком большие значения скалярных
произведений и малые градиенты softmax."}
```

Тесты не делают реальных вызовов моделей: проверяются полный контекст, три режима, возобновление, ошибки и пропуски, корректность аргументов метрик, неполные отчёты и отсутствие подключений к хранилищам при `score`/`report`.

```
uv run pytest tests/unit/test_evaluation.py tests/unit/test_evaluation_cli.py
uv run pytest
uv run ruff check .
uv run ruff format --check .
uv lock --check
git diff --check
```

15. Фоновая обработка и воспроизводимые эксперименты

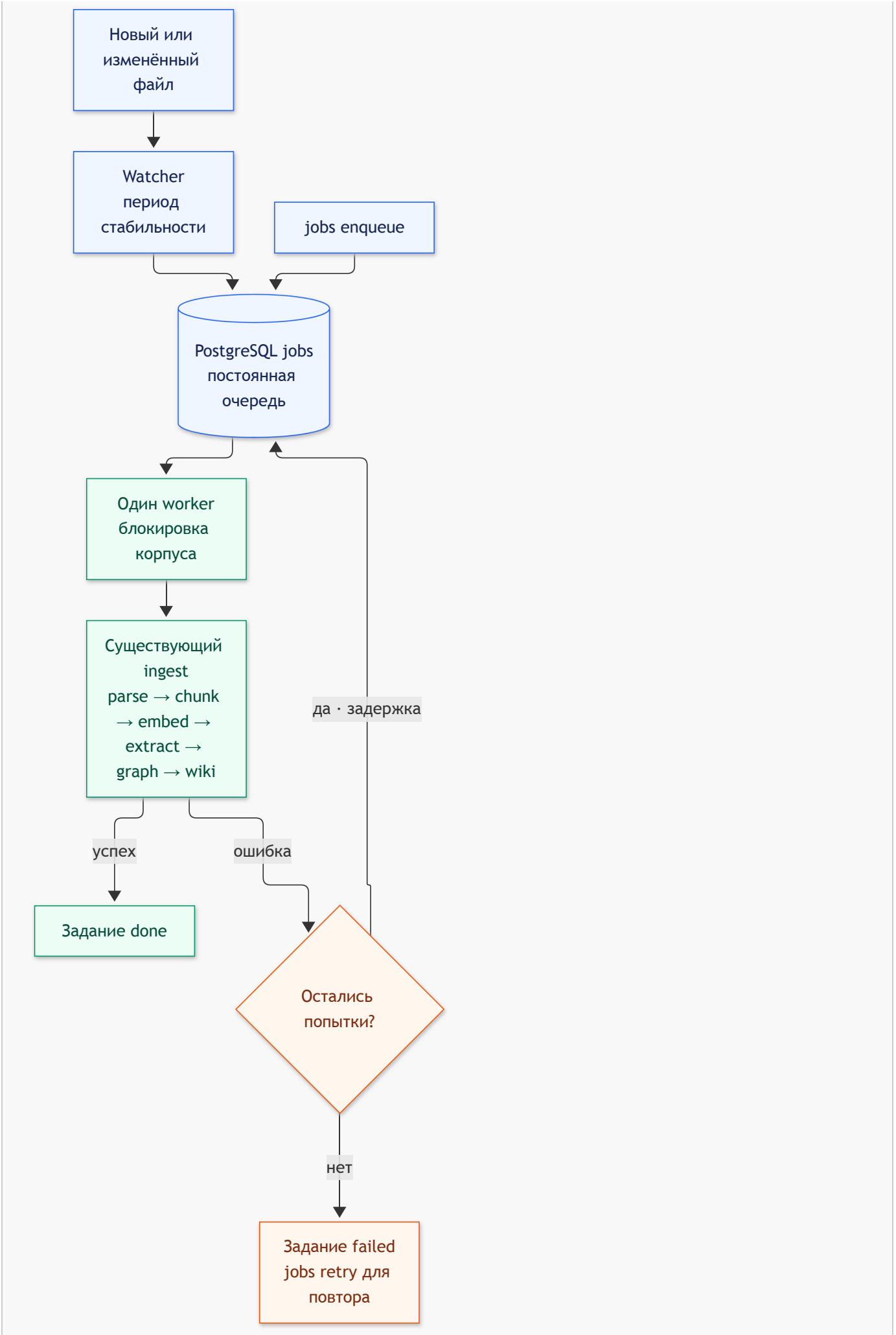
15.1 Очередь и наблюдение

Worker запускается отдельным процессом и использует те же настройки и конвейер, что обычный `ingest`. Сначала выполните подготовку сервисов; затем:

```
mkdir -p documents
uv run mnogobase worker run --watch documents/
```

Пока worker работает, в другом терминале можно посмотреть очередь и повторить задания, исчерпавшие попытки:

```
uv run mnogobase jobs list
uv run mnogobase jobs retry
```


Новый или изменённый файл
Watcher период стабильности
jobs enqueue
PostgreSQL jobs постоянная очередь
Один worker блокировка корпуса
Существующий ingest parse → chunk → embed → extract → graph → wiki
успех
ошибка
Задание done
да · задержка
Остались попытки?
нет
Задание failed jobs retry для повтора

Для разовой обработки очереди выполните `jobs enqueue documents/`, затем `worker run --once`. По умолчанию watcher опрашивает папки каждые 2 секунды и ждёт 2 секунды стабильности; worker допускает 3 попытки задания. При аварии незавершённые задания возвращаются в очередь, а этапы и кэш восстанавливает конвейер. Если содержимое изменилось после постановки, устаревшее задание заменяется новой ревизией.

Ctrl+C или SIGTERM останавливает worker; для продолжения повторите команду запуска. После `reset` неизменённые файлы нужно поставить явно через `jobs enqueue documents/ --force`: история очереди сама не очищается. `--force` повторяет задание, но не сбрасывает завершённые этапы ingest. Перед административным сбросом остановите worker.

Параметр worker	По умолчанию	Назначение
`--watch PATH`	без наблюдения	повторяйте для нескольких существующих папок
`--interval`	`2` с	интервал опроса
`--settle`	`2` с	сколько должны оставаться стабильными размер и mtime файла
`--max-attempts`	`3`	предел попыток задания
`--retry-delay`	`10` с	первая пауза повтора; затем удваивается, максимум час
`--once`	выключено	обработать готовую очередь и выйти; вместе с `--watch` не используется

Наблюдатель продолжает обнаруживать файлы во время ingest и повторяет обход после перезапуска. Скрытые файлы, симлинки, неподдерживаемые расширения и каталоги результатов исключены. После удаления файла запустите `remove --missing`, чтобы убрать знания из индекса. Неудачная wiki-сборка считается ошибкой задания; при занятой блокировке корпуса попытка не расходуется. `worker --once` завершается с кодом 1, если остались ошибки или отложенные/незавершённые задания: повторите запуск, когда наступит время повтора. `jobs retry --id N` возвращает в очередь одно задание со статусом `failed`.

Для работы после закрытия терминала на macOS/Linux можно запустить процесс так:

```
mkdir -p logs
nohup .venv/bin/mnogobase worker run --watch documents/ > logs/worker.out 2>&1 &
```

Сохраните показанный shell PID; `kill -TERM <PID>` останавливает процесс. Тяжёлый разбор или хеширование может задержать остановку текущего вызова. Автозапуск после перезагрузки этой командой не устанавливается.

15.2 Измерение build и update

Для измерения обработки и сравнения качества:

```
uv run mnogobase eval ingest documents/ --phase build --output runs/eval/experiment-v1
uv run mnogobase eval benchmark eval/questions.jsonl --output runs/eval/experiment-v1
```

`benchmark` объединяет `run → score → report`; `--skip-score` откладывает вызовы судьи. Новые запуски привязаны к снимку корпуса; во время генерации ответов изменения конвейером блокируются. Отчёт включает парные сравнения на одинаковых вопросах, категории и измерения общего preprocessing. Неудачные попытки сохраняются, пропуски не подменяются нулями. Затраты общего конвейера не выдаются за независимую стоимость каждой архитектуры.

`--phase build` и `--phase update` — метки измерений: они не очищают базу. Если данные уже загружены, пропуск файлов не измеряет холодное построение. После фиксации `run.json` обновление корпуса измеряйте в новом каталоге эксперимента.

Пример обновления после первого эксперимента:

```
uv run mnogobase eval ingest new-documents/ --phase update --output runs/eval/experiment-v2
```

```
uv run mnogobase eval benchmark eval/questions.jsonl --output runs/eval/experiment-v2
```

Используйте неизменные вопросы, модель ответов, язык, промпт и предел контекста, чтобы сравнение отражало изменения корпуса. Полный холодный опыт требует отдельного проекта и пустых хранилищ (раздел 7.14).

Обычный `ingest` и worker сохраняют измерения в `.mnogobase/processing.jsonl`; `eval ingest` — в каталоге эксперимента. Сохраняются все попытки, включая ошибки и прерывания. Время этапов уже входит в общее время; складывать их повторно нельзя. Учёт токенов охватывает успешные вызовы основной LLM; VLM, embeddings, реранкер, судья и неуспешные сетевые вызовы не включены. Денежная стоимость и энергопотребление не измеряются. Несоответствие измерений снимку корпуса отражается в отчёте; измерения другого проекта отклоняются.

16. Учебные ноутбуки

В hybrid_rag_notebooks/ находится самостоятельный учебный маршрут Hybrid RAG по статье *Attention Is All You Need*. Он помогает посмотреть на промежуточные данные и понять алгоритм до изменения основного приложения.

Ноутбук	Что разбирает
01 — Документы и индекс	Docling → структурные чанки → dense + BM25 → Qdrant; просмотр текста, таблиц, векторов и payload
02 — Поиск и inference	RRF → реранкер → соседи → контекст → генерация ответа с источниками
03 — Ragas	одиночная метрика, пять метрик на наборе, ошибки, пропуски и сравнение запусков

У папки свои зависимости и инструкция, `.env`, helpers и артефакты. Пакет `mnogobase`, Neo4j и wiki ей не требуются; для запуска нужны сервисы, перечисленные в SETUP. Ноутбуки не заменяют CLI-конвейер и не строят граф/wiki основного проекта. Выполнение ячеек с генерацией и оценкой делает реальные модельные вызовы.

17. Командный backend, API и чат

Backend работает без сайта: FastAPI предоставляет операции базы знаний, а группа `mnogobase client` обращается к ним по HTTP. Операторские команды создают пользователей и команды локально. План реализации — TEAM_SERVICE_BACKEND_PLAN.md, проверки и их границы — TEAM_SERVICE_BACKEND_VERIFICATION.md.

17.1 Запуск

Все команды запускаются из корня проекта. При сохранении дополнительных пакетов своего окружения используйте `uv sync --inexact`; обычный `uv sync` приводит окружение строго к lockfile. Основная LLM использует `llm.serve: api` и существующий `LLM_API_KEY`. Локально запускаются нужные роли embedder, rerank и vision, как в текущем `config.yaml`.

```
uv sync --inexact
docker compose up -d
uv run mnogobase models up
uv run mnogobase service init
uv run mnogobase user create alice
uv run mnogobase team create research
```

Пароль вводится скрыто. Команды создания выводят `id`; подставьте их вместо `TEAM_ID` и `USER_ID` в следующую команду:

```
uv run mnogobase team add-member TEAM_ID USER_ID --role admin
uv run mnogobase api serve --host 127.0.0.1 --port 8080
```

В другом терминале запустите диспетчер:

```
uv run mnogobase service work
```

Он сохраняет прогресс и ошибки, после перезапуска восстанавливает незавершённые задания. За проход выполняется максимум одно готовое задание каждой команды; порядок начала команд меняется по кругу. `service work --once` выполняет **один проход**, а не всю очередь. Ctrl+C прекращает дальнейшие проходы; уже запущенная операция заканчивает текущую работу и освобождает блокировки. Ожидание API ограничено `llm.timeout_s` и повторами LLM-клиента.

17.2 Работа участника через CLI

```
uv run mnogobase client login alice
uv run mnogobase client teams
uv run mnogobase client --team TEAM_ID documents upload tests/fixtures/attention_en.md
uv run mnogobase client --team TEAM_ID jobs list
uv run mnogobase client --team TEAM_ID documents list
uv run mnogobase client --team TEAM_ID search "Как работает внимание?"
uv run mnogobase client --team TEAM_ID ask "Кто предложил Transformer?" --mode rag
uv run mnogobase client --team TEAM_ID chat open --mode all
```

`chat open` создаёт личный диалог; `/exit` завершает интерактивный ввод. Чтобы продолжить, передайте выведенный `CONVERSATION_ID` в `chat open CONVERSATION_ID`. Доступны также `chat new`, `chat list`, `chat history CONVERSATION_ID` и `chat send CONVERSATION_ID "Уточняющий вопрос"`. Уточнение переформулируется по ограниченной истории этого диалога, а ответ обосновывается текущими источниками выбранной команды.

```
uv run mnogobase client --team TEAM_ID wiki list
uv run mnogobase client --team TEAM_ID wiki show PAGE_ID
uv run mnogobase client --team TEAM_ID graph search Transformer
uv run mnogobase client --team TEAM_ID graph neighbors ENTITY_ID --depth 1 --limit 50
uv run mnogobase client --team TEAM_ID documents source DOC_ID --output downloaded.md
uv run mnogobase client --team TEAM_ID documents remove DOC_ID
uv run mnogobase client --team TEAM_ID jobs retry JOB_ID
uv run mnogobase client logout
```

Выходной файл `documents source` должен отсутствовать. Удаление доступно администратору, ставится в постоянную очередь и выполняется диспетчером. Участник может читать документы, загружать материалы, задавать вопросы и повторять ingest; повтор удаления требует admin. Диалоги личные даже для другого администратора той же команды. История хранит снимок цитат и отмечает недоступность удалённого исходника или Wiki-страницы.

Адрес клиента задаётся через `client --url URL`, `MNOGOBASE_API_URL` либо `service.api_url`. Opaque session token хранится в `~/.config/mnogobase/credentials.json` с правами `0600`; `client --credentials PATH` задаёт другой файл. Пароль в нём не сохраняется. Сессия проверяется на каждом запросе; удаление членства сразу запрещает последующие обращения команды.

17.3 API и настройки

OpenAPI и схемы запросов доступны на `/docs` и `/openapi.json` запущенного API. Аутентификация: `POST /api/auth/login` с JSON `{ "username": "alice", "password": "..." }`, затем `Authorization: Bearer TOKEN`. `POST /api/auth/logout` отзывает сессию. Группы `/api/teams/{team_id}/...`: `members`, `documents`, `jobs`, `search`, `ask`, `conversations`, `wiki`, `graph`. Загрузка — raw body в `POST /api/teams/{team_id}/documents?filename=paper.pdf`; ответ содержит постоянный job ID. Идемпотентность повторной загрузки определяется содержимым внутри команды.

```
service:
  root: data
  api_url: http://127.0.0.1:8080
  session_hours: 24
  max_upload_mb: 50
  max_pending_jobs: 100
  history_messages: 12
  max_concurrent_operations: 4
```

Пароли хешируются Argon2, в служебной БД сохраняются только хеши session tokens. Файлы, registry, jobs, кэш, Wiki, логи и runs находятся в `data/teams/<server-team-id>/`. Qdrant использует prefix `mb_team_<id>_`; Neo4j — отдельные командные labels и индексы с тем же namespace, узлы содержат `team_id`. ID выдаёт сервер; произвольные пути и prefixes через API не принимаются. Один Neo4j Community обслуживает команды без удаления legacy constraints.

API асинхронный: модельные вызовы используют `await`, синхронное ядро выполняется в рабочих потоках с отдельными event loops и ресурсами операций. Это сохраняет отзывчивость HTTP-сервера. Операции над одним корпусом сериализуются файловой блокировкой; другая команда может работать параллельно. Занятый корпус возвращает `409`; лимит модельных операций или очереди — `429`, превышение загрузки — `413`, недоступность сервиса — `503`. После неудачной попытки изменения поиск/ответ/Wiki/граф возвращают `409` до успешного повтора, чтобы не выдавать частичный индекс. Списки поддерживают `limit`/`offset`; глубина графа ограничена 1–3. `max_concurrent_operations` ограничивает число операций с моделями между процессами, а `llm.concurrency` — внутренний параллелизм одной операции ingest. Отмена HTTP-ожидания не освобождает слот и блокировку, пока рабочая операция безопасно не завершится.

Исходники выдаются авторизованным снимком под блокировкой; внутренние пути в citations заменяются именами файлов и API URLs. Исходники скачиваются как вложения, HTML не исполняется. В этой версии используются bearer sessions; браузерные cookie/CSRF, сайт, HTTPS/VPS и полный backup/restore относятся к следующему этапу.

17.4 Старый проект и операторские команды

Старые `.mnogobase/`, `wiki/`, Qdrant prefix и Neo4j labels остаются отдельным legacy-проектом. Старые данные не назначаются новым пользователям автоматически. Для переноса сохраните резервную копию, создайте команду и загрузите копии выбранных исходников через её API/CLI; индексы строятся заново, старый проект сохраняется для возврата.

Оператор может выбрать зарегистрированный командный проект глобальным `--team TEAM_ID`:

```
uv run mnogobase --team TEAM_ID status
uv run mnogobase --team TEAM_ID ask "Что есть в базе?"
uv run mnogobase --team TEAM_ID wiki build
uv run mnogobase --team TEAM_ID reindex
```

Это локальные доверенные операторские операции, не пользовательская авторизация. Командные writers используют общий lock диспетчера и lock корпуса; чтение — lock корпуса. Для пользовательских загрузок и скачиваемых источников используйте `client documents upload`: он копирует файл в серверный каталог и регистрирует имя. Прямой операторский `ingest` может индексировать внешние локальные пути, которые HTTP намеренно не выдаёт. `--team ... worker run` запрещён: командные ingest/remove задания выполняет `service work`.